REST Principles

resources, verbs, HATEOAS and "cancel order"

Watch the 4-min video

In one line: REST means resources named by URIs, standard HTTP methods as the verbs, and a set of constraints (stateless, cacheable, layered, uniform interface); "JSON over HTTP" alone is not REST.

1 Resources are nouns, methods are verbs

Almost every company calls its API RESTful. Many just mean "JSON over HTTP". Interviewers know the difference, so learn what REST actually requires.

REST is built around **resources**: the nouns of your domain, such as users, orders and products. Each resource has a **URI**, like `/users/42`. You do not put verbs in the path (`/getUser`, `/createOrder`). The **HTTP method is the verb**.

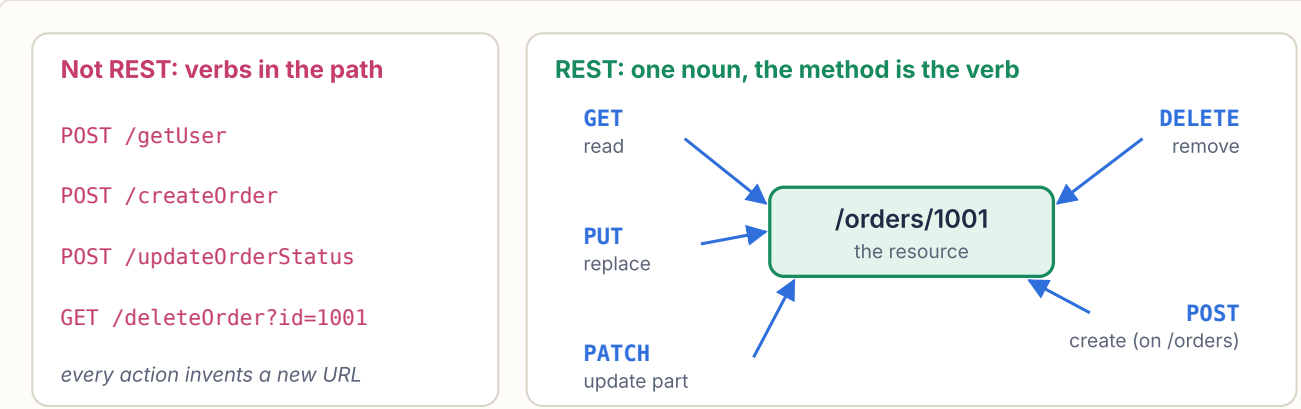


Figure 1. Left: an RPC-style API invents a URL per action. Right: one resource URI, and the standard methods say what to do with it.

Method	Meaning	Safe?	Idempotent?
`GET`	Read a resource or a collection	Yes	Yes
`POST`	Create a new item in a collection (or run an action)	No	No, unless you add a key
`PUT`	Replace the whole resource	No	Yes
`PATCH`	Update some fields	No	Not guaranteed
`DELETE`	Remove the resource	No	Yes

Safe means it changes nothing on the server. **Idempotent** means sending it twice has the same effect as sending it once. This matters because networks retry.

2 The architectural constraints

REST (from Roy Fielding's 2000 dissertation) is defined by constraints, not by JSON or URLs:

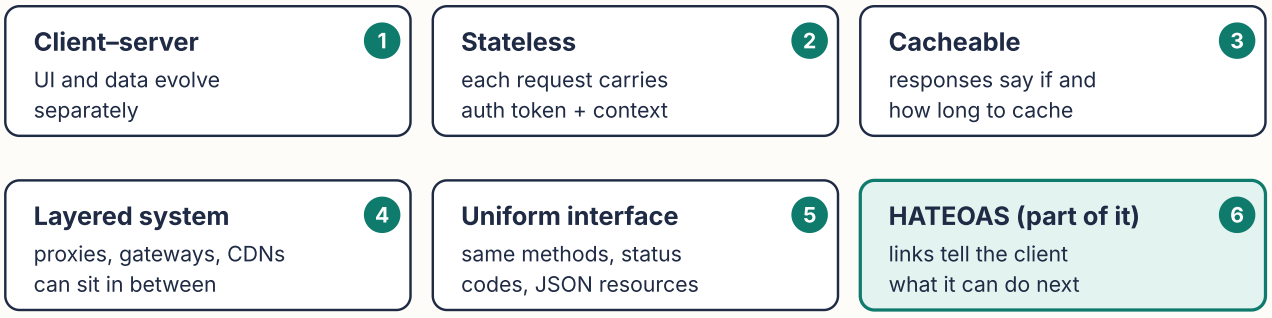


Figure 2. The REST constraints. Most "RESTful" APIs follow 1–5; the hypermedia part of the uniform interface (6) is the one usually skipped.

Two of them carry most of the weight in system design. **Stateless**: no session lives in server memory, so any server behind the load balancer can answer any request. **Cacheable**: headers like `Cache-Control` and `ETag` let browsers, CDNs and proxies (the **layered** part) serve repeat reads without touching your servers.

3 HATEOAS: the constraint everyone skips

HATEOAS stands for *hypermedia as the engine of application state*. Responses include **links to what the client can do next**. A pending order carries a cancel link; a shipped order does not. The client follows links instead of building URLs itself.

```
{ "id": 1001,
  "status":    "pending",
  "_links": {
    "self": "/orders/1001"
    "pay": "/orders/1001/payment"
    "cancel": "/orders/1001:cancel"
} }
```

```
{ "id": 1002,
  "status":    "shipped",
  "_links": {
    "self": "/orders/1002"
    "track": "/orders/1002/tracking"
} }
```

no cancel link: too late to cancel

Figure 3. The same resource type, two states. The server, not the client, decides which actions are available right now.

It is elegant, but most clients hardcode URLs anyway. So few public APIs go all the way, and that is fine for most teams. In an interview, say that you know it is part of the full definition, and explain why you would or would not use it.

4 Designing the e-commerce endpoints

Endpoint	What it does
`GET /users/42/orders`	List one user's orders
`POST /users/42/orders`	Create an order for that user (returns `201 Created` + `Location`)
`GET /orders/1001`	Fetch an order directly: orders deserve their own top-level path
`GET /orders?status=shipped`	Filter a collection with query parameters

The rules: **plural nouns**, **lowercase with hyphens** (`/shipping-addresses`), **nesting at most one level**, and **query parameters for filtering**.

5 How to model “cancel order”

Cancel is not plain CRUD, so it is the classic interview question. Three options:

Option	Trade-off
`PATCH /orders/1001` body `{"status":"cancelled"}`	Simple, but refunds and restocking hide behind a field update, and clients may try to set any status.
`POST /orders/1001/cancellation`	Treat the action as a resource: a cancellation is created, and can record who and why.
`POST /orders/1001:cancel`	A **custom method**, the style in Google's API design guidelines. Clear intent, one POST.

Whichever you pick, use status codes properly: **200** with the updated order, or **202 Accepted** if the refund runs asynchronously; **409 Conflict** if it already shipped; **404** if the order does not exist. And make it **idempotent**: cancelling twice must not refund twice.

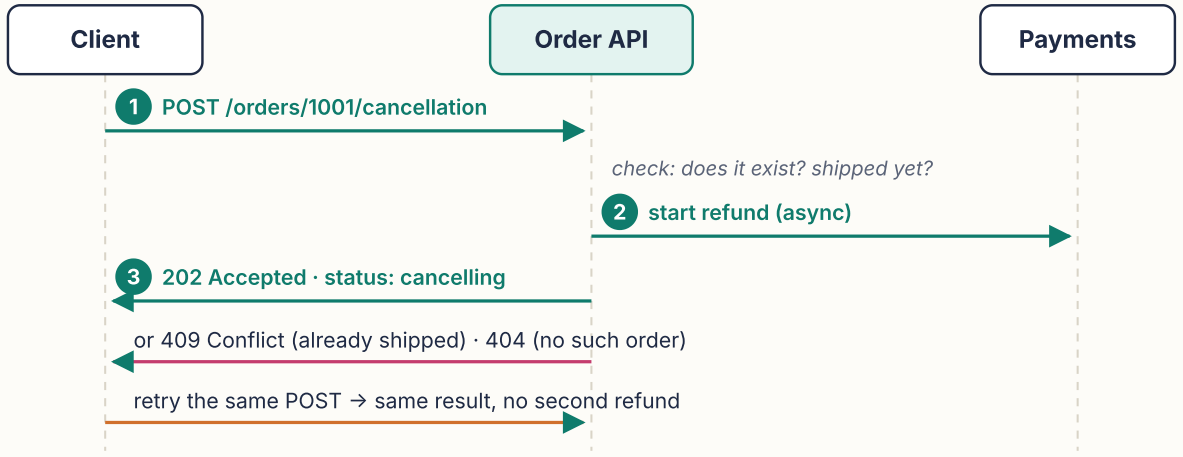


Figure 4. Cancel as a sub-resource. The API checks the business rules, starts the refund, and answers 202; a retry finds the existing cancellation and returns the same result.

WORKED EXAMPLE

```
POST /orders/1001/cancellation
Authorization: Bearer eyJ...
Idempotency-Key: 7f3c-91ab
{ "reason": "ordered by mistake" }

→ 202 Accepted
{ "order_id": 1001, "status": "cancelling", "refund": "pending" }
```

The client's mobile network drops before the reply arrives, so it retries with the same `Idempotency-Key`. The server sees the key (or the existing cancellation for order 1001) and returns the same 202. One refund, not two. If the order had shipped an hour earlier, the first call returns `409 Conflict` with a message such as “order already shipped; request a return instead”.

COMMON MISTAKES

- Verbs in paths: `/getOrders`, `/deleteUser?id=5`.
- Returning `200 OK` with `{"error": ...}` in the body instead of a real 4xx/5xx code.
- Storing session state in server memory, which breaks statelessness and load balancing.
- Making cancel a status PATCH, so a retry or a bad client can trigger a second refund or set "shipped".

6 Key terms

Resource: a noun of the domain, named by a URI

Uniform interface: same methods, codes, representations

Custom method: POST /orders/1001:cancel

URI: the resource's address, e.g. /orders/1001

HATEOAS: responses link to next actions

409 Conflict: request clashes with current state

Stateless: each request carries all its context

Idempotent: repeat = same effect as once

202 Accepted: accepted, finishing asynchronously

7 Interview questions

Q1. What makes an API RESTful? BASIC

Resources identified by URIs; standard HTTP methods as the verbs; stateless requests; cacheable responses; a layered system; and a uniform interface, with hypermedia links (HATEOAS) as the full ideal. JSON over HTTP with verb URLs is not REST.

Q2. Which HTTP methods are idempotent? BASIC

GET, PUT and DELETE (and HEAD, OPTIONS). Deleting /orders/1001 twice leaves it deleted. POST is not: two POSTs to /orders make two orders, unless you add an idempotency key. PATCH is not guaranteed either.

Q3. What does "stateless" mean, and why does it help scaling? BASIC

Each request carries its own context (auth token, IDs), so no server needs memory of earlier requests. Any of, say, 20 servers behind the load balancer can handle any request, and you can add a 21st without moving sessions.

Q4. How would you model "cancel order" in REST? SCENARIO

As a sub-resource, POST /orders/1001/cancellation, or a custom method, POST /orders/1001:cancel, not a hidden status PATCH. Return 200 (or 202 if the refund is async), 409 if it already shipped, 404 if missing, and make it idempotent.

Q5. What is HATEOAS, and why do so few APIs use it? TRICKY

Responses carry links to valid next actions, so a pending order shows a cancel link and a shipped one does not. Clients in practice hardcode URLs and SDKs, so the benefit rarely pays for the extra payload and tooling; most teams stop at the other constraints.

Q6. A client cancels, the response times out, and it retries. What must happen? TRICKY

The second call must not issue a second refund. Detect the existing cancellation (or the same Idempotency-Key) and return the same result, 202 or 200. Only a different, conflicting request should get 409.

Next: SD-C-002 · GraphQL for System Design backendroadmap.com

GraphQL for System Design

resolvers, N+1, DataLoader and query limits

Watch the 4-min video

In one line: GraphQL lets each client ask a typed schema for exactly the data a screen needs in one request; the server pays for that flexibility with N+1 risk (fix: DataLoader), expensive-query risk (fix: depth and cost limits) and harder caching.

1 The problem: over-fetching and under-fetching

A mobile product page needs the product, its price, three reviews, the seller and related items. With a typical REST API that is **five round trips** over a slow cellular network. That is **under-fetching**: one call does not return enough. And each response carries fields the screen never shows: **over-fetching**.

GraphQL lets the client ask for exactly what it needs, in one request.

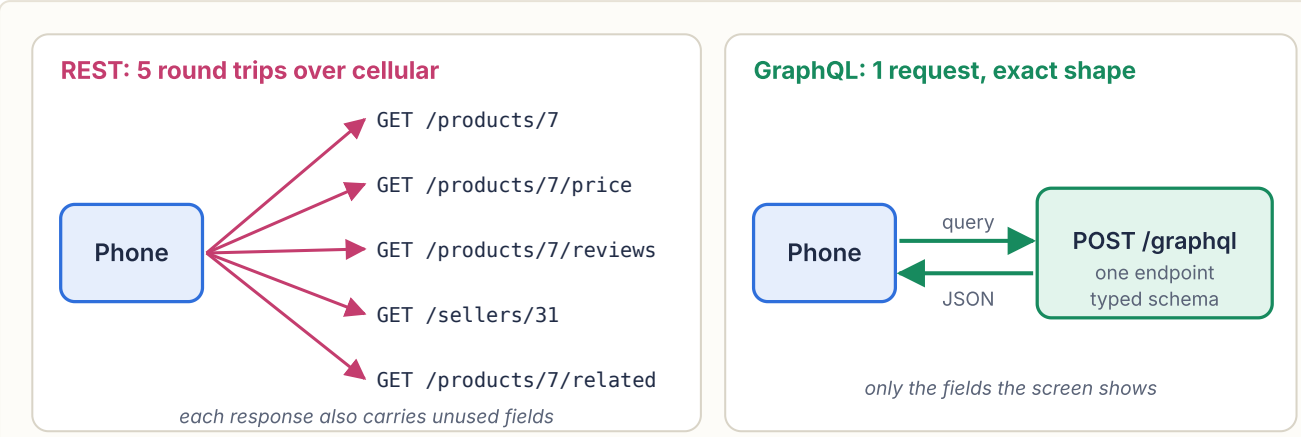


Figure 1. At ~100 ms per cellular round trip, five calls made one after another cost about half a second before any rendering; one GraphQL request costs one round trip.

2 Schema, operations and one endpoint

GraphQL starts with a **typed schema**: types such as `Product`, `Review` and `Seller`, with their fields and relationships.

```
type Product { id: ID!  name: String!  price: Float!
               reviews(first: Int): [Review!]!  seller: Seller!  related: [Product!]! }
type Review  { rating: Int!  text: String }
type Seller  { name: String! }
type Query   { product(id: ID!): Product }
```

- **Queries** read data.
- **Mutations** change it (e.g. `addReview`).
- **Subscriptions** push updates to the client, usually over WebSockets.

Everything goes through a **single endpoint**, typically `POST /graphql`. The schema is the contract, and tools validate every query against it.

3 The product page as one query

Query (client writes it)

```
query ProductPage {
  product(id: "7") {
    name
    price
    reviews(first: 3) {
      rating
      text
    }
    seller { name }
    related { name }
  }
}
```

Response (same shape)

```
{ "data": {
  "product": {
    "name": "Trail shoe",
    "price": 89.0,
    "reviews": [
      {"rating":5,"text":".."},
      ... 2 more
    ],
    "seller": {"name":"Acme"},
    "related": [{"name":..}]
  }
} }
```

Figure 2. The client names the product by ID and lists the fields it wants: name, price, the first three reviews with rating and text, the seller's name, related product names. The JSON has exactly that shape.

No over-fetching of unused fields, and no under-fetching that forces extra calls.

4 Resolvers and the N+1 problem

On the server, **every field has a resolver**: a function that fetches its data. That is flexible, but it hides a trap. Fetch a list of 50 products, and the `reviews` resolver runs once per product: **1 query for the list + 50 for the reviews = 51 database queries** for a single request.

The fix is **DataLoader**. Instead of querying immediately, each resolver asks a loader for a key. The loader collects all keys requested in the same tick (one turn of the event loop) and runs **one batched query**: reviews where `product_id IN (…)`. It also **caches within the request**, so the same seller requested ten times is loaded once.

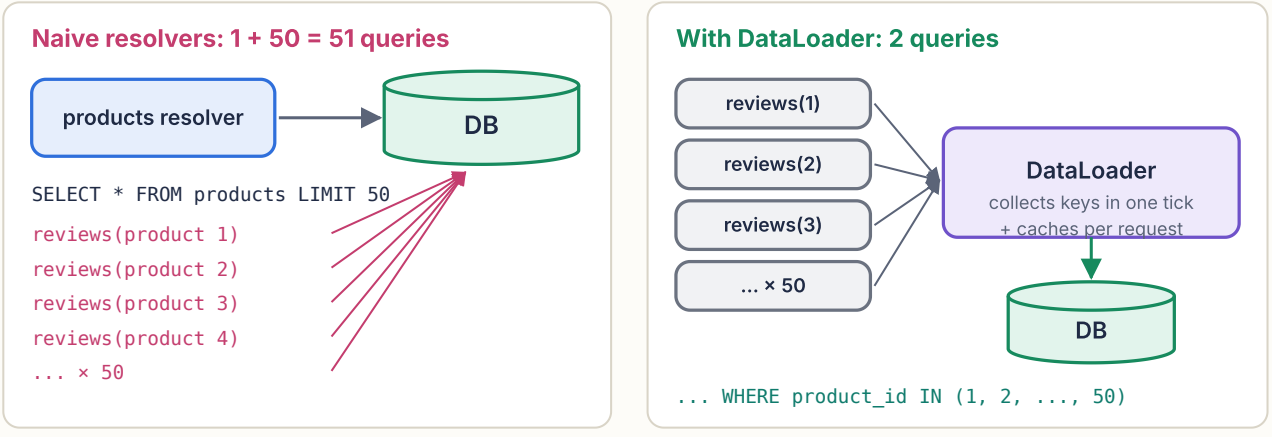


Figure 3. Left: one query per parent row. Right: the loader batches all 50 keys into one IN query. 51 queries become 2.

WORKED EXAMPLE

A list page shows 50 products, each with reviews and its seller. Naive: 1 (products) + 50 (reviews) + 50 (sellers) = **101 queries**. At 2 ms each that is about 200 ms of database time, and the database works 50× harder than it needs to.

With DataLoader: 1 (products) + 1 (reviews `IN` 50 ids) + 1 (sellers `IN` the distinct seller ids, say 12) = **3 queries**, about 6–10 ms.

5 Protecting the server from expensive queries

Clients write the queries, so a careless or malicious one can nest deeply: products → reviews → each reviewer's orders → each order's products → and so on. Each level multiplies the work. Defend in layers:

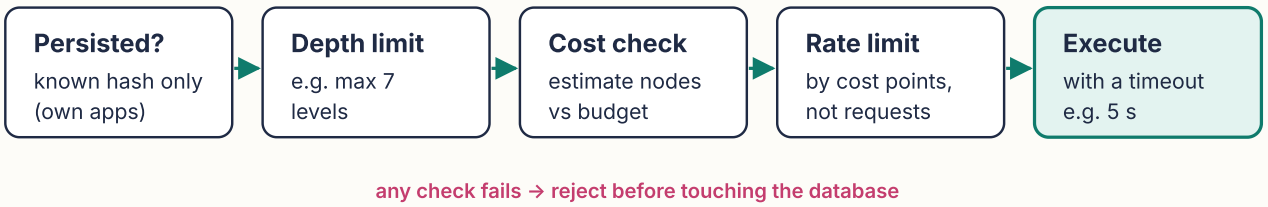


Figure 4. Checks run on the parsed query, before any resolver runs.

- **Depth limit**: reject queries nested beyond a fixed depth.
- **Query cost analysis**: estimate how many nodes a query can return, using list sizes. `products(first: 50) { reviews(first: 10) { author { orders(first: 20) } } }` is 50 × 10 × 20 = **10,000** nodes; reject anything above the budget.
- **Timeouts** on execution, and **rate limits based on cost**, not request count: one huge query should use as much budget as many small ones.
- **Persisted queries** for your own apps: at build time each query is registered, and clients send only its hash. The server runs only known queries.

6 Caching, and when to choose GraphQL

Caching is harder than with REST. Everything is a POST to one URL, so CDNs and browser HTTP caches cannot help by default. Two things fill the gap: **persisted queries sent over GET** (the hash and variables go in the URL, so a CDN can cache them), and **normalized client caches** such as Apollo's, which store each object once by type and ID.

GraphQL fits well	REST is usually simpler
Many different clients (especially mobile) needing flexible, aggregated views of related data	Simple CRUD services
A backend-for-frontend that aggregates several services per screen	Public, cache-friendly APIs served through a CDN
Fast-changing UIs that would otherwise need new endpoints each release	File uploads and downloads

COMMON MISTAKES

- Shipping resolvers without DataLoader, then wondering why one request makes hundreds of queries.
- Exposing a public GraphQL endpoint with no depth or cost limits.
- Rate limiting by request count: one request can be 10,000× more expensive than another.
- Expecting the CDN to cache `POST /graphql` responses.
- Choosing GraphQL for a simple CRUD service with one client, and taking on its complexity for nothing.

7 Key terms

Schema: typed contract of types and fields

Resolver: function that fetches one field

Over-fetching: fields you receive but don't use

Query / mutation: read / change data

N+1 problem: 1 list query + 1 query per item

Query cost: estimated nodes a query may touch

Subscription: server push, usually over WebSockets

DataLoader: batches and caches keys per request

Persisted query: pre-registered query sent by hash

8 Interview questions

Q1. When is GraphQL better than REST? BASIC

When diverse clients, like mobile apps and a backend-for-frontend, need flexible, aggregated data in one round trip without over- or under-fetching. Five REST calls at ~100 ms on cellular become one request.

Q2. What is the N+1 problem in GraphQL? BASIC

A list resolver runs one query, then a child resolver runs once per item. 50 products with reviews = 1 + 50 = 51 queries. It comes from resolving each field independently.

Q3. How does DataLoader fix N+1? BASIC

Resolvers ask the loader for keys instead of querying. It gathers all keys requested in the same tick and runs one batched query (WHERE product_id IN (...)), and caches results for the rest of that request. 51 queries become 2.

Q4. How do you prevent expensive GraphQL queries? SCENARIO

Depth limits; cost analysis that multiplies list sizes (50 × 10 × 20 = 10,000 nodes) and rejects queries over budget; execution timeouts; cost-based rate limiting; persisted queries for first-party apps; and DataLoader so legitimate queries stay cheap.

Q5. Your product page API is slow on mobile. Would you add GraphQL? SCENARIO

If the page needs data from several resources and each call is a cellular round trip, a GraphQL BFF that aggregates them on the server side helps: the slow hops move to the fast data-centre network. If the slowness is one slow database query, GraphQL will not fix it.

Q6. Why is caching GraphQL harder than caching REST, and what helps? TRICKY

All requests are POST to one URL, so HTTP caches and CDNs cannot key on the URL. Use persisted queries over GET so the URL identifies the query, and a normalized client cache (e.g. Apollo) keyed by type + ID.

Q7. A single client sends 10 requests per minute but your database is overloaded. How can that be? TRICKY

Each request can be a deeply nested query touching thousands of rows. Rate limits by request count miss that. Score each query by cost and limit points per client per minute, and set depth and node caps.

Next: SD-C-003 · gRPC & Protocol Buffers backendroadmap.com

gRPC & Protocol Buffers

.proto contracts, HTTP/2, deadlines and safe evolution

Watch the 4-min video

In one line: gRPC lets services call each other through code generated from one `.proto` contract, sending compact binary protobuf over HTTP/2 with deadlines and standard status codes; field numbers, not names, are the identity you must never break.

Your orders service is written in Python. The inventory service is written in Go. They call each other **thousands of times per second**. Hand-written JSON clients drift apart (one side renames a field, the other breaks at runtime), and every call pays for verbose text. gRPC with protocol buffers fixes both problems.

1 One contract: the .proto file

You describe the API once, in a `.proto` file, using an **interface definition language** (IDL). It defines a **service** with RPC methods, and **messages** with typed fields. Each field has a **field number**.

```
syntax = "proto3";
package inventory.v1;

service Inventory {
  rpc CheckStock(CheckStockRequest) returns (CheckStockReply);
}
message CheckStockRequest { string sku = 1;  int32 quantity = 2; }
message CheckStockReply   { bool available = 1;  int32 in_stock = 2; }
```

The **protoc** compiler generates client and server code for Python, Go, Java and more. Both sides share one contract, **checked at build time**, not discovered in production.

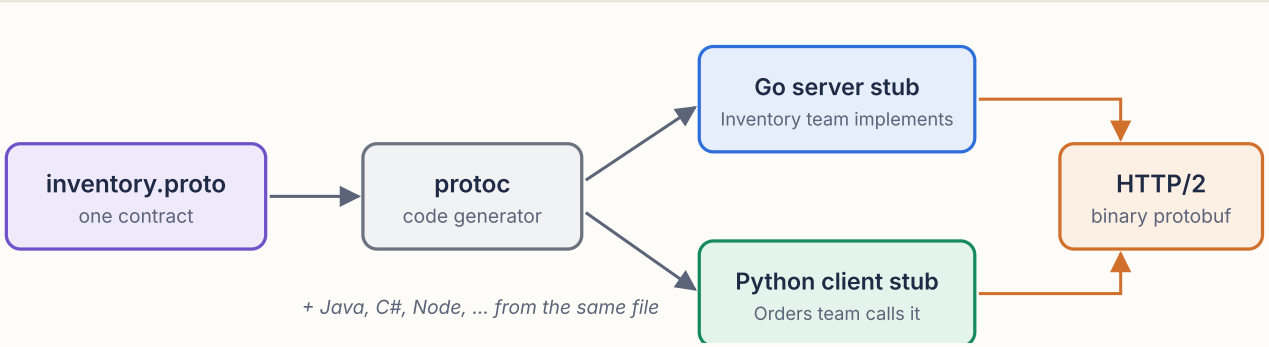


Figure 1. One .proto file produces typed code for every language. Neither team writes serialization code.

2 Compact on the wire

Protobuf is a **binary** format. JSON repeats field names in every message; protobuf tags each field with its small number. Integers use **variable-length encoding** (varint): 3 takes one byte, 300 takes two. Messages are often several times smaller than JSON, and much faster to parse because there is no text to scan.



Figure 2. The same request. Each protobuf tag byte packs the field number and wire type: 0x0A = field 1, length-delimited; 0x10 = field 2, varint.

3 HTTP/2, call types, deadlines and status codes

gRPC runs over **HTTP/2**, so many calls share one TCP connection as **multiplexed streams**: no connection per call, and one slow call does not block the others. There are four call types:

Type	Shape	Example
Unary	one request → one response	`CheckStock`
Server streaming	one request → stream of responses	watch stock changes for a SKU
Client streaming	stream of requests → one response	upload a batch of stock counts
Bidirectional	both sides stream at once	live warehouse sync

Every call should carry a deadline: the latest time the caller will wait. It **propagates** across services, so downstream work stops once nobody is waiting for the answer. Errors use a standard set of **status codes**, such as `NOT_FOUND`, `DEADLINE_EXCEEDED` and `UNAVAILABLE` (usually safe to retry with backoff).

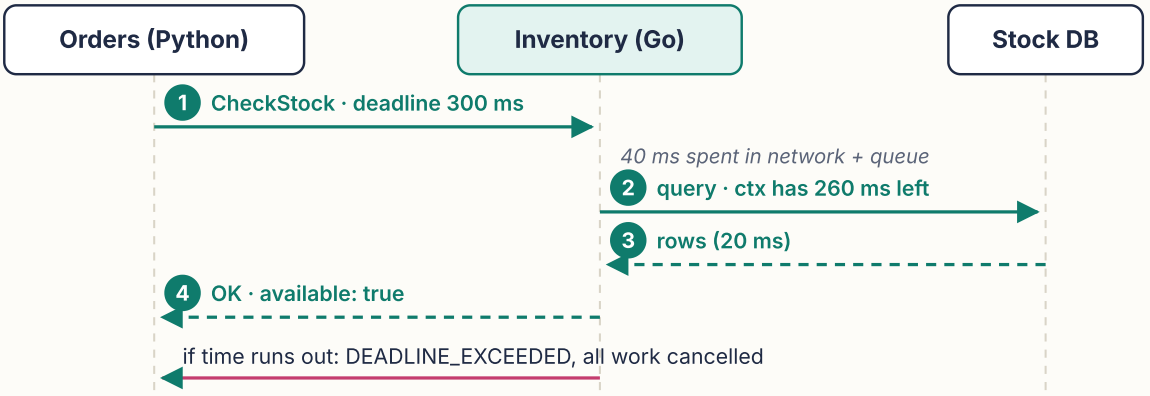


Figure 3. The 300 ms budget travels with the call. Inventory knows it has 260 ms left and passes that on; if time runs out, every hop stops instead of doing useless work.

WORKED EXAMPLE: CHECKSTOCK END TO END

The Go team implements the generated server interface; the Python orders service calls the generated stub with a 300 ms deadline. Neither side writes serialization code.

```
# Python (orders)
stub = inventory_pb2_grpc.InventoryStub(channel)
reply = stub.CheckStock(
    inventory_pb2.CheckStockRequest(sku="A-1001", quantity=3), timeout=0.3)
// Go (inventory)
func (s *server) CheckStock(ctx context.Context, r *pb.CheckStockRequest)
(*pb.CheckStockReply, error) {
    n, err := s.db.Stock(ctx, r.Sku)   // ctx carries the remaining deadline
    return &pb.CheckStockReply{Available: n >= r.Quantity, InStock: n}, err
}
```

4 Evolving a protobuf schema safely

Field numbers are the real identity on the wire, not names. A renamed field still works; a renumbered one silently breaks every old client.

- **Add** fields with **new numbers**. Safe: old readers simply skip unknown fields.
- **Never change** a field's number or type.
- **When you remove a field, reserve its number** (and name), so nobody reuses it with a different meaning.
- **Missing fields read as default values** (0, empty string, false). Design for that: a missing `quantity` looks like 0. Use `optional` or a wrapper when “not set” must differ from zero.

```
message CheckStockRequest {
  reserved 3;  reserved "warehouse";   // removed in v1.4, never reuse
  string sku = 1;
  int32 quantity = 2;
  string region = 4;                  // new, old servers ignore it
}
```

5 gRPC inside, REST outside

Browsers cannot fully control HTTP/2 frames and trailers, which gRPC relies on. Web apps need **gRPC-Web**, with a proxy such as **Envoy** translating. That is one reason for the common pattern: **gRPC internally, REST externally**. Inside, you control both sides and want speed, strict contracts, streaming and deadlines. Outside, partners expect plain HTTP and JSON that works with curl, browsers and every language.

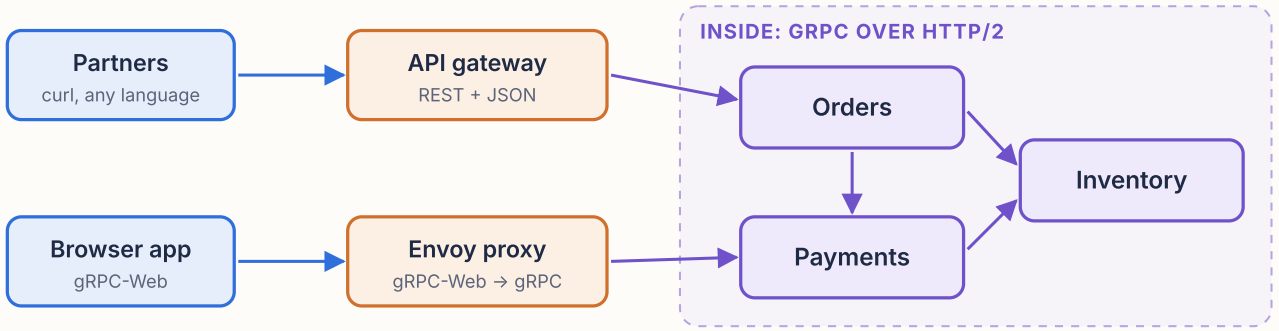


Figure 4. External traffic enters as REST or gRPC-Web; everything behind the edge speaks gRPC.

COMMON MISTAKES

- Calls without deadlines: one slow dependency ties up threads across the whole call chain.
- Reusing or renumbering a field number, so old clients decode garbage without any error.
- Treating 0 or "" as “the client sent this value” when it may simply be missing.
- Opening a new channel per call instead of reusing one, losing HTTP/2 multiplexing.

6 Key terms

Protobuf: binary format defined in .proto

Deadline: absolute time limit, propagated

protoc: compiler that generates stubs

reserved: blocks reuse of removed numbers

Field number: a field's identity on the wire

gRPC-Web: browser variant via a proxy

7 Interview questions

Q1. Why is protobuf smaller and faster than JSON? BASIC

It sends a 1-byte tag (field number + type) instead of the field name, and varint integers instead of digits as text. {"sku":"A-1001","quantity":3} is 29 bytes in JSON and 10 in protobuf, and parsing needs no text scanning.

Q2. Why use gRPC internally and REST externally? SCENARIO

Inside you control both ends: compact binary, HTTP/2 multiplexing, generated clients, streaming and deadlines pay off at thousands of calls per second. Outside, REST with JSON is universal, human-readable, cache-friendly and works in any browser and with curl.

Q3. How do you evolve a protobuf schema safely? SCENARIO

Add fields with new numbers; never change or reuse existing numbers or types; mark deleted numbers (and names) as reserved; and remember missing fields read as defaults, so design for 0 and "".

Q4. Why should every gRPC call have a deadline, and why must it propagate? TRICKY

Without one, a stuck dependency holds the caller forever and threads pile up. If Orders gives up after 300 ms but Inventory keeps querying for 2 s, that work is wasted load. Propagation passes the remaining 260 ms on, so every hop stops together with DEADLINE_EXCEEDED.

Q5. Someone renamed field quantity to qty and kept number 2. Did anything break? TRICKY

Not on the wire: the bytes carry only number 2, so old and new binaries still talk; only code (and JSON mappings) using the name change. Changing the number to 5 would silently break every old client.

Next: SD-C-004 · REST vs GraphQL vs gRPC backendroadmap.com

REST vs GraphQL vs gRPC

how to choose an API style, and defend it

Watch the 3-min video

In one line: there is rarely a single winner: use REST for public and partner APIs, GraphQL as a backend-for-frontend when clients need flexible, aggregated views, and gRPC for fast, strongly typed calls between your own services. Most large systems run all three.

1 The real interview question

You have learned REST, GraphQL and gRPC. In an interview, the real question is: **which one would you pick here, and why?** "It depends" is only a good answer if you say what it depends on. Compare on six criteria: **performance and payload size**, **caching**, **browser and tooling support**, **coupling** between client and server, **streaming**, and, often forgotten, **your team's skills**.

	REST	GraphQL	gRPC
Performance & payload	●○○ JSON text	●●○ exact fields	●●● binary, HTTP/2
HTTP caching	●●● built in	●○○ POST: hard	●○○ custom
Browser & tooling	●●● everywhere	●●○ good clients	●○○ needs gRPC-Web
Loose coupling	●●● OpenAPI, loose	●●○ shared graph	●○○ shared .proto
Streaming	●○○ SSE / WS aside	●●○ subscriptions	●●● first-class
Team familiarity	●●● everyone knows	●●○ some	●○○ fewer

more dots = stronger on that criterion (a rule of thumb, not a benchmark)

Figure 1. A quick scorecard. No column wins every row, which is why big systems mix styles.

2 Strengths and costs of each style

REST: the most universal

Every language, tool, proxy and CDN understands it, and **HTTP caching works out of the box** (`Cache-Control`, `ETag`). Payloads are JSON text, so they are larger, and clients may need several calls for one screen. Contracts are usually documented with **OpenAPI**, and coupling stays loose: a client uses only the endpoints and fields it needs.

GraphQL: flexible data for many clients

It shines when many clients need different shapes of related data. One request fetches exactly what a screen needs. But HTTP caching is harder, the server must **guard against expensive queries** (depth and cost limits), and the schema becomes a **shared graph** that needs careful governance across teams.

gRPC: the fastest

Compact binary messages, HTTP/2 multiplexing, generated clients, **built-in deadlines** and first-class streaming. But browsers need gRPC-Web, payloads are not human-readable, and both sides must share the `.proto` files. So it fits best **where you control both ends**.

3 A common architecture uses all three

External partners call a **public REST API** through an API gateway. The mobile and web apps call a **GraphQL backend-for-frontend** (BFF) that aggregates data for each screen. Behind both, the **internal microservices talk over gRPC**.

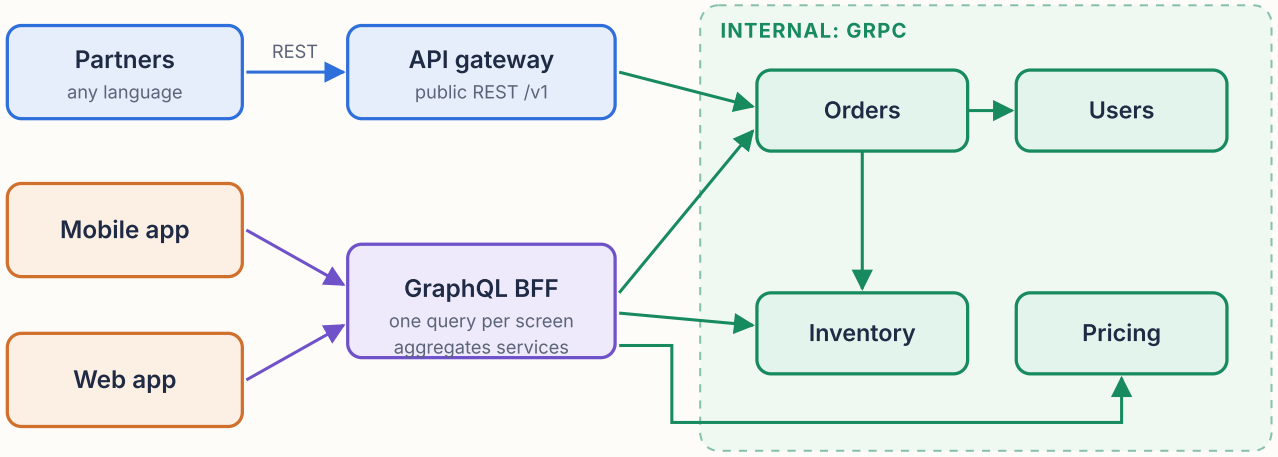


Figure 2. Each style sits where its strengths matter: REST at the public edge, GraphQL for first-party UIs, gRPC between services.

WORKED EXAMPLE: ONE PRODUCT PAGE, THREE STYLES

The mobile app sends one GraphQL query to the BFF over cellular (~100 ms round trip). The BFF fans out four gRPC calls inside the data centre (~1–3 ms each, in parallel), for product, price, stock and reviews, each with a 200 ms deadline. It assembles one JSON response shaped for the screen.

Meanwhile a partner's nightly job calls `GET /v1/products?updated_after=2026-11-01` on the public REST API, cached at the gateway for 60 s. Same data, three consumers, three styles, and each one is the simplest thing that works for its caller.

4 Choosing for a public partner API

Usually **REST**. Partners use every language and toolchain, and REST with JSON needs nothing special: **curl, Postman and any HTTP library just work**. It is cache-friendly, easy to document with OpenAPI, easy to version (`/v1`) and easy to secure with standard gateways (API keys, OAuth, rate limits).

- **GraphQL** can work for public APIs too (GitHub offers one), but you take on **query cost control** for clients you do not know.
- **gRPC** is a poor fit for unknown external clients: they need your `.proto` files, a gRPC toolchain, and cannot just try a call in a browser.

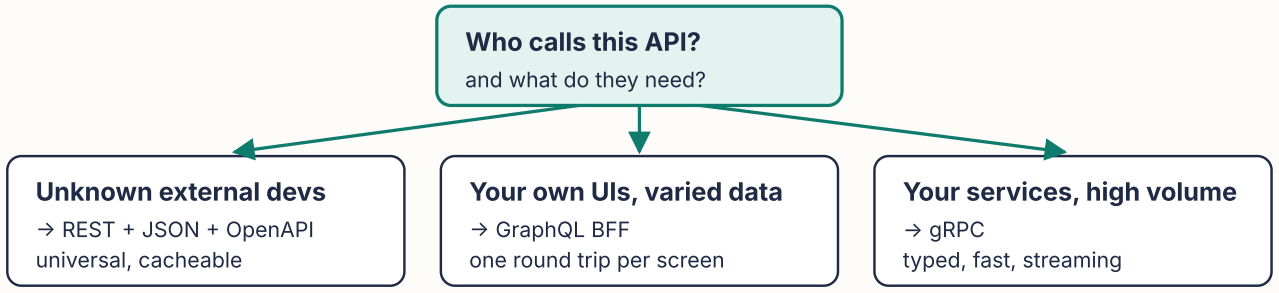


Figure 3. Start from the caller, not the technology.

COMMON MISTAKES

- Declaring one style “best” instead of matching it to the caller.
- Choosing gRPC for a public API, then building SDKs and proxies for every partner.
- Ignoring team skills: a style nobody on the team has run in production is a risk, not a feature.

5 Key terms

REST: resources + HTTP methods, usually JSON

BFF: backend-for-frontend: API per client type

GraphQL: client-shaped queries on a typed schema

OpenAPI: standard REST contract format

gRPC: binary RPC over HTTP/2 from .proto

API gateway: edge: auth, rate limits, routing

6 Interview questions

Q1. Which API style would you choose for a public partner API, and why? BASIC

REST. It is universal, works from any language with curl or any HTTP library, is cacheable, easy to document with OpenAPI, easy to version and easy to secure with standard gateways.

Q2. Compare REST, GraphQL and gRPC in one sentence each. BASIC

REST: universal and cacheable, but can over- or under-fetch. GraphQL: exact data per screen in one request, but harder caching and query-cost risk. gRPC: fastest and strongly typed with streaming, but needs shared .proto files and is awkward in browsers.

Q3. Design the API layer for a shop with partners, a mobile app and 30 microservices. SCENARIO

Public REST behind an API gateway for partners; a GraphQL BFF that serves each mobile or web screen in one round trip; gRPC with deadlines between the 30 services.

Q4. Two internal services exchange 20,000 small messages per second. Which style? SCENARIO

gRPC: binary payloads several times smaller than JSON, HTTP/2 multiplexing over a few connections, generated typed clients, deadlines, and streaming if needed.

Q5. Why not use GraphQL everywhere, including between services? TRICKY

Service-to-service calls have fixed shapes, so GraphQL's flexibility buys little, while you still pay for resolvers, query parsing and cost limits. gRPC gives stricter contracts and lower latency there.

Q6. When could GraphQL be right for a public API? TRICKY

When external developers build varied UIs over a rich, connected data model, as with GitHub's GraphQL API. You must then enforce cost-based rate limits and node limits, since you cannot trust unknown queries.

Next: SD-C-005 · API Resource Modeling & URI Design backendroadmap.com

API Resource Modeling & URI Design

nesting, actions and bulk operations

Watch the 4-min video

In one line: a good API is predictable: plural nouns and IDs, one level of nesting for true ownership, query parameters for filters, explicit POST actions for non-CRUD operations, and capped bulk endpoints with per-item results.

1 Naming rules

If a developer can **guess your endpoint before reading the docs**, you have modeled it well. Start with four rules:

- **Plural nouns** for collections, an **ID** for each item: `/orders` and `/orders/1001`.
- **Lowercase with hyphens** in paths: `/shipping-addresses`, not `/ShippingAddresses` or `/shipping_addresses`.
- **One casing style for JSON fields**, `snake_case` or `camelCase`, used everywhere.
- **Version up front**: `/v1/...`.

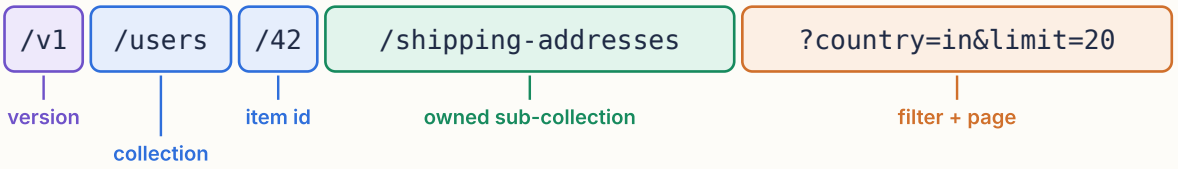


Figure 1. Every part of the URI has one job. Reading it left to right tells you the version, the resource, the owner and the filter.

2 Nested resources: one level, true ownership

Nest only when the child truly belongs to the parent, and keep it to one level. `/users/42/addresses` makes sense: an address belongs to one user, and the path **scopes authorization naturally** (check that the caller is user 42, then everything below is theirs).

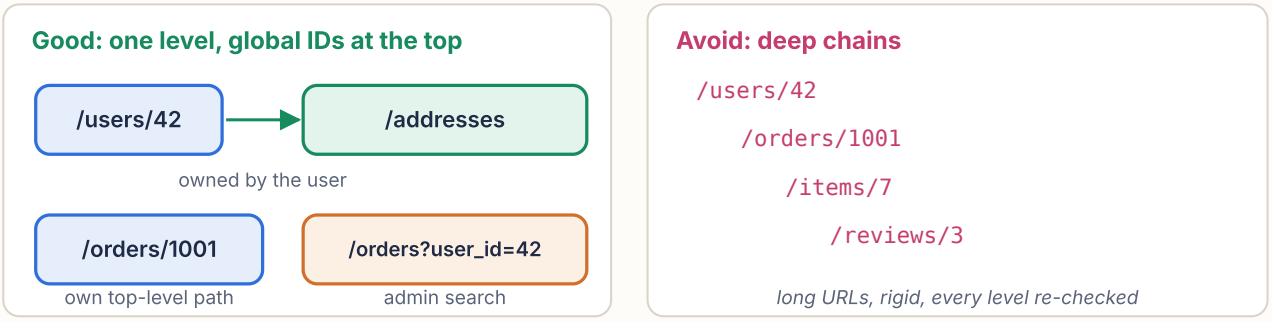


Figure 2. Left: nest owned children one level deep and promote globally identified resources to the top. Right: a chain that forces clients to know four IDs to reach one review.

Avoid deep chains such as `/users/42/orders/1001/items/7/reviews/3`. Once a resource has its own **global ID**, give it a **top-level path**: `/orders/1001`. Order 1001 is unique on its own, so the user ID in the

path adds nothing but length and another check.

3 Sub-resource or query parameter?

Use	Example	Meaning
Sub-resource	`/users/42/orders`	Ownership: the orders that belong to user 42
Query parameters	`/orders?status=shipped&created_after=2026-10-01`	Filter a collection by attributes
Both	`/orders?user_id=42`	Handy for admin tools that search across users

Rule of thumb: if removing the parent should make the child meaningless, it is a sub-resource. If it is just an attribute you search by, it is a query parameter.

4 Non-CRUD actions: cancel, refund, resend

Do not force actions into a fake field update. Two good options:

- **Custom method**: `POST /v1/orders/1001:cancel`, the style used in Google's APIs.
- **Action as a resource**: `POST /v1/orders/1001/cancellations`. It also **keeps a record** of who cancelled and why, which you can later `GET`.

Either way it is a **POST**, it must be **idempotent** (a retried cancel does not refund twice), and it returns **409 Conflict** if the order can no longer be cancelled.

Why not just PATCH the status?

Because a cancel is not a simple field change. It **triggers refunds, restocks inventory and sends emails**. A PATCH that sets `status` to `cancelled` hides all that behavior, makes validation awkward, and invites clients to set any status they like, such as `shipped`. Explicit actions make **intent and permissions clear**: "may cancel" is a different permission from "may edit the delivery note".

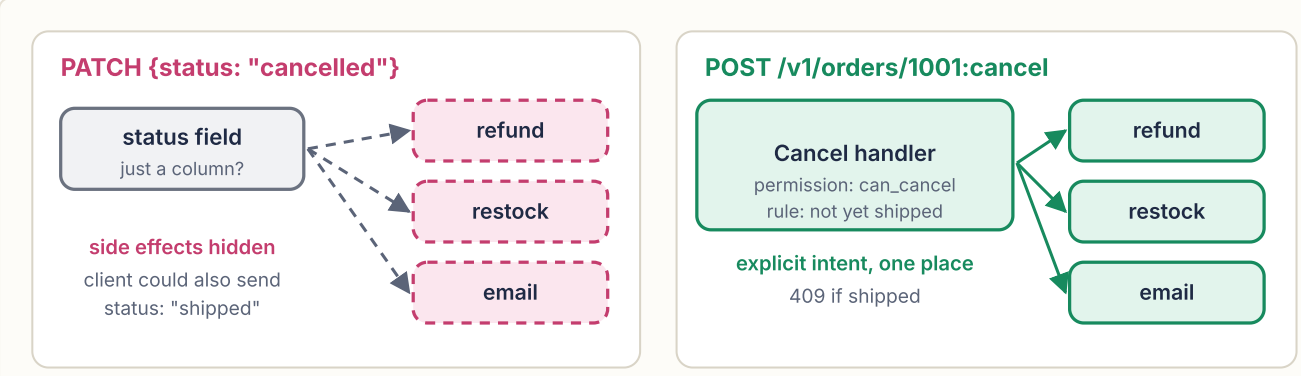


Figure 3. The same business operation. On the left the side effects hang off a field write; on the right one handler owns the rules, the permission and the effects.

5 Bulk operations

Creating a thousand items one call at a time is slow: 1,000 calls × 50 ms is 50 seconds of round trips. Offer a **bulk endpoint**, such as `POST /v1/orders:batchCreate`, with a **cap on items per request** (say 500).

- **Return a result for each item**, because some may succeed while others fail.

- For **very large jobs**, accept with **202 Accepted** and let clients **poll a job resource** for progress.

WORKED EXAMPLE: PARTIAL FAILURE

A client sends 3 orders in one batch. The second has an unknown SKU. The response reports each item, in request order:

```
POST /v1/orders:batchCreate        → 200 OK
{ "results": [
  { "index": 0, "status": 201, "id": "1001" },
  { "index": 1, "status": 422, "error": "unknown sku ZX-9" },
  { "index": 2, "status": 201, "id": "1002" } ] }
```

The client retries only index 1 after fixing the SKU. A batch of 2,000 items exceeds the cap of 500, so it gets `400` with a clear message, and the client splits it into 4 requests.

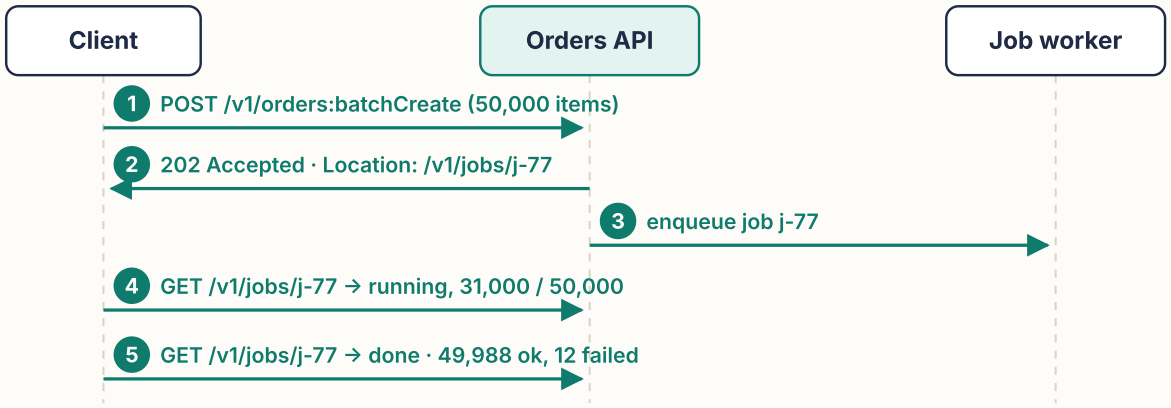


Figure 4. Large jobs run asynchronously. The 202 response points to a job resource the client polls; the final job state lists the failed items.

COMMON MISTAKES

- Mixing styles: `/Orders`, `/order_items` and `/shippingAddresses` in one API.
- Nesting three or four levels deep, so clients must know every parent ID.
- Modeling cancel as `PATCH status`, then validating every possible status transition by hand.
- A bulk endpoint that returns one overall "failed" when 1 of 500 items was bad.
- Unbounded batches: one 1 GB request ties up a server and its database.

6 Key terms

Collection: plural noun, e.g. /orders

Sub-resource: owned child, e.g. /users/42/addresses

Global ID: unique on its own; earns a top-level path

Query parameter: filter, sort or page a collection

Custom method: POST /orders/1001:cancel

Action resource: POST /orders/1001/cancellations

Bulk endpoint: many items, one call, per-item results

Job resource: tracks a long-running request

202 Accepted: accepted, will finish later

7 Interview questions

Q1. How do you design endpoints for nested resources? BASIC

Nest one level only for true ownership, like /users/42/addresses. Give resources with global IDs their own top-level path (/orders/1001), and use query parameters for filtering (/orders?user_id=42&status=shipped).

Q2. What naming rules make an API predictable? BASIC

Plural nouns plus IDs (/orders/1001), lowercase hyphenated paths (/shipping-addresses), one JSON casing everywhere, and a version prefix (/v1).

Q3. How do you expose non-CRUD actions like cancel or refund? SCENARIO

As a POST to a custom method (/v1/orders/1001:cancel) or to an action resource (/v1/orders/1001/cancellations, which also records who and why). Make it idempotent and return 409 if the order can't be cancelled.

Q4. A customer must import 10,000 products. Design the API. SCENARIO

POST /v1/products:batchCreate capped at 500 per request with per-item results (20 calls), or one async job: 202 Accepted with Location /v1/jobs/{id}, polled for progress and a final list of failures.

Q5. Why not just PATCH the order status to cancelled? TRICKY

Cancel triggers refunds, restocking and emails; a field update hides that, makes validation awkward, and lets clients set any status, like shipped. An explicit action makes intent, rules and permissions clear.

Q6. Sub-resource or query parameter: /users/42/orders or /orders?user_id=42? TRICKY

Both can exist. The sub-resource expresses ownership and scopes authorization to user 42; the query parameter suits admin tools searching across users. Expose the second only to callers allowed to see other users' orders.

Next: SD-C-006 · API Versioning Without Breaking Clients backendroadmap.com

API Versioning Without Breaking Clients

URL vs header vs date, and how to retire a version

Watch the 4-min video

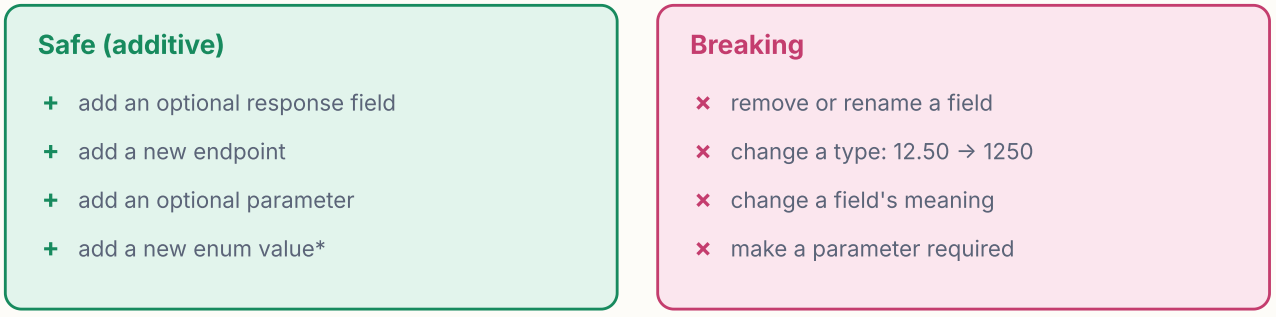
In one line: evolve your API additively so you rarely need a new version; when you must break, run the new version side by side (one shared core, thin adapters) and retire the old one with notice, Deprecation and Sunset headers, usage tracking and brownouts.

1 The problem: a breaking change with live clients

Your payments API sends amounts as a decimal number of dollars: `"amount": 12.50`. You want an integer number of cents plus a currency: `"amount_cents": 1250, "currency": "USD"`. Integers avoid floating-point rounding, and the currency field makes the API multi-currency. But thousands of partner integrations parse the old format, and you cannot make them all change overnight. API versioning lets old and new clients live side by side.

2 Know what actually breaks clients

Before you version anything, ask whether the change is breaking at all. A change is **backward compatible** if every existing client keeps working without touching its code.



** safe only if clients ignore unknown values; strict clients may still break*

Figure 1. Additive changes are safe as long as clients ignore fields they don't know. Anything that removes, renames, retypes or tightens is breaking.

So the best versioning strategy is to **need it rarely**. Add new fields next to old ones, keep the old ones working, and tell clients to be **tolerant readers**: ignore unknown fields instead of failing on them. Many teams ship for years on one version this way.

3 Three ways to signal the version

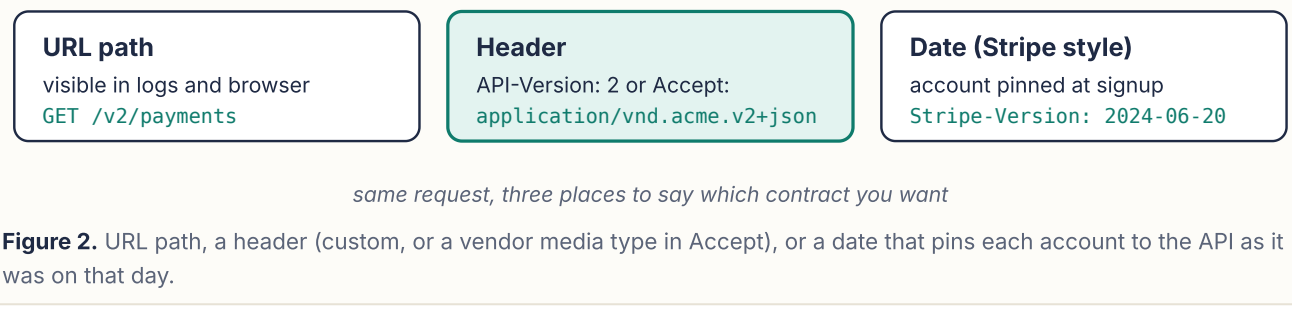


Figure 2. URL path, a header (custom, or a vendor media type in Accept), or a date that pins each account to the API as it was on that day.

- **URL path**: `/v2/payments`. The most common choice for public APIs.
- **Header**: a custom `API-Version: 2`, or content negotiation with `Accept: application/vnd.acme.v2+json`. URLs stay clean, but the version is invisible in a browser and easy to forget in caches.
- **Date**: Stripe pins each account to the API version from the day it signed up. Clients opt in to a newer date when ready, account-wide or per request with a `Stripe-Version` header. This allows many small breaking changes instead of one big v2.

Pros and cons of URL versioning

Pros	Cons
Visible: the version shows in logs, dashboards and bug reports	Purists: a URI should name the resource, not its format
Easy to try in a browser or with curl	Versions the whole API at once, even for one changed endpoint
Easy to route at the gateway by prefix	Every client must change its URLs to upgrade
Caches like any other URL (no `Vary` header needed)	Tends to push you to big, infrequent versions

4 Example: /v2/payments next to /v1 for twelve months

The gateway routes both prefixes. The key design choice: **only one implementation holds the business logic**, written against the v2 model. A thin **adapter** translates v1 requests into v2 and v2 responses back into v1. Bugs get fixed once, not twice, and v1 cannot quietly drift.

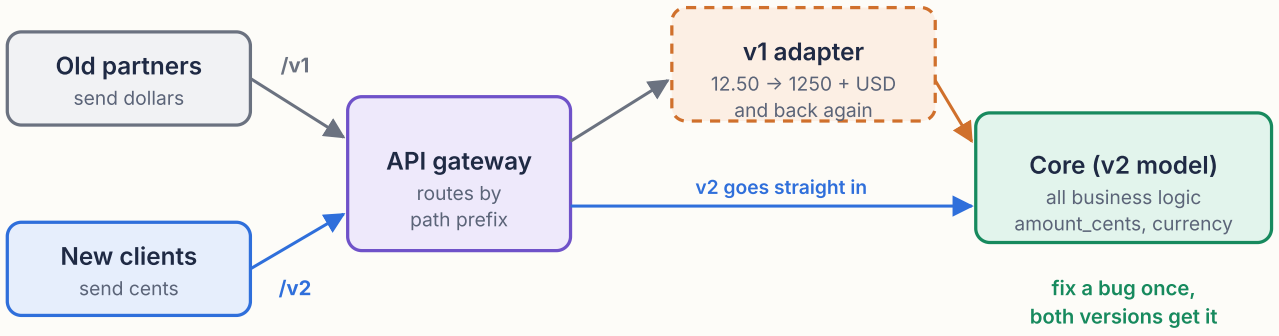


Figure 3. v1 traffic passes through a small adapter that converts dollars to cents on the way in and cents to dollars on the way out; v2 goes straight to the core.

WORKED EXAMPLE

A v1 partner sends `POST /v1/payments {"amount": 12.50}`. The adapter parses the amount as a decimal (not a float), multiplies by 100 and adds the account's default currency: `{"amount_cents": 1250, "currency": "USD"}`. The core charges the card and returns `amount_cents: 1250`; the adapter divides by 100 and the partner sees `"amount": 12.50`, exactly as before.

Cost check: if v1 still carries 30% of 10,000 RPS, the adapter handles 3,000 RPS of pure field mapping, microseconds each. Two full codebases would double every bug fix and security patch.

5 Deprecate politely



Figure 4. A twelve-month retirement. Brownouts are planned windows where v1 returns errors, to flush out forgotten integrations before the final switch-off.

- **Announce** the timeline with a migration guide that maps every v1 field to v2.
- **Mark** every v1 response with a `Deprecation` header and a `Sunset` header carrying the shutdown date, plus a link to the guide.
- **Track** v1 usage per client (API key) and contact the teams still calling it.
- **Brownout** near the end: for example 1 hour, then 4 hours, then a day, where v1 returns errors. Someone always discovers a cron job nobody remembered.
- **Switch off** for good; returning `410 Gone` with a pointer to v2 is clearer than a 404.

```
HTTP/1.1 200 OK
Deprecation: @1782864000
Sunset: Wed, 30 Jun 2027 23:59:59 GMT
Link: <https://docs.acme.com/migrate-v2>; rel="deprecation"
```

6 Rules that keep versioning manageable

- **At most two major versions live** at once. Each extra one multiplies testing and support.
- **Never change behavior inside a version**, even when it feels like a fix. Clients may depend on the old behavior. Ship a new version or an opt-in flag.
- **Version client SDKs to match**, so upgrading the SDK is how most clients move to v2.
- **Log the version on every request**, so you always know who is still on the old one.

COMMON MISTAKES

- Bumping the version for additive changes that were never breaking.
- Copy-pasting the whole codebase for v2, so every fix must be made twice.
- “Fixing” behavior inside v1 and breaking clients who relied on it.
- Switching v1 off on the announced date with no usage data and no brownouts.
- Header versioning behind a CDN without `Vary` on that header: v1 and v2 responses get mixed in the cache.

7 Key terms

Breaking change: existing clients fail without code changes

Additive change: new optional field, endpoint or parameter

Tolerant reader: client that ignores unknown fields

Media-type versioning: version inside the Accept header

Date versioning: account pinned to a dated API version

Adapter: thin layer translating old ↔ new format

Deprecation header: says this version is deprecated

Sunset header: date the endpoint will stop working

Brownout: planned short outage of the old version

8 Interview questions

Q1. How do you version an API without breaking clients? BASIC
Prefer additive, backward-compatible changes so you rarely need a version. When you must break, publish v2 side by side with a shared core and a thin v1 adapter, then deprecate v1 with notice (e.g. 12 months), Deprecation and Sunset headers, per-client usage tracking and brownouts.

Q2. Which changes are breaking? BASIC
Removing or renaming a field, changing a type (12.50 dollars → 1250 cents) or a meaning, and making a parameter required. Adding an optional field, endpoint or parameter is safe if clients ignore unknown fields.

Q3. What are the pros and cons of URL versioning? BASIC
Pros: visible in logs, testable in a browser, trivial to route at the gateway, caches like any URL. Cons: versions the whole API at once, forces every client to change URLs, mixes format into resource names, and encourages big infrequent versions.

Q4. You need v2 of one endpoint out of 80. What do you do? SCENARIO
First try an additive change (new field next to the old one). If it must break, header or date versioning lets you version just that change; with URL versioning, serve /v2 for the whole API but route the 79 unchanged endpoints to the same handlers.

Q5. v1 must be switched off in 2 months but 40 clients still call it. Plan? SCENARIO
Pull per-key usage, contact the top callers directly, send Deprecation/Sunset headers, then run brownouts (1 h, 4 h, 24 h) to surface forgotten jobs. Extend for a few big partners only with a written date.

Q6. A teammate fixes a rounding bug in v1 that changes results by a cent. Problem? TRICKY
Yes: clients may reconcile against the old behavior, so a silent change inside a version breaks them. Put the fix in v2 or behind an opt-in flag, and document it.

Q7. You version by header and put a CDN in front. What can go wrong? TRICKY
The CDN caches by URL, so a v1 response may be served to a v2 client. Send Vary: API-Version (or Accept) or include the version in the cache key; URL versioning avoids this entirely.

Next: SD-C-007 · API Pagination: Offset vs Cursor backendroadmap.com

API Pagination: Offset vs Cursor

and why deep pages are slow

Watch the 4-min video

In one line: offset pagination makes the database read and discard every skipped row, so deep pages get slow and shift when data changes; cursor (keyset) pagination remembers where you stopped and seeks there through an index, so page 500 costs the same as page 1.

1 Why pagination design matters

An infinite-scroll feed has **50 million posts**. Nobody downloads them all, so the API returns one page at a time. How you define the page boundary decides two things: whether page 500 loads in **5 ms or 5 seconds**, and whether users **see duplicates or miss posts** while scrolling.

2 Offset and limit: simple, with two big problems

“Give me 20 items, skipping the first 40”:

```
SELECT * FROM posts
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;          -- page 3
```

It is simple, and it lets you **jump straight to page 7** (offset = (page − 1) × size), which is great for admin tables with page numbers. But:

- **Deep offsets are slow.** To return rows 1,000,000 to 1,000,020, the database still walks past the first million rows (in the index or the table) and throws them away. The cost grows with the page number.
- **It is unstable when data changes.** Positions are counted fresh on every request, so an insert or delete between two requests shifts every row.

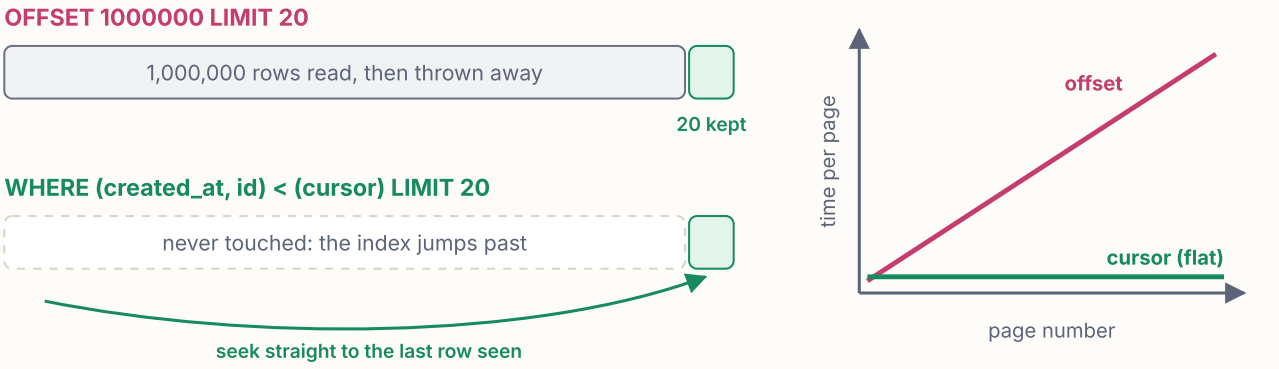


Figure 1. Offset cost grows linearly with depth because skipped rows are still read. A keyset query seeks to the boundary through the index, so cost stays flat.



Figure 2. A new post arrives between page 1 and page 2. Offset 3 now points at post 98, which the user already saw. A delete does the opposite: rows shift up and one post is silently skipped.

3 Cursor (keyset) pagination

Instead of saying **how many rows to skip**, the client says **where it stopped**: the sort values of the last row it saw.

```
SELECT * FROM posts
WHERE (created_at, id) < ($last_created_at, $last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- needs: CREATE INDEX ON posts (created_at DESC, id DESC);
```

With a composite index on `(created_at, id)`, the database jumps straight to that point in the index and reads exactly 20 entries. Page 500 costs the same as page 1. New posts at the top don't move the boundary, so there are no duplicates; deleted posts simply aren't returned, so nothing is skipped.

The row comparison `(a, b) < (x, y)` works in PostgreSQL. Where it isn't supported or doesn't use the index well, write it out: `created_at < $t OR (created_at = $t AND id < $id)`.

The tie-breaker is not optional

Ordering must be **stable and unique**. Many posts can share the same timestamp (a bulk import can write thousands in one millisecond). If you sort only by `created_at` and a page ends in the middle of a group of equal timestamps, `created_at < t` skips the rest of that group and `created_at <= t` repeats it. Adding the unique `id` as the last sort column gives every row an exact position.

4 The API shape: an opaque cursor

Don't expose raw database values as parameters. Encode the last `created_at` and `id` into an **opaque cursor**, for example Base64 of a small JSON object, ideally **signed** (an HMAC) so clients can't tamper with it or craft cursors that probe other data. The server is free to change what's inside later.

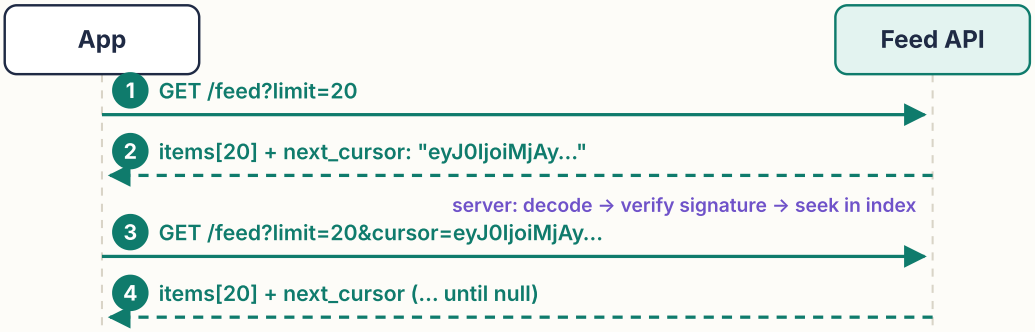


Figure 3. The response carries the items and a next_cursor. The client sends it back unchanged. When next_cursor is null, there are no more pages. The client treats the cursor as an opaque string.

```
{
  "items": [ {"id": 9182, "created_at": "2026-11-04T09:12:44Z", ...}, ... ],
  "next_cursor": "eyJ0IjoiMjAyNi0xMS0wNFQwOToxMjo0NFoiLCJpZCI6OTE4Mn0.sig",
  "has_more": true
}
```

WORKED EXAMPLE

Page size 20, user scrolls to page 500: offset = 499 × 20 = **9,980**. With offset, the database reads about 10,000 index entries to return 20; at page 50,000 it reads about **1,000,000**, and if each read is ~5 µs that's ~5 s per page. With a cursor, every page reads about **20 entries** plus one index descent (a few page reads in a B-tree of height 3–4), well under 5 ms at any depth.

Fetching `LIMIT 21` and returning 20 is a cheap way to know `has_more` without counting the table.

5 Trade-offs and limits

	Offset / limit	Cursor / keyset
Deep page cost	Grows with the offset	Constant (with the right index)
Inserts / deletes while paging	Duplicates or skipped rows	Stable
Jump to page 50	Yes	No: next (and maybe previous) only
Total count	Often shown, but `COUNT(*)` on 50M rows is expensive	Usually `has_more` instead
Change sort order	Any time	Starts a new cursor (tied to one order)
Best for	Small admin tables with page numbers	Feeds, timelines, APIs, data exports

Either way, **cap the page size**: a default of 20 and a maximum of 100, so `limit=1000000` can't turn one request into a full table dump.

COMMON MISTAKES

- Sorting by a non-unique column (only `created_at`) with no tie-breaker: rows repeat or vanish at page edges.
- No index matching the sort, so the "seek" becomes a sort of the whole table on every page.
- Exposing raw values (`?after_id=9182`) that clients start building logic on; you can never change them.
- Returning an exact total count on every page of a huge table.

6 Key terms

Offset pagination: skip N rows, return the next M

Tie-breaker: unique column that makes the order total

Keyset pagination: continue after the last sort key seen

Composite index: index on several columns, e.g. (created_at, id)

Cursor: opaque token encoding that sort key

has_more: flag instead of an exact total

7 Interview questions

Q1. Why is offset pagination slow for deep pages? BASIC

The database must read and discard every skipped row before returning the page, so OFFSET 1,000,000 reads about a million entries to return 20. Cost grows linearly with the page number.

Q2. How does cursor pagination work? BASIC

The client sends an opaque cursor encoding the last sort key, e.g. (created_at, id). The server decodes it, runs WHERE (created_at, id) < (t, id) ORDER BY created_at DESC, id DESC LIMIT 20 on a matching index, and returns the page plus a new next_cursor (null at the end).

Q3. Why add id to the ORDER BY? BASIC

Timestamps aren't unique. Without a unique tie-breaker, rows with equal created_at at a page boundary are skipped or repeated. created_at + id gives every row an exact position.

Q4. Users report seeing the same post twice while scrolling. Likely cause? SCENARIO

Offset pagination on a feed with new inserts: each new post shifts every row down, so the next offset repeats one. Switch to keyset pagination.

Q5. An admin UI needs page numbers on a 2,000-row table. Cursor or offset? SCENARIO

Offset is fine: 2,000 rows is tiny, users want to jump to page 7 and see a total. Cursors pay off on large, changing data like feeds.

Q6. Page 1 takes 4 ms but page 3,000 takes 3 s. Fix? SCENARIO

It's offset depth: 3,000 × 20 = 60,000 rows skipped. Add a (created_at, id) index and move to cursors; for old deep links, cap the maximum offset (e.g. 10,000) and suggest filters.

Q7. Why make the cursor opaque and signed instead of ?after_id=9182? TRICKY

Opaque lets you change what's inside (add a sort key, switch columns) without breaking clients. Signing stops clients from forging cursors to bypass filters or probe data.

Q8. How do you return a total count for 50M rows? TRICKY

Often you don't: return has_more (fetch 21, return 20). If the UI needs a number, use an approximate count (e.g. planner statistics) or a cached counter, labelled “about 50M”.

Next: SD-C-008 · Filtering, Sorting & Field Selection backendroadmap.com

API Filtering, Sorting & Field Selection

flexible query parameters without table scans

Watch the 4-min video

In one line: give clients consistent filter, sort and fields parameters, but only on an **allow list** of fields that each have an index designed for real queries; reject anything else with a 400, use parameterized SQL, cap page sizes, and send fuzzy search to a search engine.

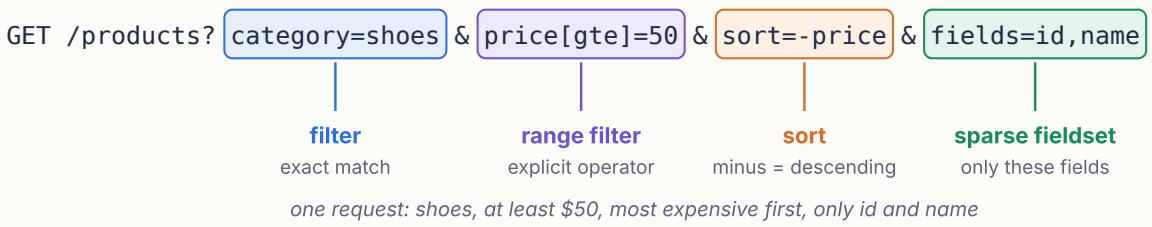
1 One request, three jobs

A shopper opens the shoes category, sorts by price from high to low, and the mobile app only needs the id and the name. One request expresses all three:

```
GET /products?category=shoes&sort=-price&fields=id,name
```

Designing these query parameters is easy. Designing them so they **don't melt your database** is the real interview question.

2 Consistent conventions



one request: shoes, at least $50, most expensive first, only id and name

Figure 1. Each kind of parameter has one predictable shape, so clients can guess the API without reading docs for every endpoint.

- **Filters** are plain query parameters named after fields: `category=shoes`, `status=active`.
- **Ranges** use explicit operators: `price[gte]=50`, `price[lt]=100` (gte = greater than or equal, lt = less than).
- **Sorting** uses one `sort` parameter, a comma-separated list where a minus sign means descending: `sort=-price,name`.
- **Field selection** uses `fields=id,name`, sometimes called **sparse fieldsets**.

Field selection is a cheap win. A mobile list needs 2 fields, not all 42. Smaller responses mean less bandwidth and faster parsing, and sometimes cheaper queries, if the server can skip joins for fields nobody asked for. The client chooses the shape without needing GraphQL.

3 The danger: filtering on anything

If **any** field can be filtered or sorted, someone will send `sort=description` or filter on a column with no index. On a table with **50 million rows**, that's a **full table scan** plus a big sort, and one slow query can saturate the database's CPU and I/O for every other user.

So never pass parameters straight into SQL:

- **Allow-list** the fields that can be filtered and the fields that can be sorted, and back each one with an index.
- **Reject unknown or unsupported parameters** with `400 Bad Request` and a helpful message, instead of silently ignoring them. A silently ignored typo like `categroy=shoes` returns every product and looks like a bug in your API.
- **Map public names to columns yourself**, so the API name `price` can point at `p.price_cents` and you can rename columns freely.
- **Always use parameterized queries** for values, to block SQL injection. Column names in `ORDER BY` can't be bound as parameters, which is one more reason they must come from your allow list.

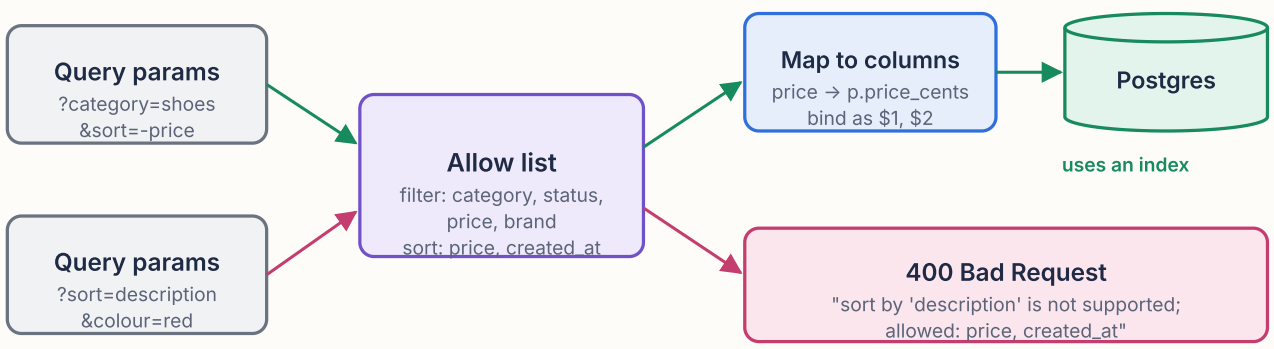


Figure 2. Allowed parameters are mapped to real columns and bound as parameters. Anything outside the list is rejected up front with a clear 400.

```
SORTABLE = {"price": "price_cents", "created_at": "created_at"}
FILTERABLE = {"category": "category", "status": "status", "brand": "brand"}

for name in query.sort:                 # e.g. ["-price"]
    col = SORTABLE.get(name.lstrip("-"))
    if col is None:
        raise HTTPException(400, f"cannot sort by {name}; allowed: {list(SORTABLE)}")
# values are never pasted into SQL:  WHERE category = $1 AND price_cents >= $2
```

4 Design indexes for the real access patterns

For *category + sort by price*, a **composite index on (category, price)** lets the database find the shoes already in price order and **stop after 20 rows**. Column order matters: the equality filter (category) comes first, the sort column (price) second. An index on price alone, or on (price, category), can't do this.



Figure 3. Inside the index all shoes sit together, ordered by price. The database seeks to the first one, reads 20 entries and stops, with no sort step and no table scan.

WORKED EXAMPLE

50M products, 2M of them shoes. `category=shoes&sort=-price&limit=20`:

- **No index:** scan 50M rows, keep 2M shoes, sort 2M rows by price, return 20. Seconds of CPU and I/O per request.
- **Index on (category) only:** find 2M shoes fast, but still sort 2M rows on every request.
- **Index on (category, price):** read ~20 entries in order. About a millisecond, and the same on page 1,000 with cursor pagination.

Add the tie-breaker for cursors: `(category, price, id)`.

Also: combine filters with **cursor pagination** so every page stays fast; **require a selective filter** on expensive endpoints (for example, orders must be filtered by customer or a date range); and **cap the page size** (default 20, maximum 100).

5 Filtering is not search

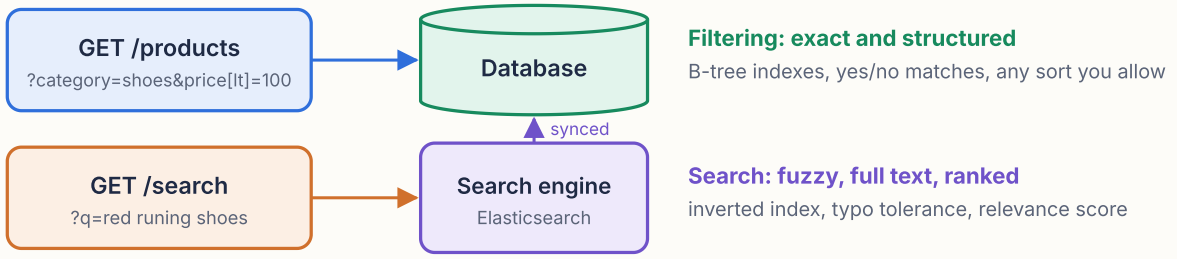


Figure 4. Exact, structured filters belong in the database with B-tree indexes. Fuzzy, ranked text search belongs in a search engine, usually behind its own endpoint.

Filtering is exact and structured: `category = shoes`, `price < 100`. A row matches or it doesn't. **Search** is fuzzy, full text and **ranked by relevance**: "red runing shoes" should still find red running shoes despite the typo, best match first. `LIKE '%red%'` can't use a normal index and has no ranking, so search belongs in a search engine such as **Elasticsearch** or **OpenSearch**, kept in sync from the database and exposed as a separate `/search` endpoint.

COMMON MISTAKES

- Accepting any column name for filter or sort: one request can trigger a 50M-row scan.
- Silently ignoring unknown parameters, so typos return everything.
- Building SQL by string concatenation from query parameters (SQL injection).
- Indexing each column separately when queries need one composite index in the right order.
- Implementing search with `LIKE '%term%'` on a big table.

6 Key terms

Filter: exact, structured condition on a field

Allow list: the only fields that may be filtered or sorted

Parameterized query: values bound separately from SQL text

Range operator: gte, gt, lte, lt in the parameter

Composite index: one index over several columns, in order

Search engine: inverted index, fuzzy, relevance-ranked

Sparse fieldset: client picks returned fields

Full table scan: reading every row to answer a query

Selective filter: condition that matches few rows

7 Interview questions

Q1. How do you prevent filter parameters from causing full table scans? BASIC
Allow-list filterable and sortable fields, back each with an index designed for real query patterns (e.g. (category, price)), reject unknown parameters with a 400, use parameterized queries, cap page size at ~100 and pair it with cursor pagination.

Q2. What's the difference between filtering and search? BASIC
Filtering is exact, structured matching in the database (category = shoes, price < 100) using B-tree indexes. Search is fuzzy, full-text, relevance-ranked matching with typo tolerance, served by a search engine like Elasticsearch or OpenSearch.

Q3. What are sparse fieldsets and why use them? BASIC
A fields parameter (fields=id,name) so clients get only what they need: 2 fields instead of 42 means smaller payloads, faster parsing, and sometimes skipped joins.

Q4. category=shoes&sort=-price is slow on 50M rows. Which index? SCENARIO
A composite index on (category, price), equality column first, sort column second, plus id as a tie-breaker for cursors. The database seeks to shoes and reads 20 entries in order with no sort step.

Q5. A client sends sort=description. What should the API do? SCENARIO
Return 400 with a message listing the allowed sort fields. Don't silently ignore it (the client would think it worked) and never pass it into ORDER BY.

Q6. Why can't you just parameterize the sort column to block injection? TRICKY
Placeholders bind values, not identifiers like column names or ASC/DESC. Sort columns must be mapped from an allow list to known column names in code.

Q7. Product wants users to filter on any of 30 attributes. How? TRICKY
Index the few that real traffic uses and allow-list those. For free-form combinations across many attributes, index the catalog in a search engine, which handles arbitrary filters and facets well, and keep the database API narrow.

Next: SD-C-009 · Idempotency Keys backendroadmap.com

Idempotency Keys

how payment APIs never charge twice on retry

Watch the 4-min video

In one line: the client sends one unique `Idempotency-Key` per operation and reuses it on every retry; the server atomically records the key, processes once, stores the response and replays it for duplicates, so a retry can never charge twice.

1 The retry that charges twice

A customer taps **Pay**. Your API charges the card, but the response **times out** on its way back to the phone. The app can't tell whether the charge happened, so it does the sensible thing and **retries**. Without protection, the customer is charged twice.

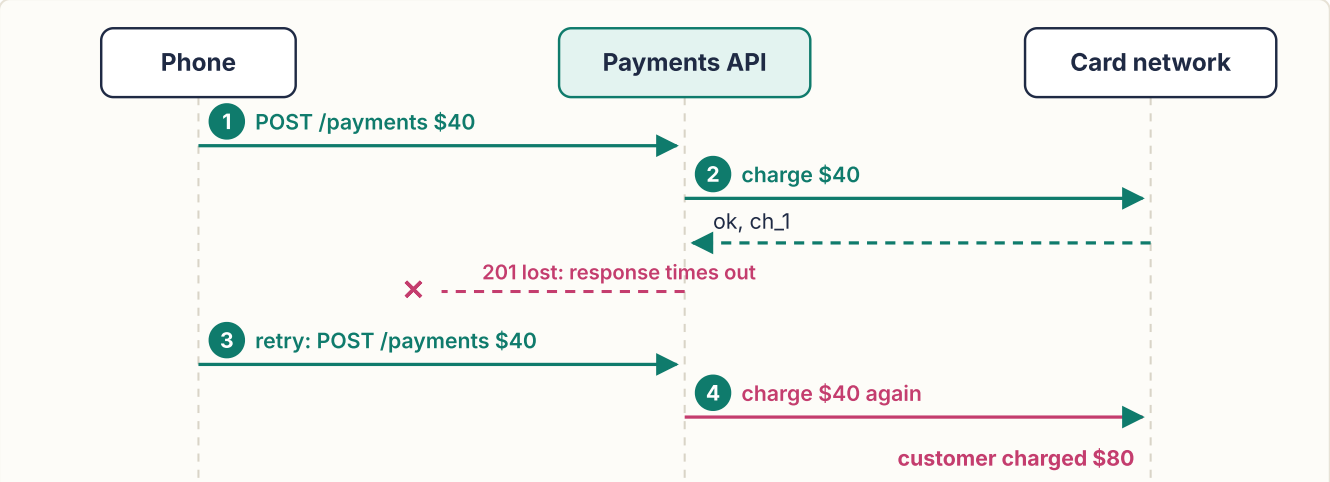


Figure 1. The first request succeeded; only its response was lost. The retry looks like a brand-new payment, so the server charges again.

2 Idempotent operations

An operation is **idempotent** if doing it once or many times leaves the **same result** on the server.

- **GET, PUT, DELETE** are idempotent by definition: putting the same profile twice, or deleting the same order twice, ends in the same state. (The second DELETE may return 404 instead of 204; the state is still the same.)
- **POST** is not: every `POST /payments` can create a new charge.
- Networks give you **at-least-once** delivery: timeouts, client retries, load-balancer retries and queue redelivery mean the same request can arrive more than once. Retries are a fact of life, so the server must make them safe.

3 The idempotency key and the server flow

The client generates a unique ID, such as a **UUID**, **once per logical operation** (one checkout, not one HTTP attempt), and sends it in a header. If it retries, it sends the **same key**.

```
POST /payments
Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7
Content-Type: application/json

{"amount_cents": 4000, "currency": "USD", "card": "tok_visa"}
```

The server stores, with the key, a **fingerprint** of the request: a hash of the method, path and body. Then:

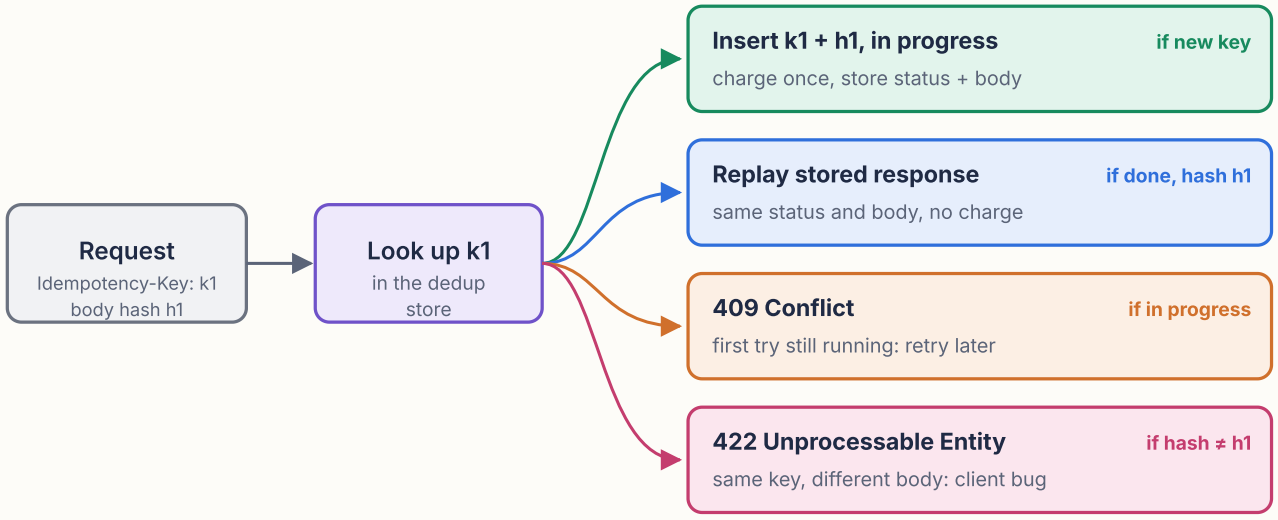
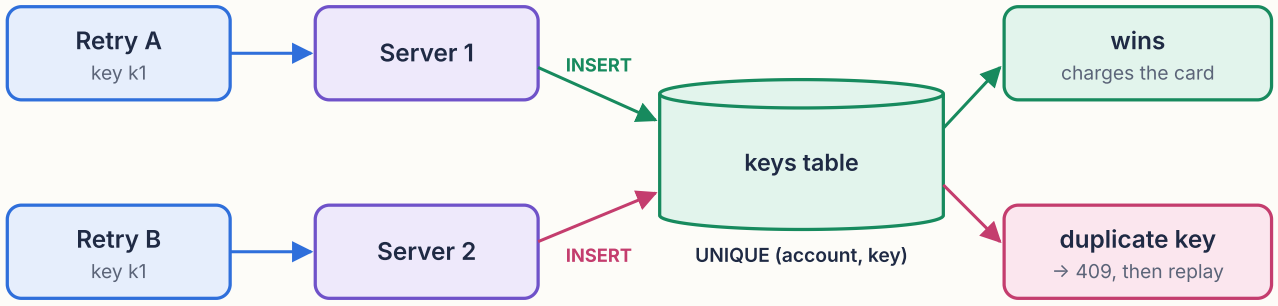


Figure 2. Every request with a key goes through the same lookup. Only a new key leads to a charge; everything else replays, waits or is rejected.

- **New key:** insert it with status *in progress*, process the payment, then save the response status and body with the key and mark it *completed*.
- **Exists and completed:** replay the stored response: same status code, same body (the original `201` with charge `ch_1`).
- **Exists and still in progress:** return `409 Conflict`, so the client retries a little later.
- **Same key, different fingerprint:** that's a client bug (it reused a key for a different payment). Reject with `422` rather than replaying a response that belongs to another request.

4 Concurrent duplicates: make the insert atomic

The tricky case: two retries arrive **at the same moment, on different servers**. If both do "look up, not found, insert", both charge. So the claim must be **atomic**: a **unique constraint** on the key in Postgres, or `SET … NX` in Redis. Only one request wins; the other sees that the key already exists.



Redis equivalent: SET idem:acct42:k1 in_progress NX EX 86400 (only one caller gets OK)

Figure 3. Both servers try to insert the same key. The database lets exactly one succeed; the loser gets a unique-violation and answers 409, and a later retry replays the stored result.

Ideally, **write the key in the same database transaction as the payment record**, so the two can never disagree: either both exist or neither does. If you use Redis for speed, add a recovery rule for keys stuck *in progress* after a crash (for example, expire the claim after a few minutes and check with the processor before charging again).

```
CREATE TABLE idempotency_keys (
  account_id    BIGINT NOT NULL,
  key           TEXT   NOT NULL,
  request_hash  TEXT   NOT NULL,
  status        TEXT   NOT NULL,          -- 'in_progress' | 'completed'
  response_code INT,
  response_body JSONB,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, key)
);
```

5 Where to store keys, and for how long

- **Scope keys per account** (`PRIMARY KEY (account_id, key)`), so two customers can never collide or read each other's responses.
- **Keep them long enough** to cover every realistic retry. Stripe keeps keys for at least **24 hours**, then prunes them.
- **Redis with a TTL** is fast and expires keys for you; a **database table** is durable and can share a transaction with the payment.
- **Pass the key downstream** to the payment processor, so the whole chain is idempotent, not just your API.

WORKED EXAMPLE

Peak **2,000 payments/s**, keys kept **24 h**. Daily volume is lower than peak; say an average of 500/s: 500 × 86,400 ≈ **43 million keys** live at once. At ~1 KB each (key, hash, stored response) that's about **43 GB**: fine for a Postgres table pruned daily by `created_at`, or a Redis cluster with `EX 86400`. Retries are a small share of traffic, so most lookups find nothing and cost one index probe.

COMMON MISTAKES

- Generating a new key on every retry: then the key protects nothing.
- “Check then insert” without a unique constraint: two concurrent retries both charge.
- Ignoring the request fingerprint, so a reused key replays the wrong payment.
- Global keys not scoped per account.
- Storing the key in Redis but the payment in Postgres with no recovery for crashes in between.
- Not sending the key to the downstream processor.

6 Key terms

Idempotent: once or many times, same result

Dedup store: table or cache of seen keys

Atomic claim: unique insert / SET NX: one winner

At-least-once: a message may arrive more than once

Request fingerprint: hash of method, path and body

TTL: how long a key is kept

Idempotency key: client-made unique ID per operation

Replay: return the stored original response

Key scope: keys unique per account

7 Interview questions

Q1. How do you make a payment API idempotent? BASIC
The client sends a unique Idempotency-Key per operation and reuses it on retries. The server atomically inserts the key with a request fingerprint, processes once, stores the response and replays it for duplicates: 409 while in progress, 422 for a different body.

Q2. Which HTTP methods are idempotent? BASIC
GET, HEAD, PUT and DELETE (and OPTIONS). POST is not, and PATCH isn't guaranteed. That's why POST /payments needs an idempotency key.

Q3. Where do you store keys, and for how long? BASIC
In a durable store, ideally in the same transaction as the payment, or Redis with a TTL; scoped per account; for at least the retry window, typically 24 hours (Stripe keeps them at least 24 h).

Q4. Two identical retries hit different servers 5 ms apart. What prevents a double charge? SCENARIO
An atomic claim: a unique constraint on (account_id, key) or Redis SET NX. Exactly one insert succeeds; the other gets a conflict, returns 409, and its next retry replays the stored response.

Q5. A client reuses the same key for a $40 and then a $90 payment. What happens? SCENARIO
The fingerprint (hash of method, path, body) differs, so the server rejects the second with 422 instead of replaying the $40 result or charging $90 under the old key.

Q6. Your server crashes after charging the card but before saving the response. What then? SCENARIO
The key is stuck in progress. If the key and payment are in one transaction, the payment row tells you the outcome. Otherwise pass the key to the processor so a retried charge is deduplicated there, and reconcile stuck keys after a timeout.

Q7. Why not just deduplicate on (card, amount) within a minute? TRICKY
Two real $40 purchases a minute apart are legal, and they'd be blocked; a slow retry after 61 s would still double charge. Only the client knows which attempts are the same operation, so it must say so with a key.

Q8. Should a replayed response come from the cache even if the original was a 500? TRICKY
Store only responses that reflect a final outcome. If the first attempt failed before any side effect, release the key so a retry can run again; replaying a transient 500 forever would block the payment.

Next: SD-C-010 · API Request Validation backendroadmap.com

API Request Validation

Pydantic, 422, unknown fields and size limits

Watch the 4-min video

In one line: validate in layers, as early and cheaply as possible: coarse size and type limits at the proxy (413), schema rules at the service edge (422), and business rules in the domain layer (409); reject unknown fields, use strict types for money and IDs, and cap every size.

1 A request that should never reach your code

A signup request arrives. The email is `bob@nowhere`, the age is **−5**, the name is **50 megabytes** long, and there's an extra field: `is_admin: true`. Every one of these should be stopped **before it touches your business logic**. That's request validation.

```
{
"email": "bob@nowhere",
"age": -5,
"name": "AAAA… (50 MB)",
"is_admin": true
}
```

---- EmailStr: not a valid email domain — **schema → 422**

---- age must be 13 to 120 — **schema → 422**

---- body over 1 MB, stopped at the proxy — **proxy → 413**

---- unknown field, extra='forbid' — **schema → 422**

Figure 1. Four problems in one body. Each is caught by a different rule; the oversized name never even reaches the app because the proxy rejects the body.

2 Validate in layers, as early as possible

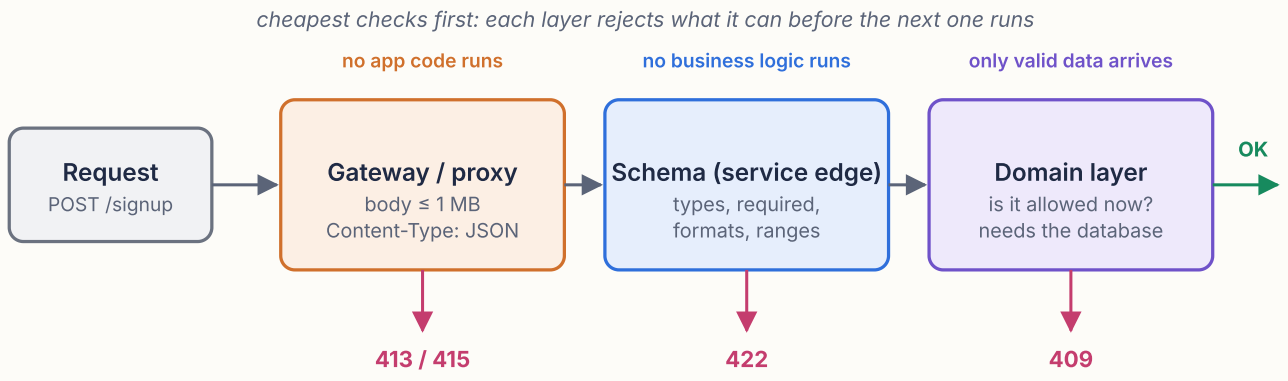


Figure 2. Each layer rejects what it can, as cheaply as it can. By the time a request reaches the domain layer, its shape is already guaranteed.

- **Gateway or reverse proxy:** coarse limits like body size and content type. Too big → `413 Content Too Large`; wrong type → `415 Unsupported Media Type`.
- **Service edge (schema):** required fields, types, formats and ranges. Fails → `422` with a list of which fields failed and why.

- **Domain layer:** business rules. Is this allowed right now? These usually need the database.

Client-side checks in the app or browser are nice for the user, but they are not security: anyone can send raw HTTP. **The server is the real boundary.**

3 Schema validation with Pydantic (FastAPI)

In FastAPI you declare a Pydantic model and use it as the request body type. FastAPI validates every request against it before your function runs:

```
from pydantic import BaseModel, ConfigDict, EmailStr, Field

class SignupRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")        # unknown fields → error
    email: EmailStr                                   # needs the email-validator package
    age: int = Field(ge=13, le=120)
    name: str = Field(min_length=1, max_length=100)
    tags: list[str] = Field(default=[], max_length=20)

@app.post("/signup", status_code=201)
def signup(req: SignupRequest): ...
```

If a request doesn't match, FastAPI returns **422** automatically, listing exactly which fields failed:

```
HTTP/1.1 422 Unprocessable Entity
{"detail": [
  {"type": "greater_than_equal", "loc": ["body", "age"],
   "msg": "Input should be greater than or equal to 13", "input": -5},
  {"type": "extra_forbidden", "loc": ["body", "is_admin"],
   "msg": "Extra inputs are not permitted", "input": true}
]}
```

Other stacks do the same with **JSON Schema** or **OpenAPI** validators (for example Ajv in Node.js, or validating at the API gateway against the OpenAPI spec).

4 Unknown fields and type coercion

Reject unknown fields. With `extra="forbid"`, the sneaky `is_admin` field fails validation instead of being silently accepted. This blocks **mass assignment** attacks, where a client sets a field you never meant to expose (for example, when code copies the request body straight into a database row). It also catches client typos early: `emial` fails loudly instead of being ignored. (Pydantic's default is to *ignore* extra fields, so you must opt in.)

Be careful with coercion. Lenient ("lax") parsing turns the string `"42"` into the integer 42, which is convenient. But it can also turn the string `"1"` into `True`, or accept a float like `19.999999` where you need an exact amount. For **money and IDs, use strict types**, so the value must already have the right type: `StrictInt`, `Field(strict=True)`, or integer cents / `Decimal` for amounts.

5 Oversized payloads

- **Cap the body at the proxy**, for example 1 MB with nginx's `client_max_body_size 1m;`. Huge bodies are rejected with **413** before your app even reads them.
- **Cap inside the schema:** maximum string lengths, maximum array items and maximum nesting depth. A body under 1 MB can still hold a 200,000-element array that costs a lot of CPU and memory to parse, validate and process.
- **Large files skip the API entirely** and upload directly to object storage with a presigned URL.

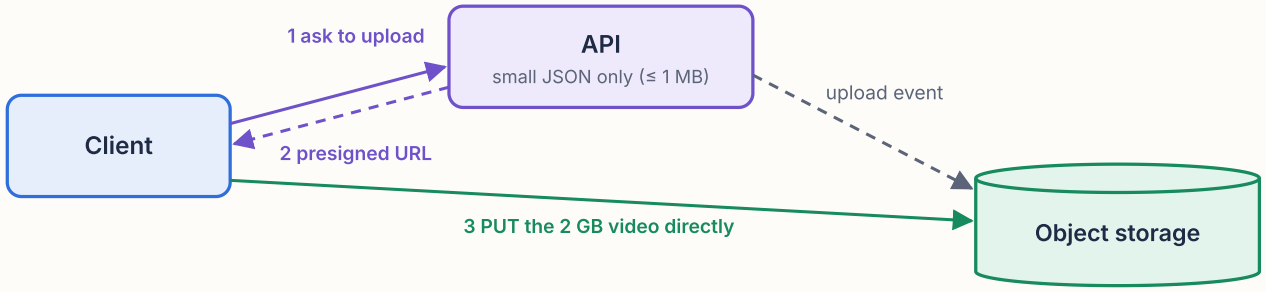


Figure 3. The API only hands out a short-lived presigned URL; the large file goes straight to object storage, which then notifies the backend.

WORKED EXAMPLE

Without limits, an attacker sends 100 requests per second, each a 50 MB name. That's **5 GB/s** of data your app servers must receive, buffer and parse: a few seconds are enough to exhaust memory. With `client_max_body_size 1m`, nginx answers 413 as soon as it sees the Content-Length header (or once a streamed body passes 1 MB), and the app never sees the requests.

Inside the limit: `max_length=100` on name and `max_length=20` on tags means one valid body can't exceed a few kilobytes of meaningful data.

6 Validation vs business rules

	Validation (schema)	Business rules (domain)
Question	Is this request well formed?	Is this allowed right now?
Examples	Is the email a valid email? Is age 13–120?	Is the email already registered? Is there enough balance?
Needs the database?	No	Usually yes
Where	Service edge (Pydantic, JSON Schema)	Domain / service layer
Typical error	`422` with field errors	`409` with a specific code, e.g. `EMAIL_TAKEN`

COMMON MISTAKES

- Trusting client-side validation as security.
- Leaving extra fields allowed (Pydantic's default) and copying the body into a model: mass assignment.
- Lax coercion for money and IDs, so `"1"` becomes `True` or a float slips in.
- No body size limit at the proxy, and no `max_length` on strings and arrays.
- Putting database lookups in the schema validators, or returning 422 for “email already taken”.

7 Key terms

Schema validation: types, required fields, formats, ranges

extra="forbid": unknown fields fail validation

422 Unprocessable Entity: well-formed JSON, invalid fields

Mass assignment: client sets fields it shouldn't

413 Content Too Large: body exceeds the size limit

Type coercion: lax conversion, e.g. "42" → 42

Strict mode: value must already have the type

Presigned URL: short-lived direct upload link

Business rule: context check, often needs the DB

8 Interview questions

Q1. Where should validation happen? BASIC

In layers: coarse limits at the gateway (body size, content type), schema validation at the service edge with models like Pydantic (422), and business rules in the domain layer (409). Cheapest checks first.

Q2. How do you protect against oversized payloads? BASIC

Cap the body at the proxy (e.g. 1 MB in nginx) so it returns 413 before the app reads it; limit string lengths, array sizes and nesting depth in the schema; send large files directly to object storage with presigned URLs.

Q3. 400 vs 422 vs 409? BASIC

400: the request is malformed or can't be parsed (FastAPI reports bad JSON as 422 too, by default). 422: valid JSON but fields fail validation (age = −5). 409: valid, but conflicts with current state (email already registered).

Q4. A user sends is_admin: true in a signup body. What stops it? SCENARIO

extra="forbid" on the request model makes it a 422. Also keep separate input and database models so even an allowed field can't map to is_admin.

Q5. A client sends amount_cents: "1000" (a string) and a flag as "1", and both are accepted. Problem? SCENARIO

Lax coercion turned them into 1000 and True. For money, IDs and flags use strict types (StrictInt, StrictBool or Field(strict=True)) so wrong types are rejected with 422, and the client fixes its bug instead of relying on guesses.

Q6. A 900 KB request with a 150,000-item array spikes CPU. The proxy limit is 1 MB. Why? SCENARIO

The body passes the size limit, but nothing caps the array. Add max_length on lists (e.g. 100 items) and limits on nesting depth, so validation fails fast.

Q7. Should “email already registered” be checked in a Pydantic validator? TRICKY

No. It needs the database and can change between check and insert. Check it in the domain layer, enforce it with a unique constraint, and return 409 EMAIL_TAKEN.

Next: SD-C-011 · API Error Handling backendroadmap.com

API Error Handling: 4xx vs 5xx & Problem Details

errors clients can act on, and which ones to retry

Watch the 4-min video

In one line: pick the right status class (4xx = client's fault, 5xx = server's fault), return one consistent Problem Details body with a stable machine-readable code and a trace ID, retry only transient errors, and never leak internals.

1 Why error design matters

Imagine a signup API that fails in three different ways. One endpoint returns `200` with `"error": true` hidden in the body. Another returns `500` with a stack trace that reveals the database host. A third returns plain text: “Something went wrong”.

No client can handle these reliably. Errors are part of your API contract, just like successful responses. Good error design gives every client the same three answers: **what kind of failure** (the status code), **exactly which error** (a stable code) and **what to do next** (fix, retry or contact support).

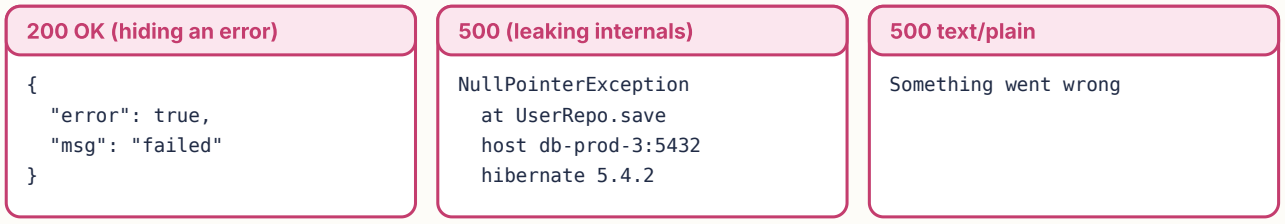


Figure 1. Three common anti-patterns. Each one breaks something different: monitoring, security, or client logic.

2 Start with the right status class

The first digit tells the client whose fault it is. A **4xx** means the client sent something wrong, so resending the same request won't help. A **5xx** means the server failed, and the same request might work later.

Code	Meaning	Typical cause
400 Bad Request	Malformed request	Invalid JSON, missing required field
401 Unauthorized	Not authenticated	Missing or expired token
403 Forbidden	Authenticated, but not allowed	User lacks the role
404 Not Found	No such resource	Wrong ID, or hidden from this caller
409 Conflict	Clashes with current state	Duplicate email, version mismatch
422 Unprocessable Content	Well-formed but invalid content	End date before start date
429 Too Many Requests	Rate limited	Over the limit; see `Retry-After`
500 Internal Server Error	Unexpected server failure	Bug, unhandled exception
502 Bad Gateway	Proxy got a bad upstream answer	Backend crashed or hung up

503 Service Unavailable	Temporarily can't serve	Overloaded, no healthy backends
504 Gateway Timeout	Proxy gave up waiting on upstream	Slow backend or database

Never return 200 with an error inside. Monitoring counts error rates by status code, caches and CDNs may store a 200, and retry logic in clients and proxies looks only at the code. A 200 error is invisible to all of them.

3 One error envelope: Problem Details (RFC 9457)

Use the same error body everywhere. The standard is **Problem Details for HTTP APIs**, **RFC 9457**, which replaced the older RFC 7807. Its content type is `application/problem+json`. It defines five fields, and you may add your own extension fields, such as a machine-readable `code` and a list of field errors.

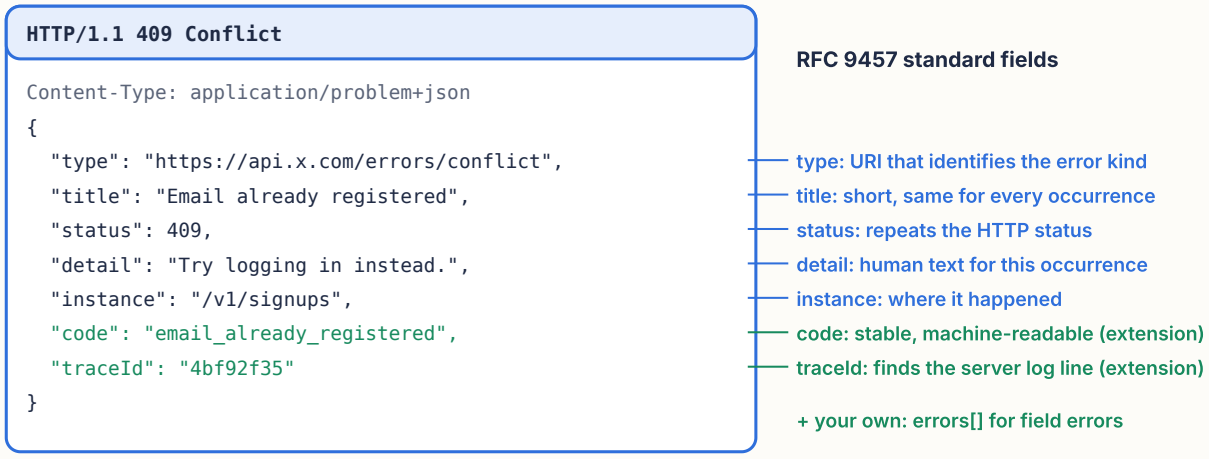


Figure 2. A Problem Details response for a duplicate signup. Blue: standard fields. Green: extension fields you add.

WORKED EXAMPLE: DUPLICATE SIGNUP

A user signs up with an email that already exists. Return **409 Conflict** with `"code": "email_already_registered"` and a friendly `detail` like “This email is already registered. Try logging in instead.”

The mobile app **switches on the code**, never on the message text: `if code == "email_already_registered"` → show the login screen. So you can reword or translate the message into Hindi or German tomorrow without breaking any client.

For a form with several bad fields, return them all in one response (status 422), so the user fixes everything in one round trip:

```
"errors": [
  { "field": "email",    "code": "invalid_format" },
  { "field": "password", "code": "too_short", "min": 12 }
]
```

4 Which errors should a client retry?

Retry only **transient** failures: problems that may disappear on their own. Always retry with exponential backoff and jitter, and with a cap on attempts, so retries don't pile onto a struggling server.



Figure 3. Retry rules. A 500 is ambiguous: the server may have done the work before failing, so a blind retry of a payment could charge twice.

- **Retry:** 502, 503, 504, timeouts and connection resets. These are usually a brief overload, deploy or network blip.
- **Retry 429 only after waiting** for the `Retry-After` delay. Retrying immediately makes the overload worse.
- **500 is ambiguous.** Retry it only if the operation is idempotent (GET, PUT, DELETE) or protected by an idempotency key.
- **Never blindly retry other 4xx** errors such as 400, 403 or 422. The same request will fail the same way. (401 is the one case where the client may refresh its token once and try again.)

5 Don't leak internals, and keep it consistent

Stack traces, SQL errors, hostnames and library versions in responses are a gift to attackers: they reveal your stack and which known vulnerabilities to try. Instead, **log the full details on the server with a trace ID**, and return only that ID to the client. When a user reports "trace 4bf92f35", support finds the exact log line in seconds.

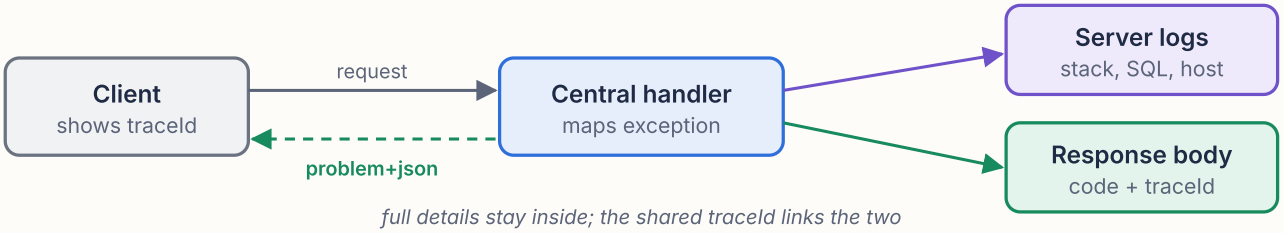


Figure 4. One central handler catches every exception, logs the full details, and returns a clean Problem Details body carrying the same trace ID.

Watch for existence leaks. If a caller asks for `/invoices/981` that belongs to another customer, a 403 confirms the invoice exists. Returning **404** hides that it exists at all. The same applies to login: "wrong email or password" instead of "no such email".

A few more habits:

- **Return all field errors at once**, not one per round trip.
- **Document every error code** in your API reference and keep codes stable. Renaming a code is a breaking change, like renaming a field.
- **Map exceptions to responses in one central handler** (middleware or an exception filter), so every endpoint speaks the same error language.

COMMON MISTAKES

- Returning 200 with `"success": false`: dashboards show 0% errors during an outage.
- Clients parsing the `detail` or `message` text instead of a stable code.
- Returning 500 for bad input. Validation failures are 400 or 422, and a flood of 500s pages the on-call engineer for a client bug.
- Retrying every error, including 400 and 403, or retrying a non-idempotent POST after a 500.

6 Key terms

4xx: client error: fix the request

5xx: server error: may work later

Problem Details: RFC 9457 error format

application/problem+json: its content type

Error code: stable machine-readable string

Trace ID: links a response to its log line

Transient error: may succeed on retry (502/503/504)

Retry-After: how long to wait before retrying

Existence leak: 403 reveals a hidden resource

7 Interview questions

Q1. What's the difference between a 4xx and a 5xx? BASIC

4xx: the client sent something wrong (400, 403, 404, 409, 422, 429); resending it unchanged won't help. 5xx: the server failed (500, 502, 503, 504); it may work later.

Q2. What should an error response contain? BASIC

The right status plus a Problem Details body: type, title, status, detail, instance, and extensions like a stable `code`, field errors and a trace ID. No stack traces or SQL.

Q3. A client calls your payment API and gets a 500. Should it retry? SCENARIO

Only if idempotent. The POST may have charged the card before failing, so a blind retry risks a double charge. With an `Idempotency-Key`, the retry returns the first attempt's stored result.

Q4. Your SDK should retry automatically. Which errors, and how? SCENARIO

502, 503, 504, timeouts and connection resets, with exponential backoff and jitter (say 200 ms, 400 ms, 800 ms, max 3 tries). 429 only after `Retry-After`. 500 only for idempotent calls. Never 400, 403, 404 or 422.

Q5. Is rewording an error message a breaking change? SCENARIO

Not if clients switch on the stable `code` field. The `detail` text is for humans and can change or be translated freely. Renaming the code, though, is breaking and needs a new version or deprecation.

Q6. A user requests another tenant's invoice. 403 or 404? TRICKY

Usually 404. A 403 confirms the invoice exists, which leaks information (an attacker can enumerate IDs). Return 404 for resources the caller isn't allowed to know about, and 403 when they can see it exists but can't perform this action.

Q7. 400 or 422 for a validation failure? TRICKY

400 for a request the server can't parse (broken JSON, wrong types). 422 for well-formed content that fails business rules (end date before start date). Many APIs use 400 for both; what matters is being consistent and returning field-level codes.

Next: SD-C-012 · Rate Limiting Algorithms backendroadmap.com

Rate Limiting Algorithms: Token Bucket vs Sliding Window

five ways to count, and which one to pick

Watch the 4-min video

> **In one line:** fixed window is cheap but allows double bursts at the boundary, sliding windows are stricter (exact with a log, approximate with two counters), token bucket allows controlled bursts with two numbers per key, and leaky bucket smooths traffic to a steady rate. For a public API, a token bucket per key is the usual choice.

1 The problem: how do you actually count?

Your public API allows **100 requests per minute per API key**, with short bursts of up to **20**. Simple to say. But to enforce it, every request needs a quick yes or no, and that answer depends on how you count. There are five classic algorithms. Each one trades three things differently: **accuracy**, **memory per key** and **burst behaviour**.

2 Fixed window counter

Keep one counter per key, per minute, for example under the key `rl:key42:12:00`. Increment it on each request, reject above 100, and start a new counter at the minute boundary. It is tiny (one integer) and fast (one increment).

But it has a **boundary problem**. The counter only knows which minute a request fell in, not how close together requests were.

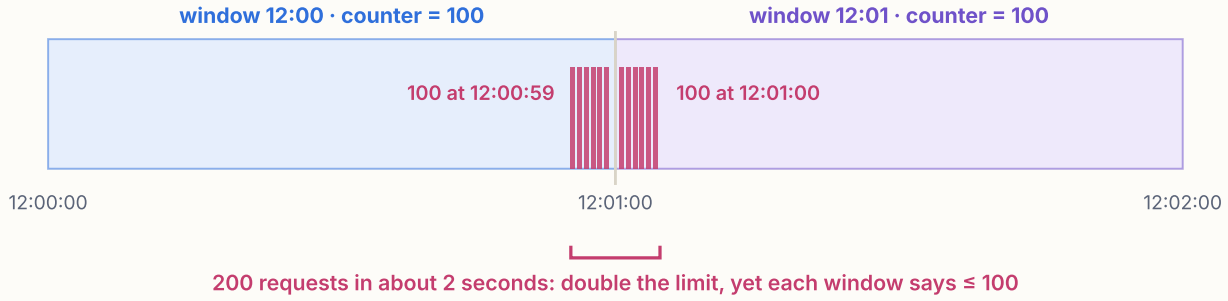


Figure 1. The fixed-window boundary burst. Each window stays within its limit, but 200 requests arrive within about two seconds.

3 Sliding window log and sliding window counter

The **sliding window log** fixes the boundary problem. Store the timestamp of every request in a sorted set. On each request, drop timestamps older than 60 seconds and count what's left. It is **perfectly accurate**: there are never more than 100 requests in any 60 seconds. But memory grows with the limit: up to 100 timestamps per key. With a million keys and higher limits, that is hundreds of millions of entries.

The **sliding window counter** is the practical compromise. Keep just two counters: the current window and the previous one. Then estimate the last 60 seconds by weighting the previous counter by how much of it still overlaps the sliding window:

```
estimate = current + previous × (1 − elapsed / window)
```

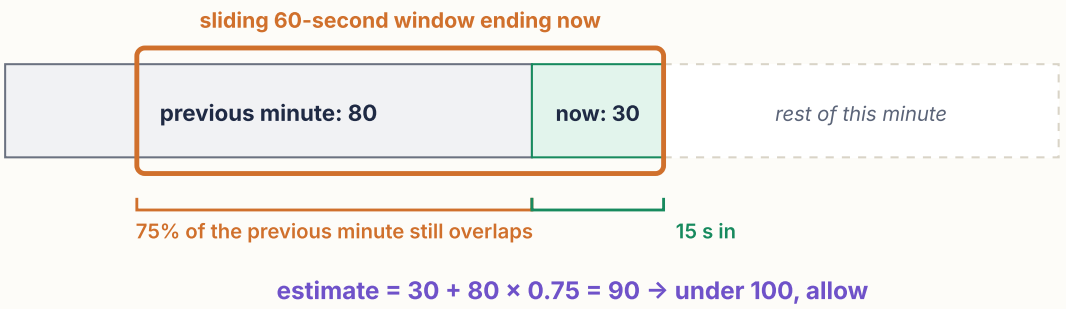


Figure 2. 15 seconds into this minute, 45 of the previous 60 seconds still fall inside the sliding window, so the previous count is weighted by 0.75.

It assumes the previous minute's requests were spread evenly, so it is an approximation, but accurate to within a few percent in practice, with only two counters per key.

4 Token bucket and leaky bucket

The **token bucket** thinks differently. Each key has a bucket that holds up to **20 tokens** and refills at **100 per minute** (one token every 0.6 s). Each request takes one token; with none left, it is rejected with 429. A quiet client builds up a full bucket, so it can burst 20 requests at once, but over time it can never exceed the refill rate. You store only **two numbers**: tokens left and the last refill time. Refill is computed lazily on each request: `tokens = min(20, tokens + elapsed × rate)`, so no background timer is needed.

The **leaky bucket** is the mirror image. Requests join a queue that drains at a constant rate, say one request every 600 ms. If the queue is full, new requests are dropped. It smooths traffic perfectly, which protects a fragile downstream service, but bursts wait in line instead of being served right away.

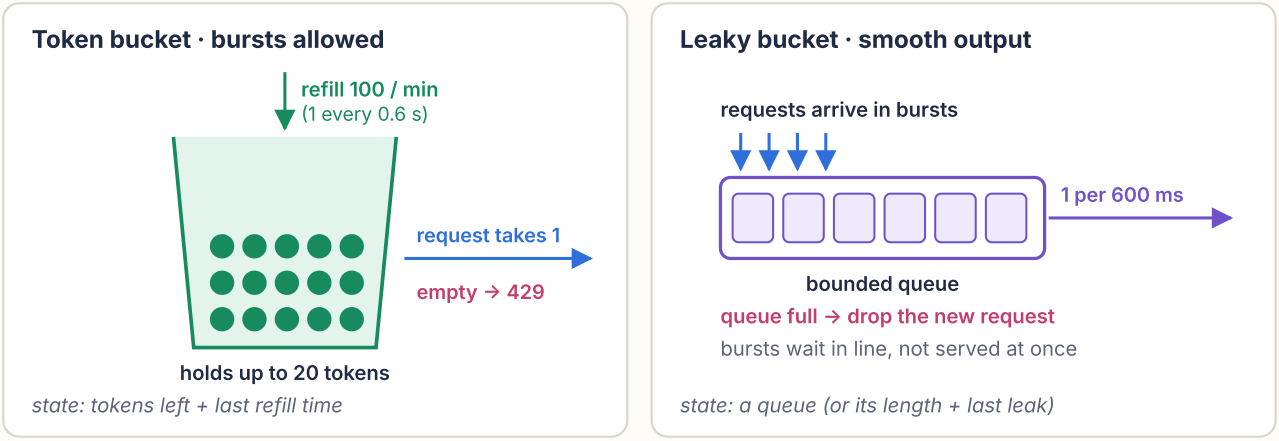


Figure 3. Left: the token bucket stores capacity for bursts. Right: the leaky bucket turns bursty input into a steady output rate.

WORKED EXAMPLE: ONE KEY, TWO ALGORITHMS

A client has been idle, so its bucket is full (20 tokens). It sends 25 requests in one second. The first **20 succeed**, the next 5 get **429**. After that it earns one token every 0.6 s, so about 1.7 requests per second.

Over its first full minute it can make at most **20 + 100 = 120** requests: the stored burst plus the refill. That is the honest summary of a token bucket: *the rate over the long run, plus up to one bucket of extra at any moment*.

With a sliding window counter instead, the same client gets at most 100 in any 60 seconds, all of which it may spend in the first second; there is no separate burst size to tune.

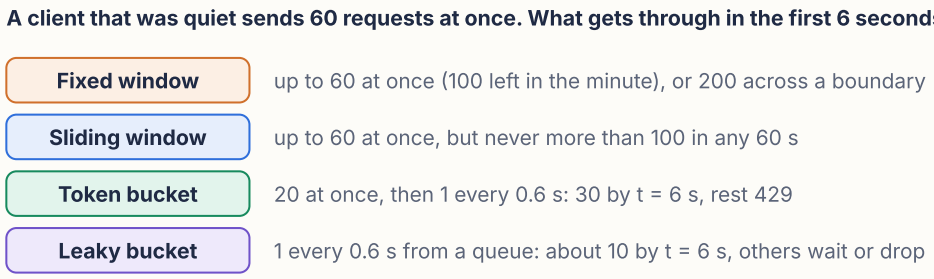


Figure 4. The same burst under each algorithm (limit 100 per minute, bucket size 20, leak one every 0.6 s).

5 Which one for a public API?

Algorithm	State per key	Accuracy	Bursts	Use when
Fixed window	1 counter	Up to 2× at edges	Double burst at boundary	Approximate is good enough
Sliding log	1 timestamp per request	Exact	None beyond the limit	Low limits, must be exact
Sliding counter	2 counters	Within a few %	None beyond the limit	Strict per-minute limits, cheaply
Token bucket	tokens + timestamp	Exact rate	Up to bucket size	Public APIs (most common)
Leaky bucket	queue	Exact output rate	Queued, not served	Downstream needs a steady rate

The **token bucket** is the usual choice. It allows the friendly bursts that real clients make (a page load firing ten calls at once), it is cheap, and it is easy to explain as “a rate plus a burst”. AWS API Gateway and Stripe both use token buckets. The **sliding window counter** is a great alternative when you want a strict per-minute limit without the fixed window's double burst. Use a **leaky bucket** when a consumer needs a steady rate, and a **fixed window** only when approximate is good enough.

COMMON MISTAKES

- Using a fixed window and quoting the limit as exact: a client can get twice the limit across a boundary.
- Storing a sliding log for high limits: 10,000 requests per minute means 10,000 timestamps per key.
- Running a background job to refill every bucket. Compute the refill lazily from the elapsed time.

6 Key terms

Fixed window: one counter per key per time window

Sliding counter: two weighted counters, approximate

Lazy refill: compute tokens from elapsed time

Boundary burst: 2× the limit across a window edge

Token bucket: refill rate + bucket size (burst)

429: Too Many Requests

Sliding log: exact, one timestamp per request

Leaky bucket: queue drained at a fixed rate

Burst: short spike above the average rate

7 Interview questions

Q1. Compare token bucket and sliding window. BASIC

Token bucket stores tokens and a timestamp, refills at a steady rate and allows bursts up to the bucket size. Sliding window counts requests in the last N seconds, exactly with a log or approximately with two weighted counters: stricter, smoother, no burst allowance.

Q2. Which would you use for a public API, and why? BASIC

A token bucket per API key, for example 100 per minute with a burst of 20. Real clients are bursty, it needs two numbers per key, and “rate plus burst” is easy to document.

Q3. Sliding window counter: previous minute 80 requests, current 30, 15 s in, limit 100. Allow? SCENARIO

Estimate = 30 + 80 × (1 − 15/60) = 30 + 60 = 90. Under 100, so allow.

Q4. Token bucket, capacity 20, refill 100/min. A full bucket gets 25 requests in one second. What happens? SCENARIO

The first 20 succeed and 5 get 429. Then about one token every 0.6 s, so the next request succeeds 0.6 s later.

Q5. A downstream SMS provider accepts at most 10 messages per second. Which algorithm? SCENARIO

A leaky bucket (a queue drained at 10/s). Bursts of messages wait in the queue instead of being rejected or overloading the provider; reject only when the queue is full.

Q6. Memory for 1 million keys at 1,000 requests/minute: sliding log vs token bucket? TRICKY

Sliding log keeps up to 1,000 timestamps per key: up to a billion entries, tens of GB in Redis. Token bucket keeps two numbers per key: about 2 million values, tens of MB. That is why logs are only used for low limits.

Q7. Can a token bucket let through more than the stated per-minute rate? TRICKY

Yes, in a single minute: capacity + rate × time. With 20 + 100, up to 120 in the first minute after a quiet period. Over the long run it averages the refill rate.

Next: SD-C-013 · Distributed Rate Limiting backendroadmap.com

Distributed Rate Limiting: Redis, Lua & 429 Headers

one limit across thirty servers, and what to do when Redis is down

Watch the 4-min video

In one line: keep per-tenant counters in a shared store like Redis, update them atomically with a Lua script (optionally with local buckets that sync in batches), reject with 429 plus Retry-After and rate limit headers, and decide deliberately whether to fail open or closed when Redis is down.

1 Why local counters are not enough

Your API gateway runs **30 instances** behind a load balancer. Each tenant may make **1,000 requests per minute**. If every instance counts on its own, a tenant whose traffic spreads across all thirty can actually make **30,000**. The limit only works if the instances **share state**.



Figure 1. Left: 30 independent counters allow 30× the limit. Right: one shared counter per tenant in Redis enforces the real limit.

2 A shared counter in Redis, and its races

The classic answer is a shared counter in Redis. For a fixed window, each instance runs `INCR` on a key built from the tenant and the current minute, such as `rl:tenant42:202611041200`. If the result is 1 (the first request this minute), it also runs `EXPIRE` so the key cleans itself up. If the count goes over 1,000, it returns 429. `INCR` itself is atomic, so concurrent increments never get lost.

But there are two races:

- **INCR and EXPIRE are two separate commands.** If an instance crashes (or the connection drops) between them, the key never expires. Its count stays above 1,000, and that tenant stays blocked forever.
- **Read-then-write logic** (read the count, decide in the app, then write) lets two instances both read 999 and both allow.

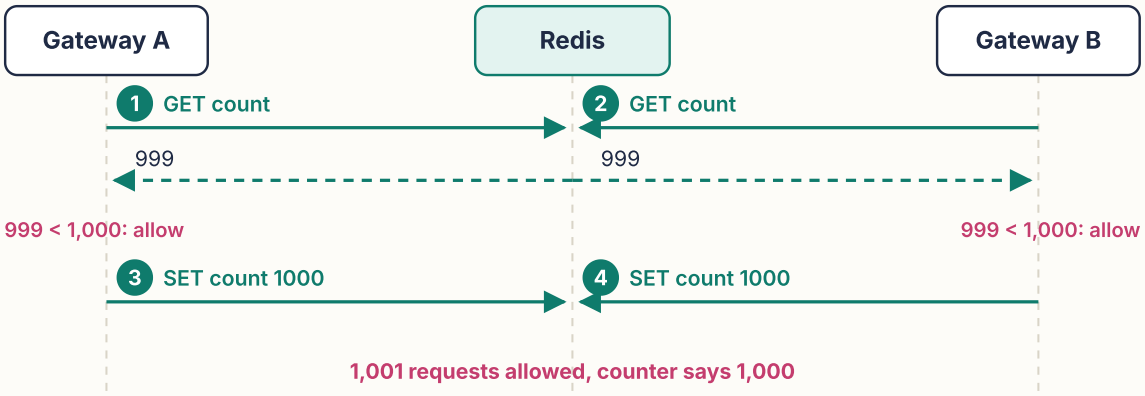


Figure 2. The read-then-write race. Both gateways decide on the same stale value, so the limit is exceeded. Under real load, many requests slip through this gap.

3 The fix: atomicity with a Lua script

Put the whole algorithm in a **Lua script**. Redis runs a script as one indivisible operation: no other command runs on the server while it executes. The script reads the bucket, refills tokens, takes one, sets the TTL and returns allowed or denied, all in **one network round trip**. Load it once with `SCRIPT LOAD` and call it by its hash with `EVALSHA`, so you don't resend the code each time.

```
-- token bucket: KEYS[1] = "rl:{tenant42}"   ARGV = capacity, tokens per ms, now (ms)
local cap, rate, now = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or cap            -- new key: start with a full bucket
local ts     = tonumber(b[2]) or now
tokens = math.min(cap, tokens + math.max(0, now - ts) * rate)   -- lazy refill
local allowed = 0
if tokens >= 1 then tokens = tokens - 1; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate))  -- idle key cleans itself up
return {allowed, math.floor(tokens)}
```

With **Redis Cluster**, keys are sharded by tenant, so the load spreads across nodes. A script may only touch keys in one hash slot; a hash tag like `{tenant42}` keeps all of one tenant's keys together. Pass the time in from the gateway, or read it inside the script with `redis.call('TIME')` so clock skew between gateways can't distort the refill.

4 Cutting the cost: local buckets that sync

A Redis call on every request adds a round trip (often 0.5–1 ms in the same zone) and makes Redis a hot dependency on the critical path. Two common optimizations:

- **Hybrid local + global.** Each instance keeps a local token bucket and syncs with Redis in batches, for example claiming **50 tokens at a time**. Only one request in fifty touches Redis: at 60,000 requests per second, Redis sees about 1,200 calls per second instead of 60,000.
- **Split the limit.** With 30 instances, each allows about 1,000 / 30 ≈ **33 per minute** locally, with no Redis at all. It works well only when the load balancer spreads each tenant's traffic evenly.

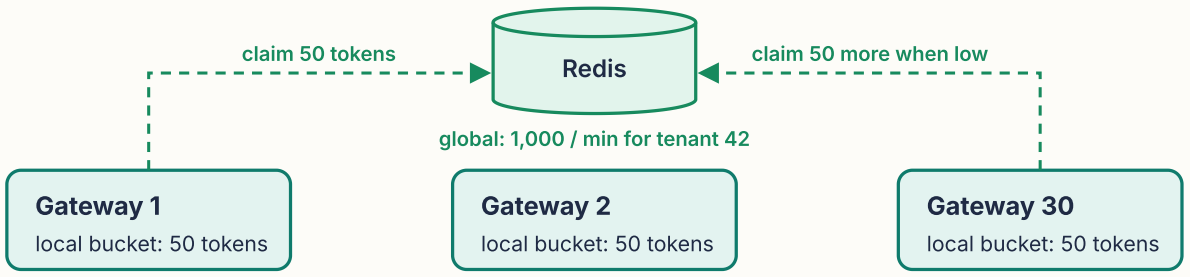


Figure 3. Hybrid limiting: gateways spend tokens locally and claim a new batch from Redis when they run low.

You trade a little accuracy for speed and resilience. Claimed tokens are already deducted in Redis, so the long-run total stays right, but timing blurs: up to 30 × 50 = 1,500 tokens can sit parked on gateways. One gateway may reject a tenant while another holds spare tokens, and parked tokens can be spent later in a burst.

5 Tell clients what's happening

When you reject, return **429 Too Many Requests** with a `Retry-After` header in seconds. On **every** response, include the limit, the remaining count and the reset time, so well-behaved clients slow down before they hit the wall:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1762257660      <- when the window resets (Unix time)

# newer IETF draft (httpapi-ratelimit-headers) style:
RateLimit-Policy: "default";q=1000;w=60
RateLimit: "default";r=0;t=12
```

The `X-RateLimit-*` headers are a widespread convention (GitHub and many others use them). An IETF draft standardizes them as `RateLimit` and `RateLimit-Policy`.

6 What if Redis is down?

	Fail open	Fail closed
Behaviour	Allow requests; local in-memory limits act as a backstop	Reject requests (429 or 503)
Protects	Availability of your API	A fragile backend from overload
Cost	Limits are approximate for a while	A Redis outage becomes a full API outage
Typical for	Most public APIs	Expensive or abuse-prone paths (login, SMS sending, payments)

Most public APIs **fail open with local fallback limits, and alert loudly**. Either way, put a **short timeout** on the Redis call (a few milliseconds) and a **circuit breaker** around it, so a slow Redis can't slow down every request. A hung Redis is worse than a dead one if every request waits a full second for it.

COMMON MISTAKES

- Rate limiting in each instance's memory and calling it a global limit.
- Running `INCR` and `EXPIRE` as separate calls with no recovery: a crash in between blocks a tenant forever.
- Returning 429 without `Retry-After`, so clients retry immediately and make it worse.

7 Key terms

Shared state: one counter all instances see

INCR: atomic Redis increment

Lua script: runs atomically inside Redis

EVALSHA: call a loaded script by hash

Hash tag: {...} keeps keys in one cluster slot

Hybrid limiter: local buckets + batched sync

Retry-After: seconds to wait before retrying

Fail open: allow when the limiter is down

Fail closed: reject when the limiter is down

8 Interview questions

Q1. How do you rate limit across many servers? BASIC

Keep counters in a shared store like Redis, keyed per tenant, updated atomically with a Lua script. Optionally use local buckets that sync in batches. Reject with 429, Retry-After and rate limit headers.

Q2. Why use a Lua script instead of separate Redis commands? BASIC

Redis runs the script atomically, so read-refill-take-write can't interleave with another gateway, and it costs one round trip. Separate commands leave gaps: two gateways read 999 and both allow, or a crash between INCR and EXPIRE leaves a key that never expires.

Q3. You can't afford a Redis call per request. What do you do? SCENARIO

Hybrid: each gateway claims tokens from Redis in batches (say 50) and spends them locally, cutting Redis calls 50×. Or split the limit: about 33 per gateway per minute, if traffic is evenly balanced. Accept slightly blurred accuracy.

Q4. What happens if Redis is down? SCENARIO

Usually fail open with local per-instance limits as a fallback, alert, and guard the call with a short timeout and a circuit breaker. Fail closed only where protecting the backend matters more, such as login or SMS sending.

Q5. A tenant stays blocked for hours after they stopped sending. Why? SCENARIO

A counter key with no TTL: INCR ran, then the app crashed before EXPIRE, so the count stays over the limit forever. Do INCR and EXPIRE in one Lua script.

Q6. Redis becomes slow (200 ms per call) but not down. What breaks? TRICKY

Every request now waits 200 ms more, so API latency and in-flight requests jump (Little's Law), and pools may run dry. A tight timeout (say 5 ms) plus a circuit breaker turns slow into “unavailable”, so the fail-open path kicks in.

Q7. Is the hybrid (batched local tokens) design exact? TRICKY

Not quite. The long-run total holds, but up to 30 × 50 = 1,500 tokens can be parked on gateways: one may reject the tenant while others hold spares, or spend them in a later burst. Smaller or expiring batches shrink the error, at the cost of more Redis calls.

Next: SD-C-014 · Throttling vs Rate Limiting vs Quotas backendroadmap.com

Throttling vs Rate Limiting vs Quotas

free and paid tiers, monthly limits, and fairness

Watch the 3-min video

> **In one line:** rate limiting rejects requests above a short-term rate with 429, throttling slows or queues them instead, load shedding drops low-priority work when the server is overloaded, and quotas are long-term per-plan budgets that need durable, billing-grade counting.

1 Why a per-second limit isn't enough

Your API has a **free tier** with 1,000 requests per day and a **paid tier** with 1,000,000 per month. A free user writes a loop and burns the whole day's allowance in ten minutes. A paying customer runs a giant batch at month end. A per-second rate limit catches neither problem on its own. You need three tools: **throttling**, **quotas** and **tiers**.

2 The words: rate limiting, throttling, load shedding

- **Rate limiting** enforces a short-term rate, like 100 per minute, and **rejects** the excess with `429 Too Many Requests`.
- **Throttling** usually means **slowing down** instead of rejecting: delaying or queuing requests, so the work still completes, just later. (Many people and docs use "throttling" loosely to mean rate limiting; say which one you mean.)
- **Load shedding** is driven by the **server's health**, not the client's limit. When the system is overloaded, it drops low-priority work to protect everything else, even if every client is within its limits.

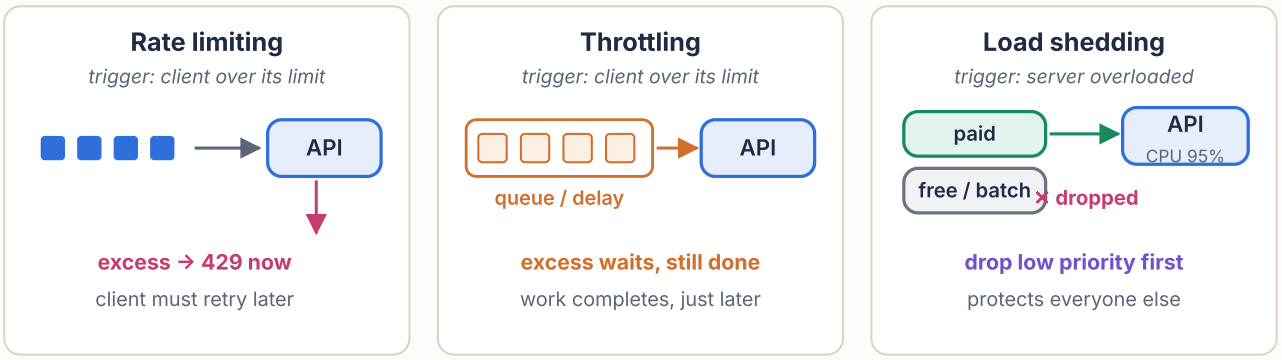


Figure 1. Same excess traffic, three responses. The first two are about one client's limit; load shedding is about the server's own health.

3 Quotas: long-term budgets tied to billing

Quotas are budgets per day or per month, usually tied to the customer's plan and invoice. Because they cost money, they need **durable, accurate** counting. A lost counter is not just a glitch: it is free usage you can't bill, or a customer blocked by mistake.

	Rate limit	Quota
Time scale	Seconds to minutes	Days to months
Purpose	Protect the system from bursts	Enforce the business plan
Counting	Fast, approximate is OK, losing state is OK	Durable, exact, must survive failures
When exceeded	429, retry in seconds	429 or 402, or overage billing, until the next period

4 Enforcing a monthly quota

1. Keep a **fast counter** per tenant, per billing period, in Redis (for example `usage:t42:2026-11`), and check it at the gateway on every request.
2. In the background, **stream usage events** to a durable store. That store, not Redis, is the **source of truth for billing**.
3. **Reconcile** the Redis counter from the durable store periodically, so losing a Redis node never resets anyone to zero.
4. **Reset at each tenant's own billing anchor date** (the day they subscribed), not at midnight on the 1st for everyone.

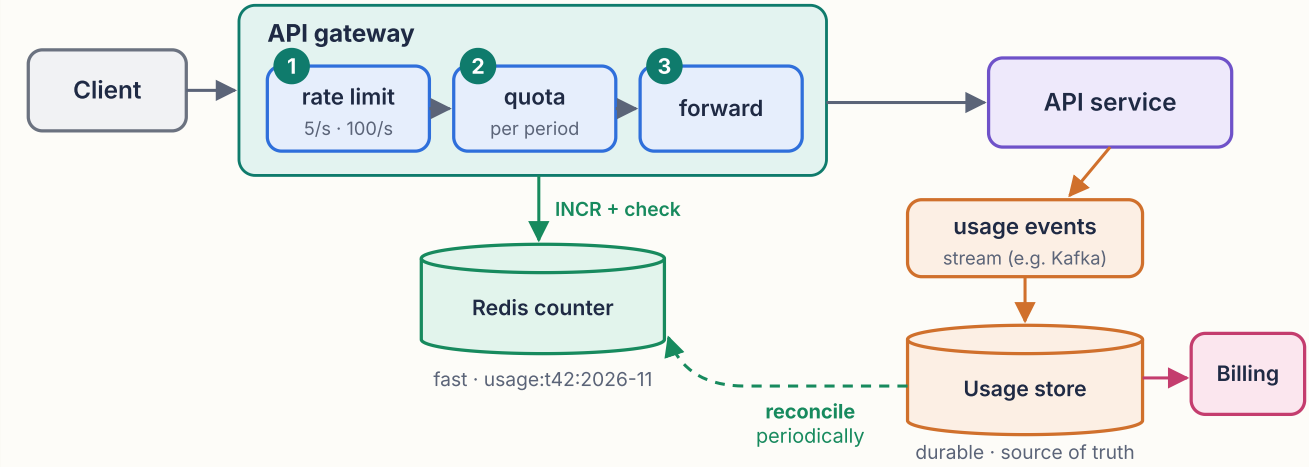


Figure 2. The gateway checks the rate limit (1), then the quota (2), then forwards (3). Redis answers fast; the event stream and usage store keep billing exact.

5 Make limits humane, and fair

Warn at 80% and 100%, by email and in response headers, so nobody is surprised. Then choose per plan:

- **Hard limit:** return 429, or `402 Payment Required`, with a link to upgrade. Right for the free tier.
- **Soft limit:** let paying customers go over and bill the overage. Right for paid tiers. **Never cut off a paying customer silently.**

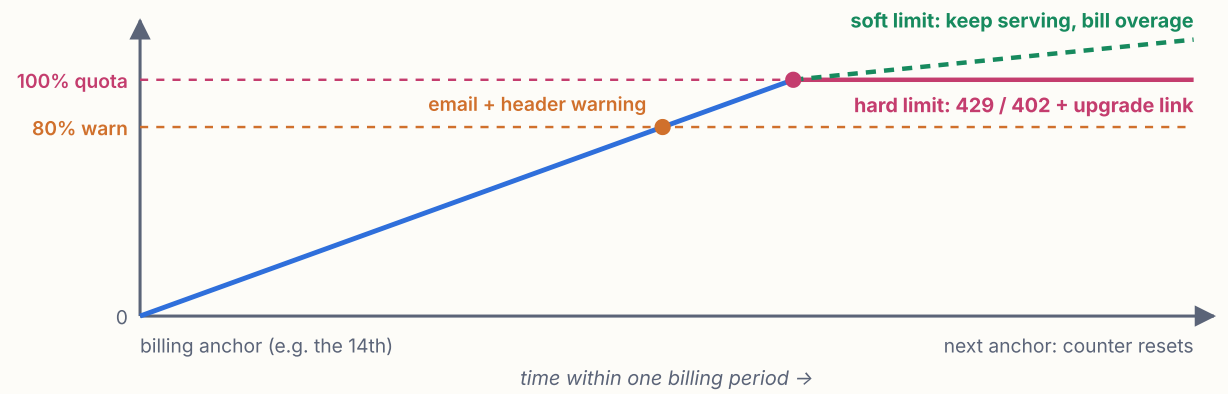


Figure 3. One billing period. A warning at 80%, then a hard stop (free tier) or continued service with overage billing (paid tier).

Tiers also decide fairness under pressure. Give each tier its own rate, burst and quota. Put paid traffic in a separate queue or worker pool, so a noisy free user can never starve paying customers. And when the system is overloaded, shed background jobs and the free tier first.

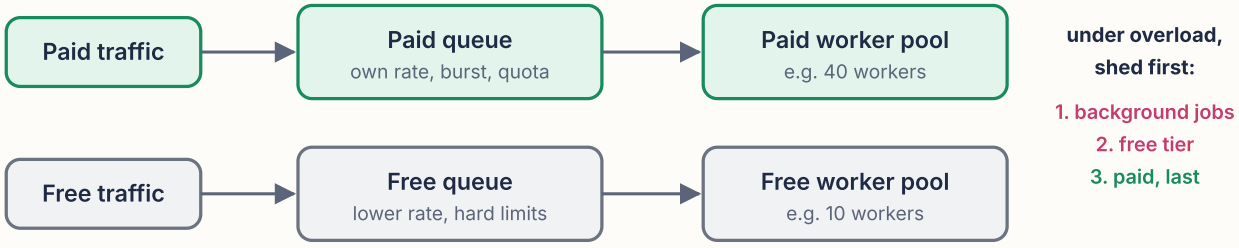


Figure 4. Isolating tiers. Separate queues and worker pools mean one tier's spike can't use up another tier's capacity.

WORKED EXAMPLE: PUTTING IT TOGETHER

	Free tier	Paid tier
Quota	1,000 requests / day	1,000,000 requests / month
Rate limit	5 / second	100 / second, with bursts
Over quota	Hard limit: 429 + upgrade link	Soft limit: overage billing

The free user's loop: at 5 per second, 1,000 requests take about 3 minutes 20 seconds, then every call gets 429 until tomorrow. The rate limit slowed them down; the quota stopped them. The paid customer's month-end batch: at 100 per second, 1,000,000 requests take under 3 hours of full-speed use. They get warnings at 800,000 and 1,000,000, then keep going and pay for the overage.

COMMON MISTAKES

- Keeping quota counts only in Redis: a node failure resets customers to zero, or loses billable usage.
- Resetting every tenant's quota on the 1st instead of their own billing anchor.
- Cutting off a paying customer with a bare 429 and no warning.
- Running all tiers through one shared queue, so free traffic starves paid traffic.

6 Key terms

Rate limiting: reject above a short-term rate (429)

Throttling: slow or queue instead of reject

Load shedding: drop low-priority work when overloaded

Quota: long-term budget per plan

Billing anchor: the tenant's own period start

Reconcile: correct the fast counter from the durable store

Hard limit: stop at the quota

Soft limit: go over, bill overage

402: Payment Required

7 Interview questions

Q1. How do throttling and rate limiting differ? BASIC

Rate limiting rejects requests above a short-term rate with 429. Throttling slows or queues them, so the work still completes, later. Load shedding is a third thing: dropping low-priority work based on server health.

Q2. How are quotas different from rate limits? BASIC

Rate limits protect the system over seconds and can tolerate approximate counting. Quotas enforce the business plan over days or months, drive billing, and need durable, exact counts.

Q3. How do you enforce a monthly quota of 1,000,000 requests? SCENARIO

A fast per-tenant, per-period counter in Redis checked at the gateway; usage events streamed to a durable store as the billing source of truth; periodic reconciliation of Redis from it; reset at each tenant's billing anchor; warnings at 80% and 100%; a hard or soft limit per plan.

Q4. A Redis node holding quota counters fails. What happens to customers? SCENARIO

With reconciliation, the counter is rebuilt from the durable usage store, so nobody resets to zero. Usage during the gap is still recorded in the event stream, so billing stays exact; at worst a tenant goes a little over their quota before the counter catches up.

Q5. Free users are slowing down paying customers during spikes. Fix? SCENARIO

Isolate tiers: separate queues or worker pools and per-tier rate limits, so free traffic can only fill its own capacity. Under overload, shed background jobs and the free tier first.

Q6. A free user is at 5 requests per second. Why do they still need a daily quota? TRICKY

5 per second is 432,000 per day. Without a quota, a loop running all day stays within the rate limit but uses 432 times the free allowance of 1,000. The rate limit shapes speed; the quota caps total consumption.

Q7. When would you return 402 instead of 429? TRICKY

429 says “too fast, retry later”, which suits rate limits. When the monthly quota is exhausted, waiting a few seconds won't help; 402 Payment Required (with an upgrade link) tells the client the fix is a plan change. Some APIs use 429 for both, with a clear error code.

Next: SD-C-015 · API Authentication backendroadmap.com

API Authentication: API Keys vs JWT vs HMAC vs mTLS

five ways to prove who is calling, and where each fits

Watch the 4-min video

In one line: API keys identify an app but are long-lived bearer secrets; session cookies suit browsers; bearer tokens (JWT or opaque) suit users and services; HMAC signing proves the sender and the body without sending the secret; mutual TLS proves service identity at the connection level.

1 Different callers, different proofs

A mobile app, a partner's backend, a webhook from a payment provider and an internal service all call your API. Each one has to prove who it is, but no single method fits all of them. There are five common methods: a browser or mobile app usually logs a user in (session cookie or OAuth token), a partner backend uses OAuth client credentials or HMAC, a webhook carries an HMAC signature, and an internal service uses mutual TLS.

2 API keys and session cookies

API keys are the simplest: a long random string sent in a header, such as `Authorization: Bearer sk_live_…` or `X-API-Key`. A key identifies the **calling application**, which is great for usage tracking and rate limits. But it is a **bearer secret**: anyone who sees it can use it, it rarely expires, and it doesn't identify a user. So:

- **Store only a hash** on the server (like a password), so a database leak doesn't expose usable keys. Show the full key to the customer once.
- **Support rotation**: allow two active keys at once, so a customer can switch without downtime.
- **Never ship a key inside a mobile app** or front-end code. Anyone can extract it from the app package.
- Send it in a header, never in the URL, where it ends up in logs and browser history.

Session cookies are the classic for browsers. After login, the server stores a session and sends a cookie holding a random session ID. Mark it `HttpOnly` (scripts can't read it, which blunts XSS), `Secure` (HTTPS only) and `SameSite` (other sites can't send it, which blunts CSRF). Sessions are easy to revoke (delete the row), but they need a session store, and cookies don't suit server-to-server calls.

3 Bearer tokens: JWT vs opaque

Bearer tokens, usually issued by an **OAuth 2** authorization server, travel in the `Authorization: Bearer …` header. There are two kinds:

- A **JWT** is self-contained and signed. Any service can verify it locally with the issuer's public key, with no database lookup. But it is hard to revoke before it expires, so keep lifetimes short, like **15 minutes**, and use refresh tokens to get new ones.
- An **opaque token** is just a random reference. The server must look it up (or call the auth server's introspection endpoint), but revocation is instant.

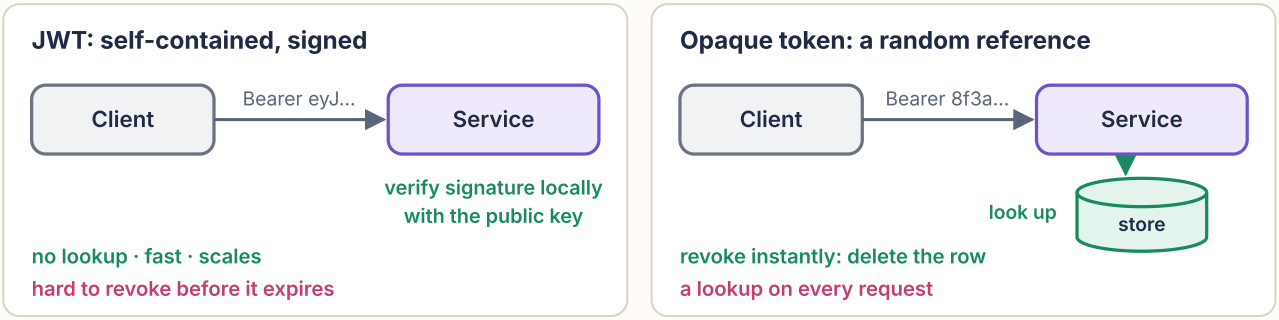


Figure 1. The core trade-off: JWTs save a lookup on every request; opaque tokens make revocation immediate.

For server-to-server calls with no user, OAuth 2 has the **client credentials** flow: the service authenticates with its client ID and secret and gets a short-lived access token. That beats a long-lived static key, because a leaked token stops working within minutes.

4 HMAC request signing

HMAC signing goes a step further. Client and server share a secret, but **the secret is never sent**. The client computes an **HMAC-SHA256** over the method, path, timestamp and body, and sends only the signature. The server recomputes it with its copy of the secret and compares.

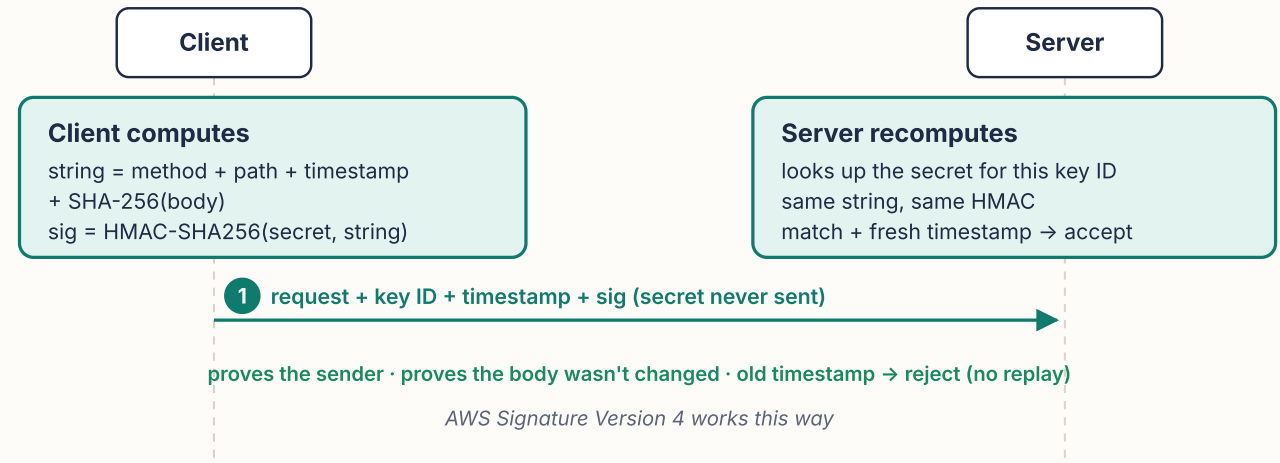


Figure 2. HMAC request signing. Changing a single byte of the body or path produces a completely different signature.

That proves **who sent the request** and that **nobody changed it in transit**. Rejecting old timestamps blocks **replay attacks**, where an attacker resends a captured request. AWS Signature Version 4 works this way (it also signs selected headers and derives a per-day, per-region signing key from the secret).

WORKED EXAMPLE: A WEBHOOK RECEIVER

A payment provider posts a `payment.succeeded` event to your endpoint with a signature header. To accept it safely:

1. Read the **raw body**, before any JSON parsing. Re-serialized JSON may differ by a single space, and the signature won't match.
2. Compute the HMAC with your webhook secret, over the timestamp and the body.
3. Compare using a **constant-time comparison**, to avoid timing attacks (a normal `==` stops at the first wrong byte, which leaks how much matched).
4. Reject timestamps older than **5 minutes**.

5. Only then process the event, **idempotently**, by its event ID: providers retry, so the same event can arrive twice.

```
import hmac, hashlib, time

def verify_webhook(raw_body: bytes, ts: str, sig: str, secret: bytes) -> bool:
    if abs(time.time() - int(ts)) > 300:          # older than 5 minutes: possible replay
        return False
    signed = ts.encode() + b"." + raw_body          # exact bytes, BEFORE any JSON parsing
    expected = hmac.new(secret, signed, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sig)       # constant-time comparison

# then: if event_id already processed -> return 200 and skip (idempotent)
```

5 Mutual TLS

In normal TLS only the server shows a certificate. In **mutual TLS (mTLS)**, both sides present certificates during the handshake, so the server knows exactly which service is calling **before reading a single byte of HTTP**. It is the strongest option for service-to-service traffic, and service meshes like **Istio** or **Linkerd** issue and rotate short-lived certificates automatically (by default they last about a day).

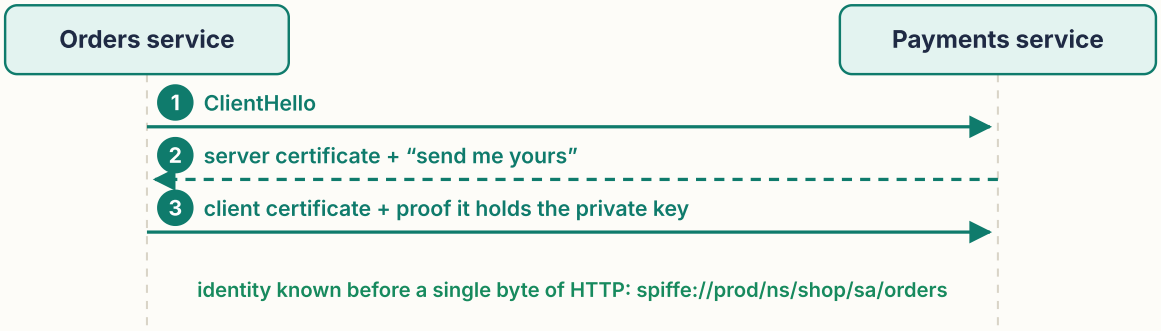


Figure 3. Mutual TLS. The caller's identity comes from its certificate, here a SPIFFE ID naming the service.

Method	Identifies	Secret sent?	Revocation	Best for
API key	The calling app	Yes, every request	Delete the key	Simple public APIs, usage tracking
Session cookie	A user in a browser	Session ID, every request	Instant (delete session)	Web apps
JWT	User or service + claims	Token, every request	Hard: wait for expiry	Many services verifying locally
Opaque token	User or service	Token, every request	Instant	When revocation matters
HMAC signing	Key holder + body integrity	**No**, only a signature	Rotate the secret	Webhooks, partner APIs
mTLS	The calling service	**No**, private key stays local	Short-lived certs	Internal service-to-service

COMMON MISTAKES

- Shipping an API key in a mobile app or JavaScript bundle.
- Long-lived JWTs (days or weeks) with no way to revoke them.
- Verifying a webhook signature over parsed and re-serialized JSON, or comparing signatures with `==`.
- Signing requests without a timestamp, so a captured request can be replayed forever.

6 Key terms

Bearer secret: whoever holds it can use it

Client credentials: OAuth 2 flow for services

Constant-time compare: takes the same time on any mismatch

JWT: signed, self-contained token

HMAC: keyed hash proving sender + integrity

mTLS: both sides present certificates

Opaque token: random reference, looked up

Replay attack: resending a captured request

Service mesh: automates mTLS between services

7 Interview questions

Q1. How would you authenticate server-to-server calls? BASIC

Mutual TLS, or OAuth 2 client credentials with short-lived tokens, or HMAC-signed requests, rather than long-lived static API keys alone. Inside a mesh, mTLS is usually automatic.

Q2. Why sign requests with HMAC? BASIC

It proves both the sender and the integrity of the body without ever sending the secret. With a timestamp in the signed string, and a window like 5 minutes, it also prevents replay attacks.

Q3. Your mobile app needs to call your API. Can it use an API key? SCENARIO

No: any key in the app can be extracted. Have the user log in and use OAuth 2 (authorization code with PKCE) to get short-lived access tokens tied to that user. Use API keys only for server-side callers.

Q4. How do you verify a payment provider's webhook? SCENARIO

Read the raw body before parsing, compute HMAC-SHA256 over timestamp + body with the webhook secret, compare in constant time, reject timestamps older than 5 minutes, then process idempotently by event ID.

Q5. A customer's API key leaked on GitHub. What should your design already support? SCENARIO

Hashed storage (the leak isn't in your DB), instant revocation, two active keys per customer so they can rotate without downtime, per-key scopes to limit damage, and usage logs per key to see what the attacker did.

Q6. Why compare HMAC signatures in constant time? TRICKY

A normal comparison returns as soon as one byte differs, so its timing reveals how many leading bytes were right. An attacker can guess the signature byte by byte. `hmac.compare_digest` takes the same time regardless.

Q7. With mTLS everywhere, do you still need authorization? TRICKY

Yes. mTLS proves which service is calling; it doesn't say what that service may do, or on behalf of which user. You still need policies (orders may call payments' charge endpoint, not refunds) and user context, such as a forwarded token.

Next: SD-C-016 · API Authorization & BOLA backendroadmap.com

API Authorization & BOLA

scopes, ownership checks and where they live

Watch the 4-min video

In one line: a valid token proves who the caller is, not what they may touch; check scopes at the gateway, but check that the caller owns *this* object in the service, in the query itself, on every request.

1 Authentication is not authorization

User 42 logs in, gets a valid token, and calls `GET /orders/1001`. That order is theirs. Then they change the number to 1002 and get someone else's order: name, address, everything. The token was valid, so **authentication** worked. **Authorization** did not.

Authentication answers *who are you?* Authorization answers *what are you allowed to do, to this specific thing?* A valid token only proves identity. Every request still needs a decision: may this **caller** perform this **action** on this **resource**?

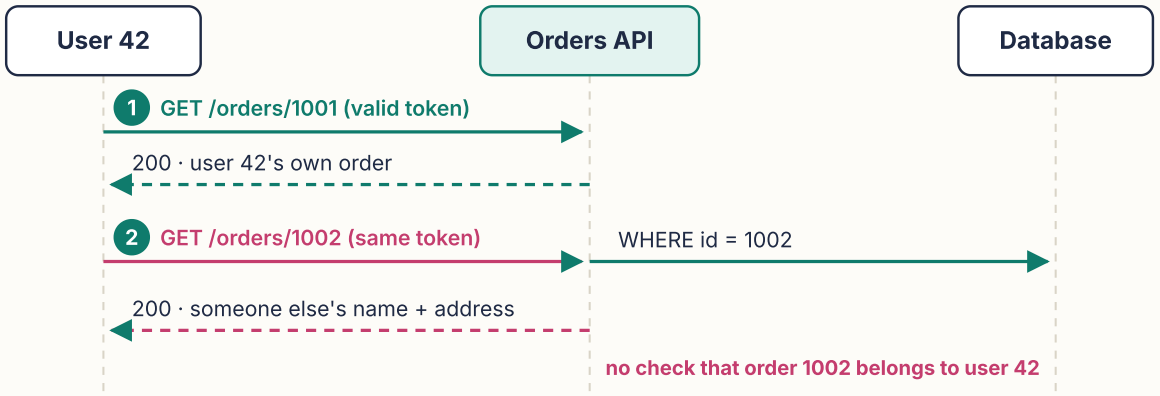


Figure 1. The token is valid both times. The API looks up order 1002 by ID alone and hands it over. This is BOLA, the top API security risk.

2 Scopes: the coarse layer

An OAuth token can carry **scopes** such as `orders:read` or `orders:write`. The gateway can reject a token without `orders:read` before the request even reaches the service.

But scopes describe **kinds of actions**, not **which records**. A token with `orders:read` can still read *any* order, unless the service checks ownership. Scopes narrow what a token can do; they never answer "is this order yours?".

3 BOLA: the missing ownership check

BOLA (Broken Object Level Authorization) is number one in the **OWASP API Security Top 10**. It is also called **IDOR** (Insecure Direct Object Reference). The API takes an object ID from the client and trusts it, without checking that the caller may access that object.

The fix lives in the service, at the data access. Load the order where the ID matches *and* the owner matches the caller's user ID. Take that user ID from the verified token, **never from the request body** or a query parameter the client controls. If nothing comes back, return **404**, so you don't even reveal that order 1002 exists.

In **multi-tenant** systems, also scope every query by tenant ID, ideally automatically: with database **row-level security**, or a repository layer that always adds the filter, so no developer can forget it.

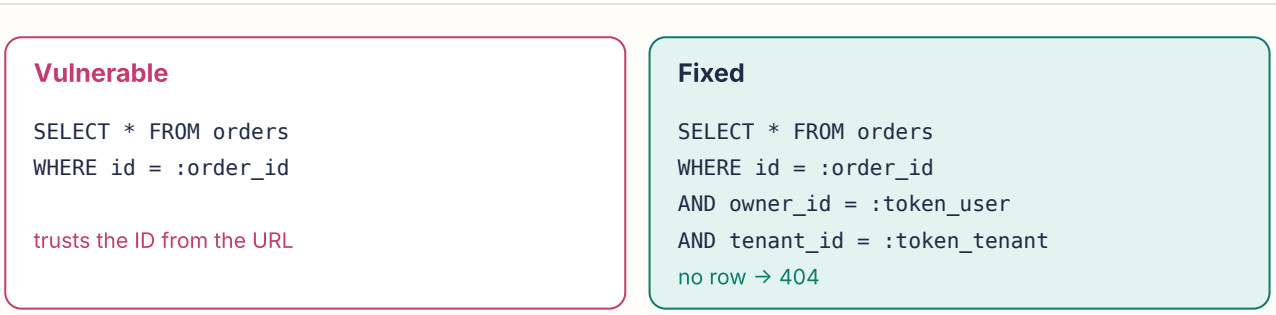


Figure 2. The ownership check belongs in the query. One missing `AND owner_id = …` is the whole vulnerability.

WORKED EXAMPLE: THE ORDER ENDPOINT IN FASTAPI

The route reads `user = Depends(current_user)`, which decodes the token and returns user 42 in tenant 7. The handler runs `SELECT … WHERE id = 1002 AND owner_id = 42 AND tenant_id = 7`. No row comes back, so it returns **404 Not Found**. A support agent with the `support` role takes a different, audited path that checks the role and the tenant, not ownership.

Test it: a test logs in as user A, creates an order, logs in as user B and asserts that `GET`, `PATCH` and `DELETE` on A's order all return 404.

4 Where each check lives

Why not do it all at the gateway? Because **the gateway doesn't know who owns order 1002**. Only the service and its database do. The gateway is great for authentication, token validation and scopes. Object-level checks must live in the service, close to the data.

Internal calls are not exempt. A service calling another service should **pass the user's identity along** (for example the original token, or a signed internal token carrying the user and tenant), instead of skipping checks just because the call came from inside the network.

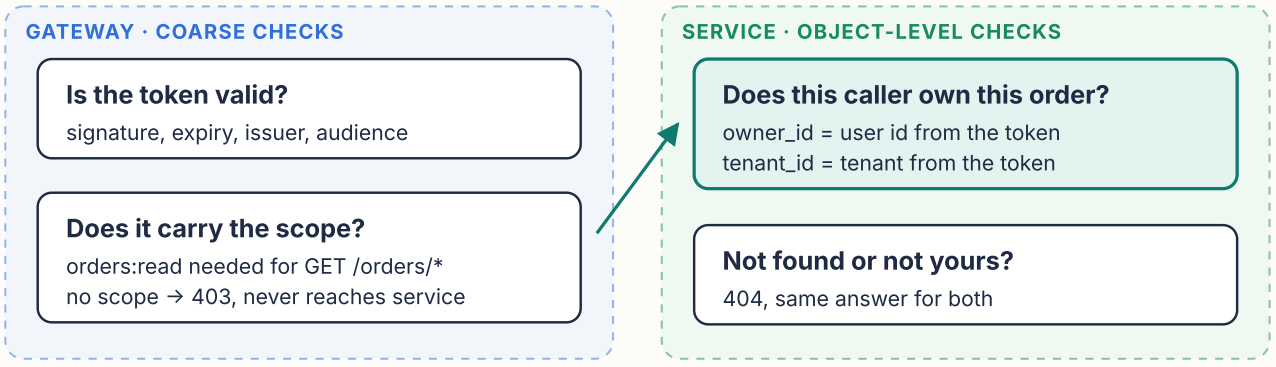


Figure 3. Coarse checks at the edge, fine-grained checks next to the data. A request must pass both.

5 As rules grow, use a model

- **RBAC** (role-based): permissions grouped into roles such as admin or support. Simple, but cannot say “only your own orders” on its own.
- **ABAC** (attribute-based): decide on attributes of the user, resource and context: region, department, owner.
- **ReBAC** (relationship-based), like Google's **Zanzibar**: decide on relationships, such as a document shared with a team the user belongs to.

Centralize the policy, for example in Open Policy Agent, but **enforce it in every service**. Random IDs like UUIDs make guessing harder and are good defense in depth, but they are **never the fix**: IDs leak through links, logs and shared screenshots.

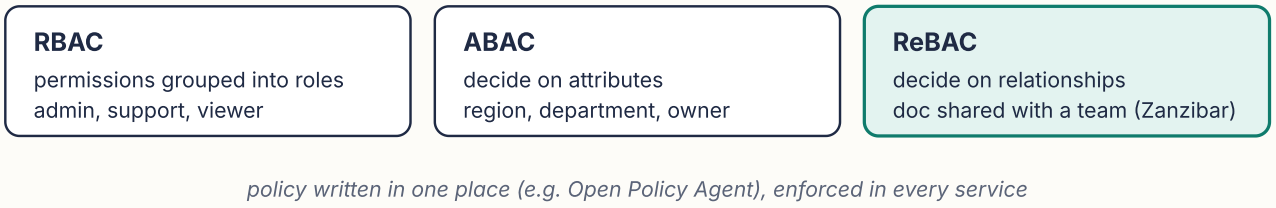


Figure 4. Three ways to express rules. Most real systems combine roles with an ownership or relationship check.

COMMON MISTAKES

- Treating a valid token, or the right scope, as permission to access any record.
- Taking the user or tenant ID from the request body instead of the token.
- Relying on the gateway for object-level checks it cannot make.
- Skipping checks on internal calls, so one compromised service can read everything.
- Switching to UUIDs and calling BOLA fixed.
- Returning 403 for other users' objects, which confirms they exist.

6 Key terms

Authorization: may this caller do this action on this resource

BOLA / IDOR: object ID trusted without an ownership check

Row-level security: database adds the tenant/owner filter itself

Scope: coarse permission on a token, e.g. orders:read

OWASP API Top 10: list of the top API security risks; BOLA is #1

RBAC / ABAC / ReBAC: roles / attributes / relationships

7 Interview questions

Q1. What is BOLA? BASIC

Broken Object Level Authorization: the API accepts an object ID from the client and returns or changes that object without checking the caller may access it, like changing order 1001 to 1002. It is #1 in the OWASP API Security Top 10, also called IDOR.

Q2. What is the difference between authentication and authorization? BASIC

Authentication proves identity (a valid token for user 42). Authorization decides whether that identity may perform this action on this specific resource. A request can pass the first and must still fail the second.

Q3. If a token has the orders:read scope, can it read any order? BASIC

Only if the service lets it. Scopes say what kind of action is allowed, not which records. The service must still check that the order belongs to the caller (and tenant).

Q4. Where should authorization checks live in a gateway + microservices design? SCENARIO

Token validity and scopes at the gateway, rejecting early. Object-level checks in the service at the data layer: every query filtered by owner and tenant from the token, on every request, including internal service-to-service calls.

Q5. You find /invoices/{id} returns any invoice to any logged-in user. How do you fix it and stop it recurring? SCENARIO

Add owner and tenant filters to the query and return 404 when no row matches. Then make it structural: a repository layer or row-level security that always adds the tenant filter, and cross-user tests (user B requests user A's invoice, expect 404) for every endpoint that takes an ID.

Q6. Why return 404 instead of 403 for another user's order? SCENARIO

403 confirms the order exists, which leaks information (an attacker can count orders or probe IDs). 404 gives the same answer for “doesn't exist” and “not yours”.

Q7. We switched order IDs to UUIDs. Is BOLA solved? TRICKY

No. UUIDs only make guessing hard. IDs still leak through URLs, emails, logs and other APIs, and anyone holding one can use it. UUIDs are defense in depth; the ownership check is the fix.

Q8. An internal reporting service calls Orders with a service account that can read everything. What is the risk? TRICKY

It bypasses per-user checks: a bug or compromise in the reporting service exposes every customer. Pass the end user's identity along (original or signed internal token) so Orders still applies owner and tenant rules; give service accounts only the narrow rights they need.

Next: SD-C-017 · API Gateway Explained backendroadmap.com

API Gateway Explained

what belongs in it, BFFs and avoiding a SPOF

Watch the 4-min video

In one line: an API gateway is one front door that handles the cross-cutting work every request needs (TLS, auth, rate limits, routing, observability); keep business logic out of it, and run it as redundant stateless instances so it never becomes a single point of failure.

1 Why one front door

Your company runs twenty Python microservices: orders, users, payments and more. Mobile apps and partners need to call them. Do you expose twenty hostnames, each doing its own TLS, authentication and rate limiting? No. You put **one front door** in front of them: an **API gateway**.

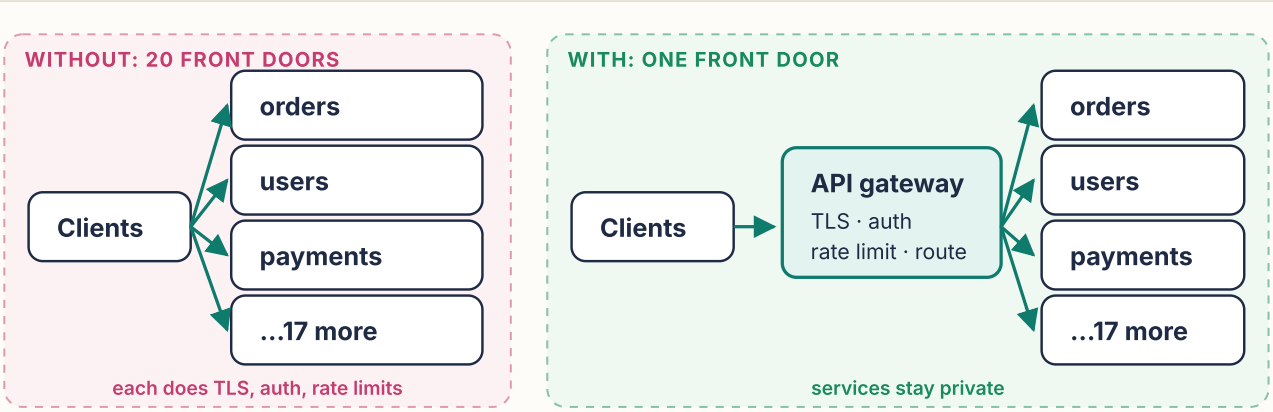


Figure 1. Without a gateway every service repeats the same edge work and is exposed to the internet. With one, the services stay private behind a single hostname.

2 What a gateway does

A gateway is a single entry point for the **cross-cutting concerns**: the jobs every request needs regardless of which service handles it.

- **Routing** by path or host: `/orders` goes to the orders service.
- **TLS termination**: certificates live in one place.
- **Authentication**: validate API keys or JWTs (signature, expiry, audience) and check scopes.
- **Rate limits and quotas** per key, user or partner.
- **Transformations**: add headers, or translate REST to gRPC.
- **Observability**: logs, metrics and traces for every call, in one consistent format.

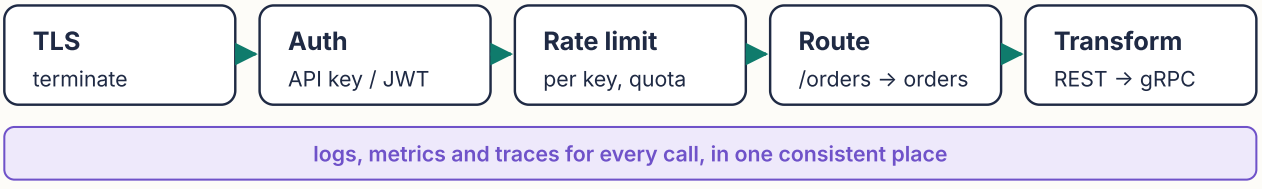


Figure 2. The typical policy chain. Each step is generic: it works the same for every API behind the gateway.

Aggregation and the backend for frontend (BFF)

Some gateways also **aggregate**: one client call fans out to several services and the results are merged, so a mobile screen needs a single round trip instead of five. This often lives in a separate **backend for frontend**, one per client type (mobile BFF, web BFF), rather than in the shared gateway. The BFF is owned by the team that builds that client, so it can change at their pace without touching the platform.

3 A concrete setup: Azure API Management

Clients call one public hostname. **Azure API Management** validates the JWT issued by **Entra ID**, applies a rate limit per subscription key, and routes to **FastAPI** services running privately, inside a virtual network or in Kubernetes behind an internal load balancer. Developers also get a portal with docs and keys. Similar options: **Kong**, **AWS API Gateway**, **Apigee**, and **Envoy**-based gateways.

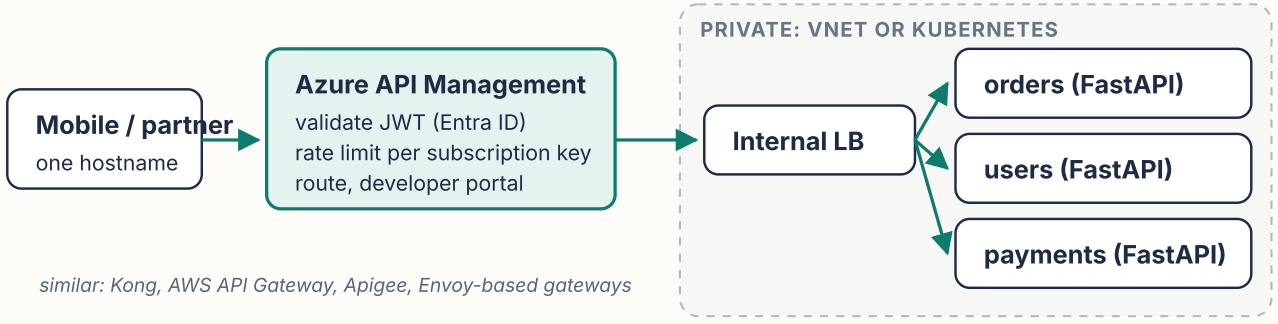


Figure 3. The gateway is the only public piece. The services have no public IPs; they trust only traffic that came through it.

4 What does not belong in the gateway

Business logic. Order validation, pricing rules, orchestrating business workflows. Also **object-level authorization**, because that needs the data (lesson SD-C-016): the gateway can check "has orders:read", not "owns order 1002".

Once teams push domain logic into the gateway, you get a **god gateway**: every change waits on the platform team, it becomes a release bottleneck, and one bad config breaks every API at once. Keep it **thin**, with generic policies only.

RULE OF THUMB

If a rule would be the same for any API (validate a token, limit to 100 requests/min, add a trace ID), it can go in the gateway. If it mentions a business noun (an order total, a discount, a stock level), it belongs in a service.

5 Don't let it become a SPOF

The gateway sits on **every** request, so if it goes down, everything goes down.

- Run **multiple stateless instances** across availability zones, behind a load balancer.
- Keep **shared state**, like rate-limit counters, in an **external store**, and decide in advance what happens when that store is down (allow traffic, or reject it).
- Use a **zone-redundant managed tier**, or multiple regions with **DNS failover**.
- Roll out **config changes gradually**, with canaries and quick rollback: a bad route or policy hits every API.
- Watch the gateway's **own latency and errors**: it adds a hop to every call.

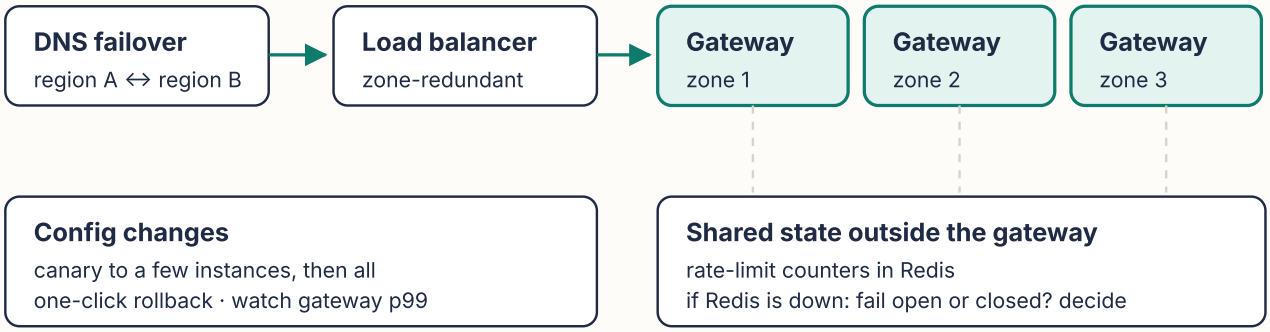


Figure 4. Redundant at every level: regions, zones and instances. The instances share no memory, so any one can serve any request.

Don't confuse it with its neighbours

Component	What it does	Traffic
Load balancer	spreads connections across servers; knows little about APIs	any
Reverse proxy (NGINX)	forwards requests; can do some gateway work (TLS, routing)	any
API gateway	API-aware policies: keys, JWTs, quotas, transforms, developer portal	north-south (outside → in)
Service mesh	mutual TLS, retries and telemetry between services	east-west (service ↔ service)

COMMON MISTAKES

- Putting pricing, validation or workflows in gateway policies (the god gateway).
- Trusting the gateway for object-level authorization.
- A single gateway instance, or rate-limit counters held in one instance's memory.
- Pushing a config change to all instances at once, with no rollback.
- Services still reachable from the internet, bypassing the gateway.

6 Key terms

API gateway: single entry point for cross-cutting API concerns

BFF: backend for frontend, one per client type

North-south: traffic from outside into the system

Cross-cutting concern: work every request needs: auth, TLS, limits

God gateway: gateway overloaded with business logic

East-west: traffic between internal services

7 Interview questions

Q1. What belongs in an API gateway, and what doesn't? BASIC

Cross-cutting concerns: routing, TLS, authentication, rate limiting, transformations and observability. Not business logic, domain validation or object-level authorization; those stay in the services.

Q2. API gateway vs load balancer vs service mesh? BASIC

A load balancer spreads connections and knows little about APIs. A gateway applies API-aware policies to north-south traffic from outside. A mesh handles east-west traffic between services with mutual TLS and retries.

Q3. What is a backend for frontend? BASIC

A thin API layer per client type (e.g. mobile, web) that aggregates several service calls into the shape that client needs, so one screen needs one round trip. It is owned by the client team, not the platform team.

Q4. How do you keep the gateway from being a single point of failure? SCENARIO

Several stateless instances across zones (or regions with DNS failover) behind a load balancer; shared state such as rate-limit counters in an external store with a clear fallback; gradual config rollout with canaries and rollback; and alerts on gateway latency and errors.

Q5. A team wants the gateway to reject orders over $10,000. Good idea? SCENARIO

No. That is a business rule; it needs domain data and changes with the business. Put it in the orders service. Otherwise every rule change waits on the platform team and one bad policy can break every API.

Q6. Rate-limit counters live in Redis and Redis goes down. What should the gateway do? SCENARIO

Decide in advance. Fail open (allow traffic, maybe with a coarse local per-instance limit) to keep the product up, or fail closed for expensive or abuse-prone endpoints. Never block on Redis with no timeout.

Q7. The gateway adds 15 ms to every call. Is that a problem? TRICKY

It depends on the SLO, but it is paid on every request, so measure it. Keep policies cheap (cache JWKS keys, avoid synchronous calls to other services in policies), keep the gateway in the same region as the services, and track its own p99 separately.

Q8. The gateway already validated the JWT. Can services skip all checks? TRICKY

They can trust the gateway for token validity if the network guarantees all traffic passes through it, but they must still do object-level authorization, which needs their data. Many teams also re-verify the token in the service as defense in depth.

Next: SD-C-018 · Retries, Exponential Backoff & Jitter backendroadmap.com

Retries, Backoff & Jitter

how retries cause outages, and how to stop them

Watch the 4-min video

In one line: retry only transient failures on safe operations, wait with exponential backoff plus random jitter, cap attempts and total time with a deadline and a retry budget, and retry at one layer only.

1 Retries are powerful, and dangerous

Your checkout service calls the inventory service, which briefly returns **503**. One retry would have fixed it. But a thousand clients retrying instantly, all at the same moment, can turn a **two-second blip into a twenty-minute outage**. Retries add load at exactly the moment the dependency has the least capacity to spare.

2 Decide what to retry

Retry **transient** failures: connection errors, timeouts, **502, 503 and 504**, plus **429** after waiting the `Retry-After` delay the server asked for. Don't retry **400, 401, 403, 404 or 422**: the request itself is wrong, so it will just fail again.

And only retry operations that are **safe to repeat**. A timeout doesn't tell you whether the server did the work. Reads are safe; so are idempotent writes (set status = shipped) and writes protected by an **idempotency key**. Blindly retrying "charge card" can charge twice.



Figure 1. The two questions before any retry: is the failure likely to go away, and is it safe to do the operation again?

3 Exponential backoff

Don't retry immediately. Start with a **base delay** and **double it** on each attempt: 100 ms, then 200, 400, 800, up to a **cap** such as 10 seconds. That gives the struggling service room to recover, instead of hammering it harder.

$$backoff(n) = \min(cap,\ base \times 2^{n-1}) \quad \cdot \quad n = 1,\ 2,\ 3\ \ldots$$

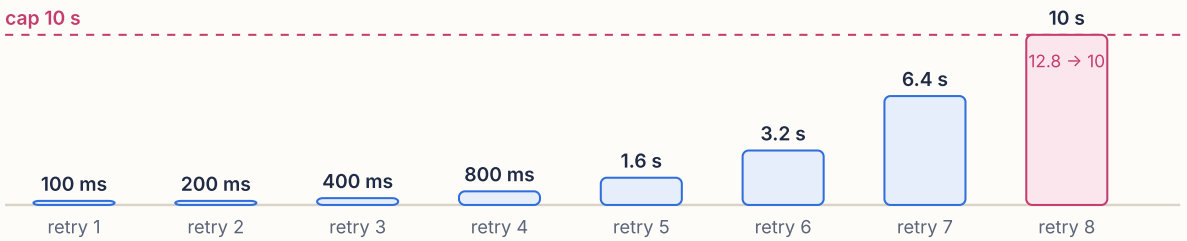


Figure 2. Each wait is twice the one before. The cap matters: without it, retry 11 would wait 100 ms × 2^{10} ≈ 102 s.

4 Jitter breaks up the waves

Backoff alone has a problem. If a thousand clients fail at the same moment, they all wait *exactly* 100 ms and retry together, in **synchronized waves**. Each wave can overload the service again. **Jitter** adds randomness so the retries spread out.

- **Full jitter**: sleep a random time between 0 and the backoff value. `sleep = random(0, backoff(n))`
- **Equal jitter**: keep half of the backoff, randomize the other half. `sleep = backoff(n)/2 + random(0, backoff(n)/2)`

AWS's analysis (the Architecture Blog post “Exponential Backoff and Jitter”) found **full jitter** spreads the load best, finishing the work with the fewest total calls.

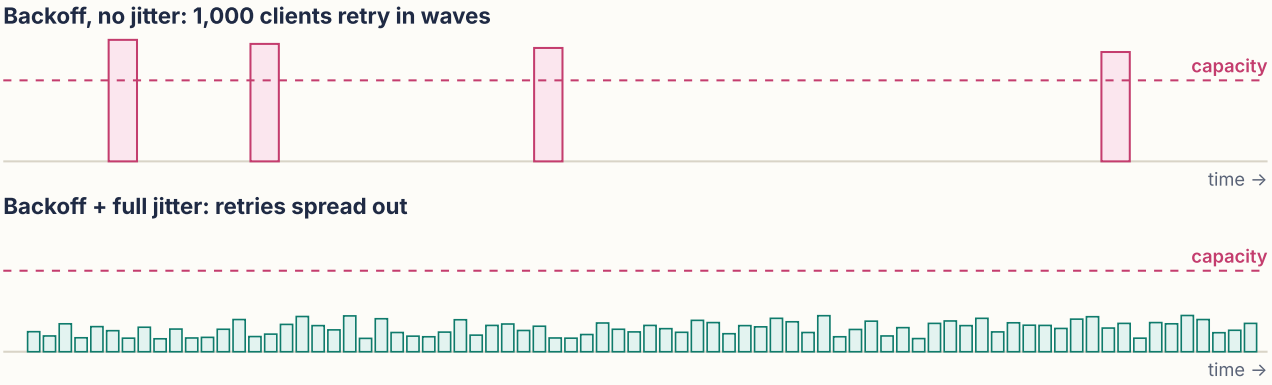


Figure 3. Top: identical delays line every client up, so each retry wave exceeds capacity. Bottom: random delays turn the waves into a steady trickle the service can absorb.

WORKED EXAMPLE: FULL JITTER DELAYS

Base 100 ms, cap 10 s, full jitter. Attempt 1 waits a random 0–100 ms, attempt 2 waits 0–200 ms, attempt 3 waits 0–400 ms. 1,000 clients that failed together now retry spread over a 400 ms window on the third try, about **2.5 per millisecond** instead of 1,000 in the same instant. By attempt 8 the window is 0–10 s (12.8 s capped to 10 s).

5 Set limits, and retry at one layer

- **Max attempts**, for example 3.
- **An overall deadline**, so retries never exceed what the caller is willing to wait (next lesson).
- **A retry budget**: allow retries only up to, say, **10% of normal traffic**, then fail fast. A budget stops retries from multiplying load when the system is weakest.

Watch out for **retry amplification**. If the gateway retries 3 times, the service retries 3 times and the database client retries 3 times, one user request can become **27 calls** to the database. Retry at **one layer**

only, usually the one closest to the user, and make inner layers fail fast. Pair retries with **circuit breakers**, which stop calling a dependency that is clearly down.

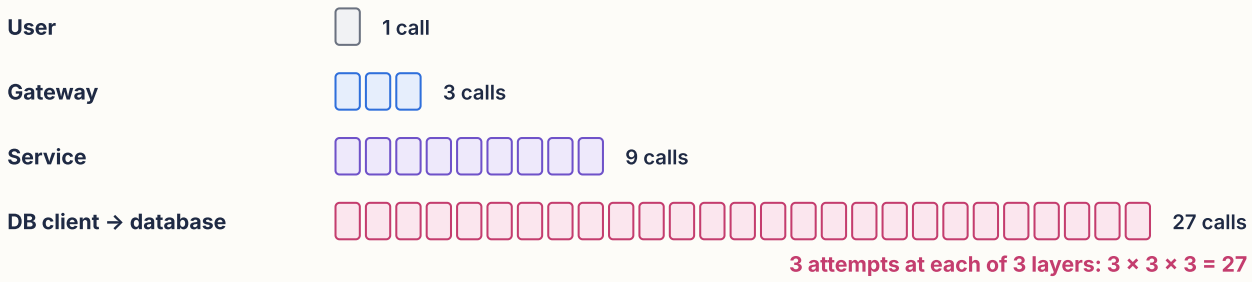


Figure 4. Retries multiply across layers. A database that is already slow receives 27 times the load from a single user click.

In Python: tenacity

```
from tenacity import (retry, stop_after_attempt, wait_random_exponential,
                      retry_if_exception_type, before_sleep_log)

@retry(stop=stop_after_attempt(3),
       wait=wait_random_exponential(multiplier=0.5, max=10),
       retry=retry_if_exception_type((Transient503, httpx.TimeoutException)),
       before_sleep=before_sleep_log(log, logging.WARNING))
def get_stock(sku): ...
```

Stop after three attempts, wait with random exponential backoff starting around half a second and capped at 10 seconds, retry only on the exceptions you consider transient, and **log every retry** so retry rates show up on your dashboards. A rising retry rate is often the first sign of trouble downstream.

COMMON MISTAKES

- Instant retries in a tight loop.
- Backoff with no jitter, so clients stay in lockstep.
- Retrying 4xx errors, or non-idempotent writes without an idempotency key.
- Retries at every layer (3 × 3 × 3 = 27).
- No attempt cap, no deadline, no budget.
- Ignoring `Retry-After` on 429 and 503.

6 Key terms

Transient failure: likely to succeed if tried again

Jitter: random part of the delay

Retry budget: retries capped at a share of traffic, e.g. 10%

Exponential backoff: delay doubles each attempt, up to a cap

Full jitter: random(0, backoff)

Retry amplification: retries multiply across layers

7 Interview questions

Q1. Which failures should you retry? BASIC

Transient ones: connection errors, timeouts, 502, 503, 504, and 429 after Retry-After. Not 400, 401, 403, 404 or 422. And only for operations that are safe to repeat: reads, idempotent writes or writes with an idempotency key.

Q2. Why add jitter to backoff? BASIC

Without it, clients that failed together retry together in synchronized waves that keep overloading the recovering service. Jitter spreads retries out randomly, smoothing the load and speeding recovery. Full jitter: random(0, backoff).

Q3. Write the backoff schedule for base 100 ms, cap 10 s. BASIC

100, 200, 400, 800 ms, 1.6, 3.2, 6.4 s, then 10 s (12.8 capped). With full jitter each is the upper bound of a random wait.

Q4. How can retries make an outage worse? SCENARIO

They multiply traffic exactly when a service is struggling: instant retries, no limits, and retries at several layers that amplify each other (3 layers × 3 attempts = 27 calls). Use backoff with jitter, attempt caps, retry budgets, one retry layer, and circuit breakers.

Q5. A payment call timed out. Should the client retry? SCENARIO

Only with an idempotency key. The timeout doesn't say whether the charge happened. With the same key, the server returns the first result instead of charging again; without one, check the payment status first.

Q6. Normal traffic is 2,000 RPS. What does a 10% retry budget mean? SCENARIO

At most about 200 retries per second across the fleet. Beyond that, calls fail fast instead of retrying, so a dependency outage adds at most 10% extra load instead of doubling or tripling it.

Q7. Gateway, service and DB client all retry 3 times. What do you change? TRICKY

Keep retries at one layer, usually the outermost or the one with the most context (often the edge or the calling service), and set the inner layers to fail fast with no retries. Add a deadline so retries stop when the user's budget is gone.

Q8. Retries vs circuit breaker: why use both? TRICKY

Retries handle brief, random failures. A circuit breaker notices a dependency is clearly down (e.g. 50% errors over 10 s) and stops calling it for a while, so retries don't keep hammering it. Retries fix blips; breakers protect during outages.

Next: SD-C-019 · API Timeouts & Deadline Propagation backendroadmap.com

API Timeouts & Deadlines

budgets, propagation, gRPC and cancellation

Watch the 4-min video

In one line: set each timeout a little above the dependency's measured p99 and inside your own SLO, then pass one absolute deadline down the whole call chain so every hop knows how much time is left, fails fast when it runs out, and cancels work nobody is waiting for.

1 The problem: independent timeouts

A user taps Search. The API gateway gives the whole request **2 seconds**. Behind it, the search service calls a ranking service, which calls a feature store. If each hop sets its own independent **10-second** timeout, the user gives up long before the backend does, and your servers keep working on an answer nobody will ever see. That wasted work holds threads, connections and database time that real users need.

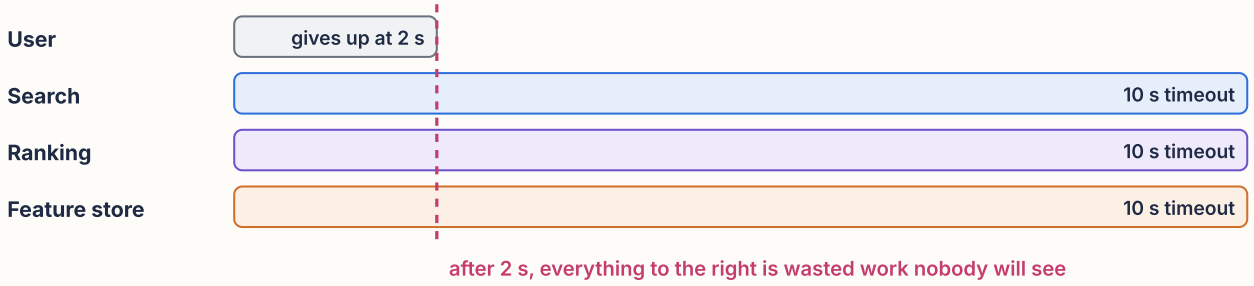


Figure 1. Each timeout looks reasonable on its own. Together they let the backend run five times longer than anyone waits.

2 Choosing a timeout value

Start from **data, not guesses**. Look at the dependency's latency distribution and set the timeout a little above its **p99**: about **1.5 to 2 times the p99**. Normal slow requests still succeed, but truly stuck ones fail fast.

Then check it against your own **SLO**. If your endpoint promises **300 ms at p99**, no dependency can be allowed 2 seconds. And if you plan to retry, all attempts have to fit in the budget too.

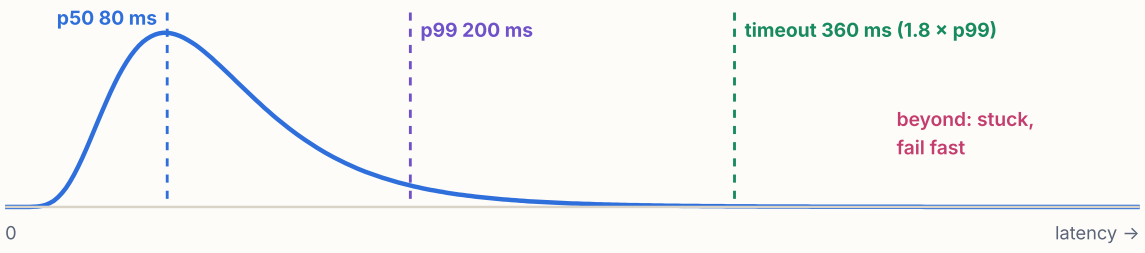


Figure 2. A timeout set from the distribution. Too tight (near p50) fails healthy requests; too loose (10 s) lets stuck calls pile up.

WORKED EXAMPLE: DOES A RETRY FIT?

Your endpoint's SLO is 300 ms at p99. It calls a pricing service with p99 = 80 ms. A timeout of 1.5 × 80 = **120 ms** leaves 180 ms. One retry after a short jittered pause (say 20 ms) costs at most 120 + 20 + 120 = **260 ms**, leaving 40 ms for your own work. Tight, but it fits.

If pricing's p99 were 150 ms, a 225 ms timeout leaves no room for a retry. Either drop the retry, or ask whether the SLO and the dependency are compatible at all.

3 Think in budgets

If the gateway's total budget is 2 seconds, every downstream call has to fit inside **what's left**. A child call should **never get a timeout longer than its parent's remaining time**. And retries are part of the same budget: three attempts at 1 s each cannot fit in 2 s.

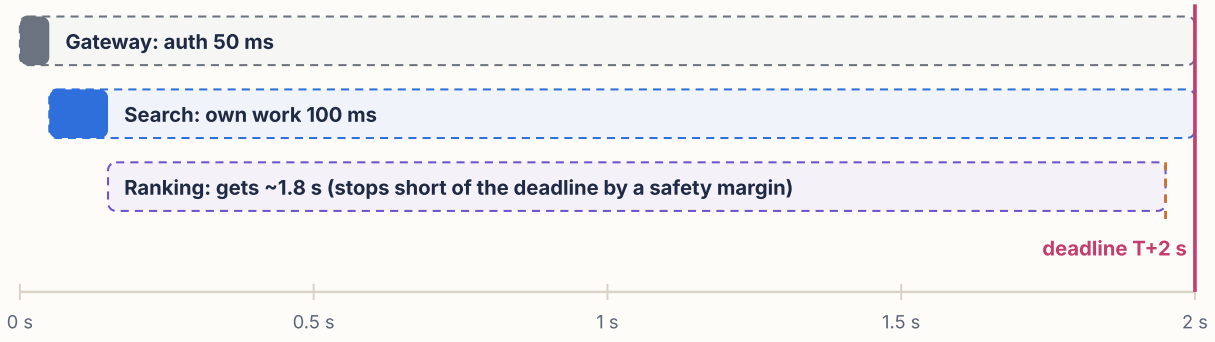
```
child timeout = min(own timeout, parent's remaining time − safety margin)
```

4 Deadline propagation

Instead of passing a *duration* ("you have 10 s"), pass an **absolute deadline**: the moment the answer becomes useless ("stop at 12:00:02.000"). Every hop then computes its remaining time the same way, however long it waited in queues.

- **gRPC** has this built in. The client sets a deadline, it travels with the call, and each server can ask how much time remains (`context.time_remaining()` in Python) and give a smaller deadline to its own calls. When it passes, the call fails with **DEADLINE_EXCEEDED**.
- **Over plain HTTP**, use a header, such as a request deadline or remaining milliseconds, and have every service honor it.

The example: the gateway receives a request at **T = 0** with a 2 s deadline. It spends **50 ms** on auth, then calls search with **1.95 s** remaining. Search spends **100 ms**, then calls ranking with about **1.8 s** remaining, minus a small safety margin. If ranking sees less than, say, **50 ms** left, it doesn't even start: it fails fast with deadline exceeded.



solid = time spent · dashed = time still allowed; each hop forwards the same absolute deadline

Figure 3. One deadline, many hops. Each service spends part of the budget and hands the rest down; nothing runs past T + 2 s.

5 Cancellation, and failing clearly

When the caller gives up, the work below should **stop**.

- In **gRPC**, cancellation propagates automatically to downstream calls.

- In **Python async** code, a timeout (`asyncio.timeout(...)`) cancels the task, so make sure your HTTP calls and database queries are awaited inside it and are cancelled with it.
- Set a **statement timeout** on the database (in Postgres, `SET statement_timeout = '1500ms'`), so an abandoned query doesn't keep burning a connection.



Figure 4. Cancellation should travel all the way down. The database timeout catches anything the layers above failed to cancel.

When time does run out, say so clearly: return **504 Gateway Timeout** at the edge, or **DEADLINE_EXCEEDED** in gRPC, not a vague 500, and **log which hop ran out of time**.

COMMON MISTAKES

- No timeout at all: many HTTP clients wait forever by default.
- Inner timeouts longer than outer ones.
- Retrying after the deadline has already passed.
- Timing out the caller but leaving the downstream query running.
- Returning a vague 500 instead of 504 / DEADLINE_EXCEEDED.

6 Key terms

p99: latency 99% of requests beat

Budget: total time a request may spend

DEADLINE_EXCEEDED: gRPC status when the deadline passes

SLO: target the endpoint promises, e.g. 300 ms p99

Deadline: absolute moment the answer becomes useless

statement_timeout: Postgres limit on one query's run time

7 Interview questions

Q1. How do you choose a timeout value? BASIC

From measured latency: about 1.5–2× the dependency's p99, within your own SLO, and within the caller's remaining budget, leaving room for any retries. A dependency with p99 = 200 ms gets roughly 300–400 ms.

Q2. What is deadline propagation? BASIC

Passing an absolute deadline down the call chain (a gRPC deadline or a deadline header) so every service knows how much time remains, gives its children less, fails fast when time is up, and cancels work nobody needs any more.

Q3. Why an absolute deadline instead of a fixed duration per hop? BASIC

A fixed 10 s per hop ignores time already spent upstream. With an absolute deadline every hop computes the true remaining time, so the chain as a whole never exceeds the user's budget.

Q4. Your endpoint's SLO is 300 ms p99 and a dependency has a 2 s timeout. What's wrong? SCENARIO

That one call alone can take 6× the whole SLO. Reduce it to fit, e.g. 1.5× the dependency's p99, and make sure it is also capped by the request's remaining time.

Q5. Gateway budget 2 s, auth takes 50 ms, search takes 100 ms. What does ranking get? SCENARIO

About 2.0 − 0.05 − 0.1 = 1.85 s, minus a safety margin for network and response time, so roughly 1.8 s. If ranking sees under ~50 ms left, it returns DEADLINE_EXCEEDED at once.

Q6. Users time out but database CPU stays high for minutes afterwards. Why? SCENARIO

The work isn't cancelled: queries keep running after the caller left. Propagate cancellation (gRPC, asyncio task cancellation), and set a statement_timeout so abandoned queries are killed.

Q7. Over HTTP between machines, what is risky about sending an absolute timestamp? TRICKY

Clock skew: if the servers' clocks differ by 300 ms, the budget is wrong by 300 ms. Sending the remaining milliseconds, which each receiver turns into a local deadline, avoids that; gRPC sends the remaining timeout on the wire for this reason.

Q8. What should a service return when the deadline is exceeded? TRICKY

504 Gateway Timeout at the HTTP edge or DEADLINE_EXCEEDED in gRPC, not 500, and log which hop ran out of time. Don't retry once the deadline has passed: nobody is waiting for the answer.

Next: SD-C-020 · Webhooks backendroadmap.com

Webhooks

reliable delivery, HMAC signing, retries and duplicates

Watch the 4-min video

In one line: a webhook is the provider calling *your* URL when an event happens; the sender queues events from an outbox, signs them with HMAC and retries for days, and the receiver verifies, records the event ID, answers 200 fast and processes idempotently without trusting order.

1 What a webhook is

A customer pays on your site. Your payment provider needs to tell your backend right away, without you polling every few seconds. So it calls *you*: an **HTTP POST to a URL you registered**. That's a webhook. The idea is simple; making delivery reliable is a classic system design question. It has two sides: the **sender** and the **receiver**.

2 The sender: outbox, queue, workers

When an event happens, like *payment succeeded*, **don't call the customer's URL inside the request** that created the payment. Write the event to an **outbox table in the same transaction** as the payment, then let a queue and a pool of **delivery workers** send it. The transaction guarantees the event is recorded if and only if the payment is. And a slow or broken customer endpoint never slows down your core API.

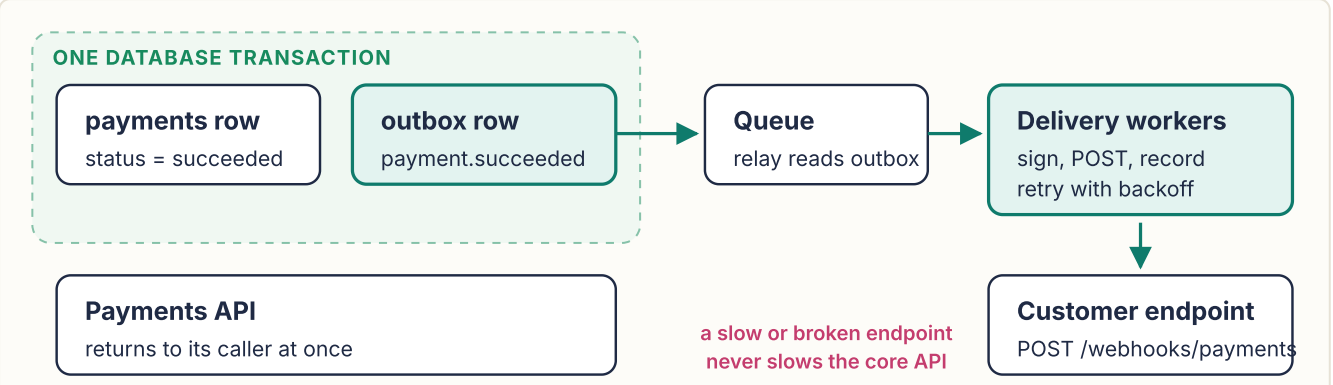


Figure 1. The payment and its event commit together. Delivery happens later, from workers, so the API's latency doesn't depend on anyone's endpoint.

Sign every delivery

The sender computes an **HMAC-SHA256** over a **timestamp and the raw body**, using a **secret unique to each endpoint**, and sends it in a signature header. Receivers verify it, so nobody can forge events. Including the timestamp in the signed string lets receivers reject old, replayed requests.

Put an **event ID**, an **event type**, a **creation time** and an **API version** in the payload, e.g. `{"id": "evt_9", "type": "payment.succeeded", "created": 1730700000, "api_version": "2026-01-01", "data": {…}}`.

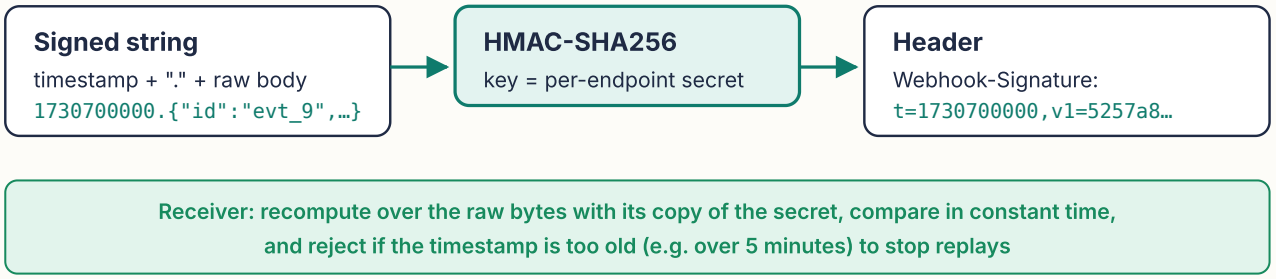


Figure 2. Signing and verifying. The receiver must hash the exact bytes it received; re-serialized JSON changes spacing and breaks the signature.

3 Retries, and visibility for customers

Endpoints fail, so retry. A delivery succeeds only on a **2xx response within a short timeout**, say 10 seconds. Anything else is retried with **exponential backoff and jitter**: for example after 1 minute, 5 minutes, 30 minutes, then hourly, for **up to three days**. After repeated failures, mark the endpoint as failing, **alert the owner**, and eventually **disable** it. Stripe, for example, retries for up to three days.

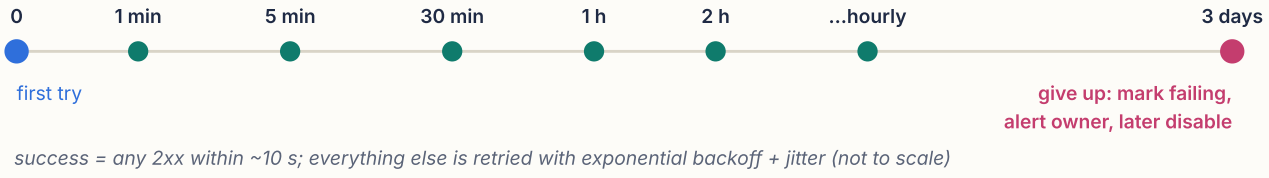


Figure 3. A typical schedule. Long gaps give a customer time to fix an outage; three days of attempts means a weekend outage is still recoverable.

- **Delivery log**: every attempt, its status code, latency and a response snippet.
- **Replay** any event from a dashboard or an API.
- **List endpoint** (e.g. `GET /events?since=…`) so customers can fetch events they missed after a long outage.
- **Verify a URL when it is registered**, with a test event or a challenge, and **block internal addresses** (private ranges such as 10.0.0.0/8, and cloud metadata at 169.254.169.254) to prevent **server-side request forgery** (SSRF).

4 The receiver: verify, store, ack, process

For example, a FastAPI endpoint. **Verify the signature on the raw body.** Then **store the event and return 200 quickly**, within a second or two. Do the real work **asynchronously**, in a background worker. If you do slow work before responding, the sender times out and retries, and you get even more duplicates.

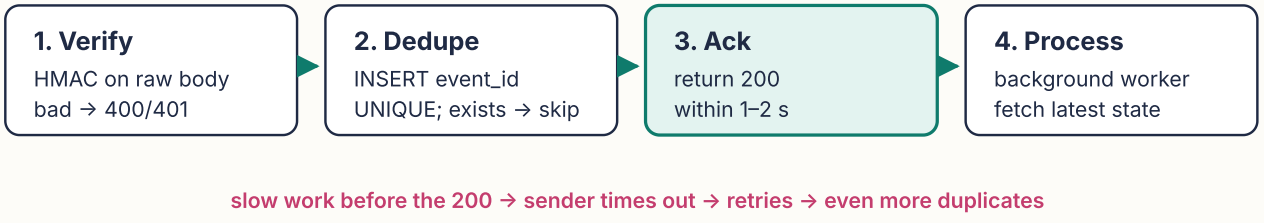


Figure 4. The receiver's job before answering is small and fast. Everything slow happens after the 200.

```
@app.post("/webhooks/payments")
async def hook(request: Request):
    raw = await request.body()                        # bytes, not parsed JSON
    t, sig = parse_header(request.headers["Webhook-Signature"])
    expected = hmac.new(SECRET, f"{t}.".encode() + raw, sha256).hexdigest()
    if not hmac.compare_digest(expected, sig) or time.time() - int(t) > 300:
        raise HTTPException(400)
    event = json.loads(raw)
    inserted = await db.insert_ignore("webhook_events", id=event["id"], body=raw)
    if inserted:
        await queue.enqueue("process_event", event["id"])
    return {"ok": True}                               # 200 either way
```

5 Duplicates and ordering

Delivery is **at least once**, so duplicates will happen: a retry after a timeout, a replay, a sender bug. Make processing **idempotent**: keep a table of processed event IDs with a **unique constraint**, and skip an event you've already handled. Return 200 for a duplicate too, or the sender will keep retrying it.

And **don't assume ordering**. A *payment refunded* event can arrive before *payment succeeded*. Use the event's timestamp or the object's version, or simply **fetch the latest state from the provider's API** before acting.

WORKED EXAMPLE: A REFUND THAT ARRIVES FIRST

At 10:00:00 the provider emits `payment.succeeded` (evt_1); at 10:00:03, `payment.refunded` (evt_2). Your endpoint was down for evt_1's first attempt, so evt_2 arrives first and evt_1 arrives on its 1-minute retry. If you blindly apply events in arrival order, the order ends up *paid*, wrong. Instead, the worker for each event calls `GET /payments/pay_7`, sees `status: refunded`, and writes that. Then evt_1 arrives a minute later, the worker fetches again and still sees refunded. Correct either way.

COMMON MISTAKES

- Calling the customer's URL synchronously inside the API request.
- Verifying the signature over re-serialized JSON instead of the raw bytes, or comparing with `==` instead of a constant-time compare.
- No timestamp check, so captured requests can be replayed.
- Doing slow work before returning 200.
- No event-ID dedupe; assuming events arrive in order.
- Letting customers register internal URLs (SSRF).

6 Key terms

Webhook: server-to-server HTTP POST on an event

HMAC-SHA256: keyed hash proving sender and integrity

Idempotent receiver: same event twice has one effect

Outbox: event row written in the same transaction

At-least-once: every event arrives, maybe more than once

SSRF: tricking a server into calling internal addresses

7 Interview questions

Q1. How do you design a reliable webhook delivery system? BASIC

Write events to an outbox in the same transaction, deliver asynchronously with a queue and workers, sign every payload with HMAC, retry failures with exponential backoff and jitter for days, and keep delivery logs with replay, endpoint verification and auto-disable for dead endpoints.

Q2. How should a receiver handle duplicates? BASIC

Verify the signature, record the event ID with a unique constraint, skip events already processed (still returning 200), respond fast, process asynchronously, and never rely on ordering.

Q3. Why sign webhooks with HMAC, and why include a timestamp? BASIC

Anyone can POST to a public URL. The HMAC proves the request came from someone holding the endpoint's secret and wasn't altered. Signing the timestamp lets the receiver reject requests older than ~5 minutes, so a captured request can't be replayed later.

Q4. Why not call the customer's URL inside the payment request? SCENARIO

Their endpoint might take 10 s or be down; your API would slow down or fail with it, and a crash between commit and call would lose the event. The outbox records the event atomically with the payment and workers deliver it independently.

Q5. A customer's endpoint is down for 6 hours. What happens? SCENARIO

Deliveries fail and are retried on a backoff schedule (1 min, 5 min, 30 min, then hourly); within the 3-day window they succeed once it's back. The customer can also replay from the dashboard or fetch missed events from a list endpoint.

Q6. The receiver's handler takes 15 s to process an order. What goes wrong? SCENARIO

The sender's ~10 s timeout fires, it counts the delivery as failed and retries, so the work runs again and again. Store the event, return 200 in 1–2 s, and process in a background worker.

Q7. Your signature check fails for valid events. Likely cause? TRICKY

Hashing parsed and re-serialized JSON instead of the raw request bytes (key order and spacing change), a framework that consumed the body, or the wrong per-endpoint secret. Hash the exact raw body.

Q8. payment.refunded arrives before payment.succeeded. How do you stay correct? TRICKY

Don't apply events by arrival order. Compare the event's timestamp or object version with what you stored, ignoring older ones, or fetch the latest state from the provider's API before acting.

Next: SD-C-021 · Async APIs for Long Jobs backendroadmap.com

Async APIs for Long Jobs

202 Accepted, job status, polling vs callbacks

Watch the 3-min video

In one line: for work that takes minutes, accept the request, enqueue the job and return `202 Accepted` with a job ID and a status URL; the job becomes a resource clients poll or get a webhook for.

1 Why you can't do it inside the request

A user uploads a **300-page PDF** to a document ingestion API. Parsing, chunking, embedding and indexing take about **ten minutes**. You can't hold an HTTP request open that long:

- **Load balancers and proxies time out** idle requests, often after 60 seconds by default.
- **Mobile connections drop** when the phone changes network or sleeps.
- **A single retry starts the whole job again**, doubling the cost and maybe the data.

So don't do the work in the request. **Validate** the input, **store** the file, **create a job record**, **put a message on a queue**, and immediately return `202 Accepted`. The body holds a job ID, and a `Location` header points to the status endpoint. The response takes milliseconds; the real work happens in the background.

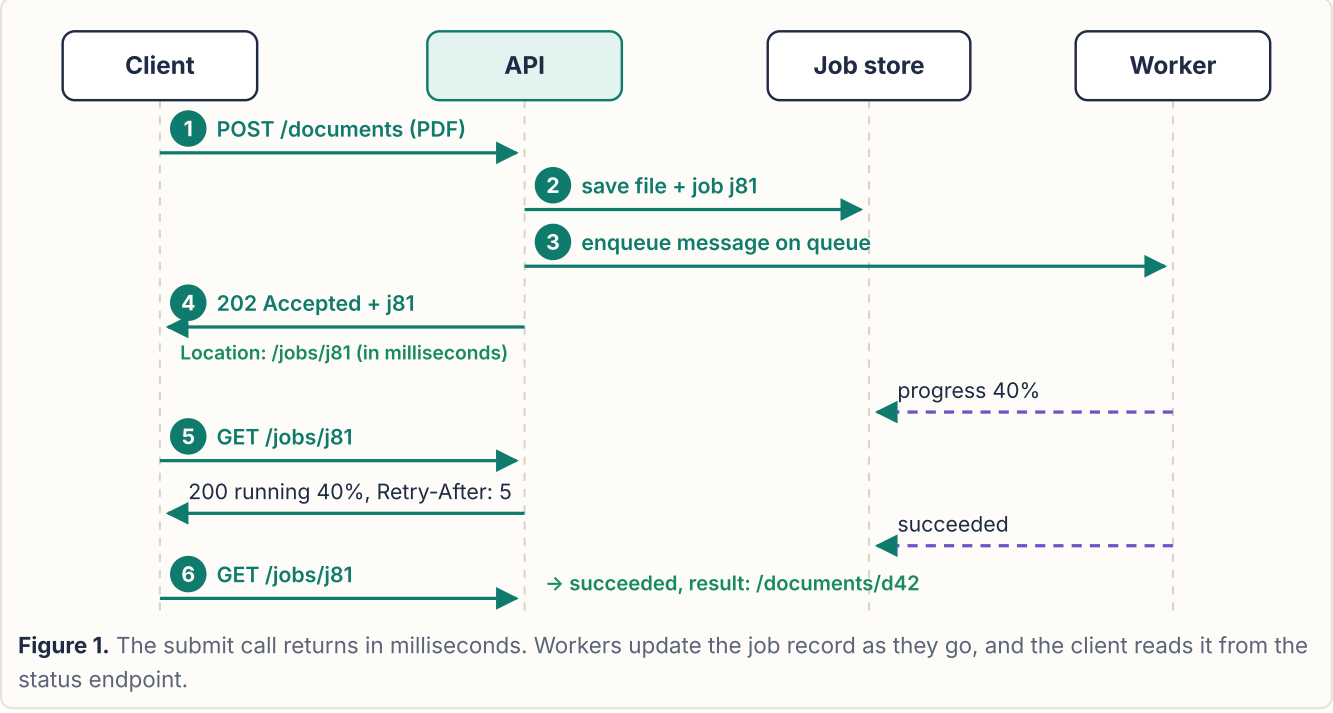


Figure 1. The submit call returns in milliseconds. Workers update the job record as they go, and the client reads it from the status endpoint.

2 The job is a resource: an operation

The job is a resource of its own, usually called an **operation**. It has an **ID**, a **status** (queued, running, succeeded, failed or cancelled), a **progress** percentage and **timestamps**. When it is done it holds either a **link to the result** or an **error**. Google's API guidelines call this a **long-running operation**; Azure uses the same pattern with an `Operation-Location` header.

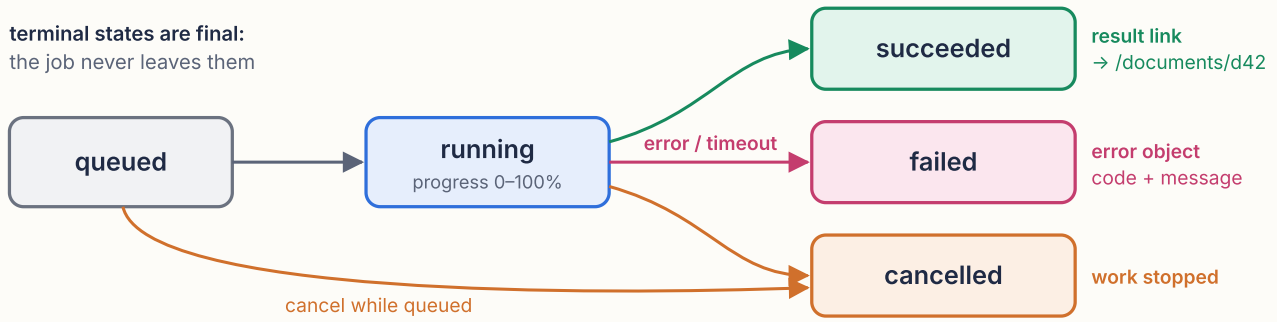


Figure 2. The job's lifecycle. Every job ends in exactly one terminal state, and a timeout turns a stuck job into a failed one.

```
HTTP/1.1 202 Accepted
Location: /jobs/j81
{"job_id": "j81", "status": "queued"}

GET /jobs/j81  →  200 OK
Retry-After: 5
{"id": "j81", "status": "running", "progress": 40,
 "created_at": "2026-11-04T10:00:00Z", "updated_at": "2026-11-04T10:04:10Z",
 "result": null, "error": null}
```

3 Polling or callbacks

Polling. The client sends `GET /jobs/j81` and gets the current status. A `Retry-After` header tells it how often to ask, say every **5 seconds**. Keep the status endpoint cheap: read from a fast store (a cache or a small indexed table), never recompute anything. When the job succeeds, the response links to the result.

Callbacks. The client passes a **callback URL** when it submits, and the server sends a **webhook** when the job finishes, **signed and retried** like any webhook. Browsers can subscribe with **server-sent events** or **WebSockets** for live progress. Many APIs offer both: callbacks for efficiency, polling as a reliable fallback when a webhook is lost.

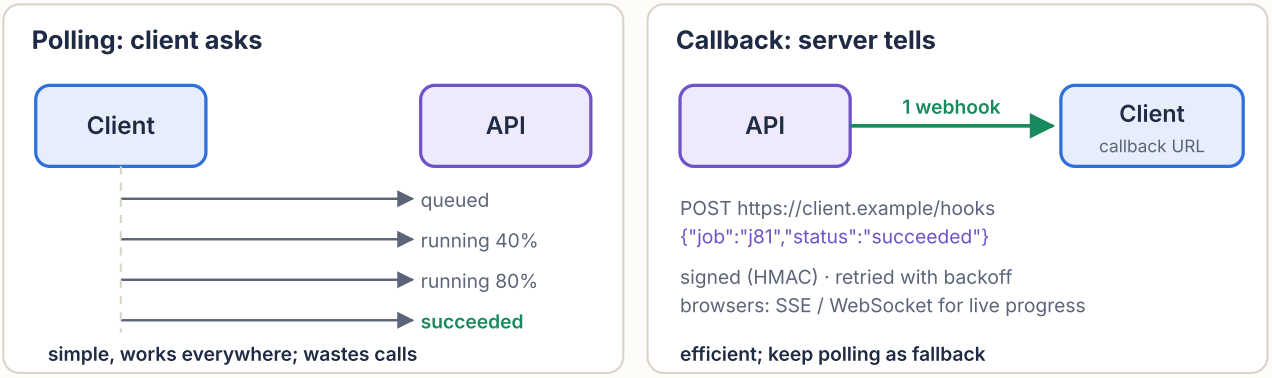


Figure 3. Polling needs nothing from the client but costs repeated calls. A callback is one message, but the client must run a reachable endpoint and verify signatures.

4 Make it robust

- **Idempotent submit.** The client sends an **idempotency key** with the upload. A retry with the same key returns the same job instead of creating a second one.
- **Checkpoint progress.** Workers save progress after each stage, so a crash resumes from the last checkpoint instead of starting over.

- **Cancellation.** Offer `POST /jobs/j81:cancel` or `DELETE /jobs/j81`; workers check the flag between stages.
- **Retention.** Expire old job records after a retention period (for example 7 or 30 days).
- **Timeouts.** Put a deadline on every job, so a stuck one is marked **failed** instead of "running" forever.

5 Example, end to end

WORKED EXAMPLE

`POST /documents` with the PDF returns **202** and job **j81**. The client polls the job: **queued**, then **running at 40%**, then **succeeded**, with a link to the new document. Meanwhile workers parse, chunk, embed and index, updating progress after each stage.

Cost of polling: **10,000** jobs in flight, each polled every 5 s, is 10,000 / 5 = **2,000 status requests per second**. A 10-minute job polled every 5 s makes about **120** calls; with a webhook it is **1** call. That is why the status read must be cheap, and why callbacks are worth offering.

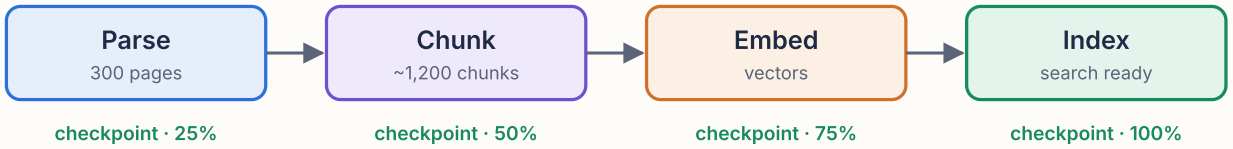


Figure 4. Each worker stage saves a checkpoint and updates progress. If the worker dies during embedding, a new one restarts from the end of chunking.

COMMON MISTAKES

- Doing ten minutes of work inside the request and raising every timeout to make it "work".
- Returning `200 OK` on submit: the work hasn't been done yet, so `202` is the honest answer.
- No idempotency key: a retried upload creates two jobs and two documents.
- A status endpoint that queries the heavy pipeline; polling traffic then slows the workers down.
- No timeout: jobs whose worker crashed show "running" forever.

6 Key terms

202 Accepted: request taken, work not done yet

Polling: client repeatedly GETs the status

Idempotency key: makes a retried submit return the same job

Location: header pointing to the status URL

Retry-After: header saying when to ask again

Checkpoint: saved progress a worker can resume from

Operation: resource that represents a long job

Callback / webhook: server calls the client when done

Retention: how long job records are kept

7 Interview questions

Q1. How do you design an API for a task that takes ten minutes? BASIC

Validate, store the input, create a job record, enqueue the work and return 202 Accepted with a job ID and a Location header for the status URL. Expose the job as an operation with status, progress, and a result or error. Clients poll (guided by Retry-After) or get a webhook.

Q2. Why 202 and not 200 or 201? BASIC

200 or 201 would claim the work is done or the resource exists. 202 says “accepted for processing, not finished”. The status endpoint later returns 200 with the job's state, and links to the created result.

Q3. What fields does a job status resource need? BASIC

ID, status (queued, running, succeeded, failed, cancelled), progress percentage, created and updated timestamps, and when finished either a result link or an error with code and message.

Q4. Mobile clients retry the upload when the network drops and you see duplicate jobs. Fix it. SCENARIO

Make submit idempotent: the client sends an idempotency key; the server stores key → job ID (e.g. for 24 h) and returns the existing job j81 on a retry instead of enqueuing a second one.

Q5. 50,000 jobs run at once and clients poll every second. What happens and what do you change? SCENARIO

That is 50,000 status reads per second. Send Retry-After: 5 to cut it to 10,000/s, serve status from a cache or small indexed table, and offer webhooks or SSE so most clients don't poll at all.

Q6. A worker crashes 8 minutes into a 10-minute job. What should happen? SCENARIO

The queue redelivers the message after the visibility timeout, and the new worker resumes from the last checkpoint (say after chunking) instead of re-parsing all 300 pages. A job timeout marks it failed if retries run out.

Q7. If you offer webhooks, why keep polling? TRICKY

Webhooks can be lost: the client endpoint is down, a firewall blocks it, or retries run out. Polling is the reliable fallback, and lets a client recover state after it restarts.

Q8. A job has shown “running” for 3 hours; normal jobs take 10 minutes. What design is missing? TRICKY

A per-job timeout or heartbeat. Workers should update a heartbeat; a sweeper marks jobs with no update for, say, 30 minutes as failed (or re-queues them), so clients get a final answer.

Next: SD-C-022 · API Contracts & OpenAPI backendroadmap.com

API Contracts & OpenAPI

contract-first, typed clients and Pact

Watch the 4-min video

In one line: an API contract (usually an OpenAPI file) says exactly what the API accepts and returns; generate docs, mocks and typed clients from it, diff it in CI, and use consumer-driven contract tests so a breaking change fails a build instead of a release.

1 The bug no test caught

The mobile team builds against your orders API. A backend engineer renames `total` to `total_amount`. All tests pass and the release goes out. That night the app crashes for every user. Nobody broke a test, because **nobody tested the contract**, the promise between the API and the code that calls it.

An **API contract** describes exactly what the API accepts and returns: **paths, methods, parameters, request and response schemas, status codes and authentication**. The standard format is **OpenAPI**, a YAML or JSON document (it was called Swagger before version 3). One file becomes the single source of truth for docs, clients, mocks and tests.

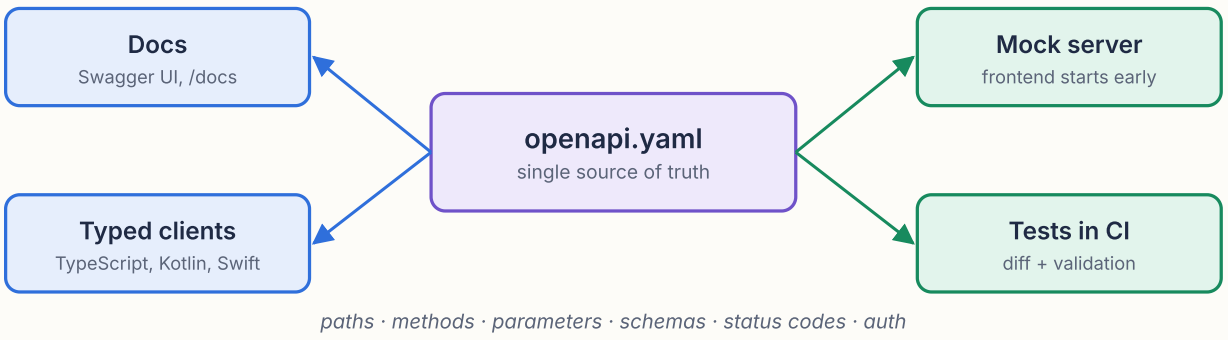


Figure 1. One reviewed file feeds everything else, so docs, clients, mocks and tests can't drift apart.

```
openapi: 3.1.0
info: {title: Orders API, version: 1.4.0}
paths:
  /orders/{id}:
    get:
      parameters: [{name: id, in: path, required: true, schema: {type: string}}]
      responses:
        '200': {content: {application/json: {schema: {$ref: '#/components/schemas/Order'}}}}
        '404': {description: Order not found}
components:
  schemas:
    Order: {type: object, required: [id, total],
            properties: {id: {type: string}, total: {type: number}}}
```

2 Code-first or contract-first

Code-first: write the code, then generate the spec. **FastAPI** does this automatically: your Pydantic models and route signatures become an OpenAPI document served at `/openapi.json`, with interactive docs at `/docs`. It is quick and always in sync with the code.

Contract-first flips it: design the spec **before** writing any code. Teams review the API shape in a pull request, frontend and backend agree, and both start in parallel, the frontend against a **mock server generated from the spec**. It is slower to start, but it catches design mistakes while they are cheap, and it suits **public and cross-team APIs**.

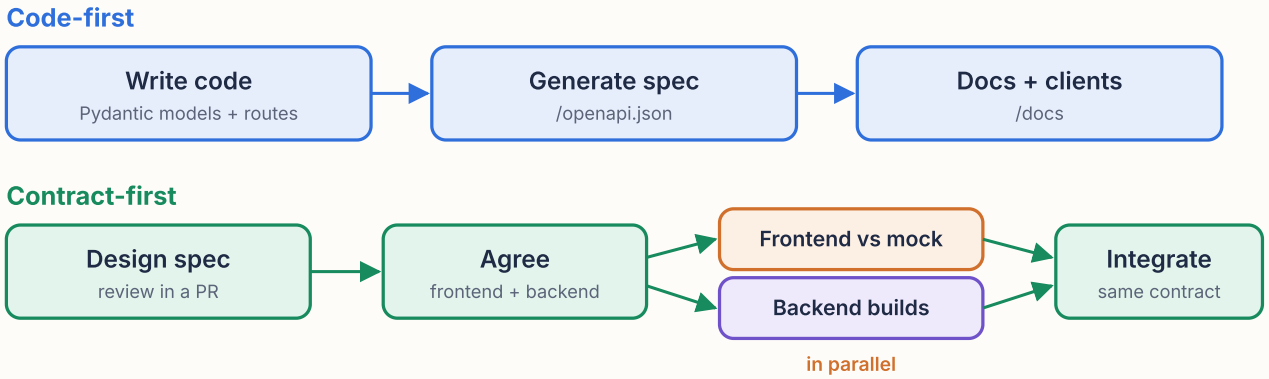


Figure 2. Code-first: the spec is an output of the code. Contract-first: the spec is the input, and both teams build against it at the same time.

	Code-first	Contract-first
Source of truth	The code	The spec file
Speed to start	Fast	Slower: design and review first
Parallel work	Frontend waits for a working backend	Frontend starts on a mock on day one
Best for	Internal APIs, one team owns both sides	Public, partner and cross-team APIs

3 Generate typed clients

Either way, **generate clients instead of hand-writing them**. From the FastAPI spec, a generator like **openapi-typescript** or **OpenAPI Generator** produces a typed TypeScript client. Now if the backend renames a field, the **frontend build fails at compile time** instead of crashing in production.

```
npx openapi-typescript https://api.example.com/openapi.json -o src/api/schema.d.ts

// after the rename, the old code no longer compiles:
const t: number = order.total;
//   error TS2339: Property 'total' does not exist on type 'Order'.
```

4 Catch breaking changes in CI

Diff the spec on every pull request. A tool like **oasdiff** compares the new OpenAPI file with the one running in production and **fails the build on breaking changes**: a removed field, a new required parameter, a changed type. **Additive changes pass**.

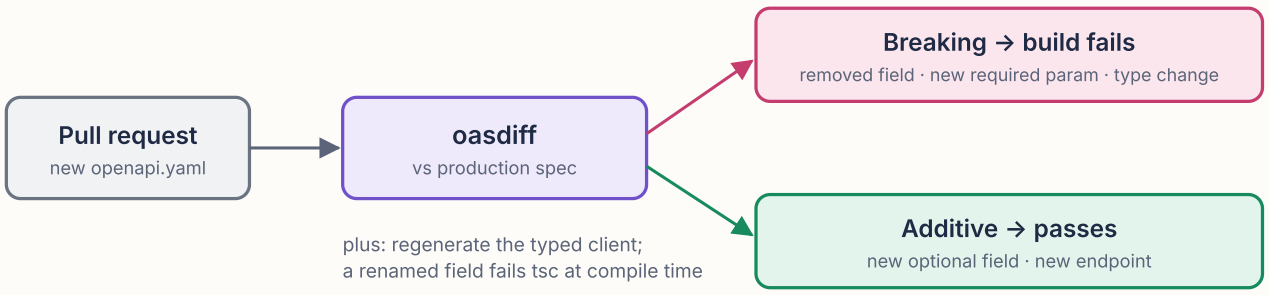


Figure 3. Spec diffing is cheap and runs in seconds. It catches the total → total_amount rename before merge, not at midnight.

5 Consumer-driven contract tests (Pact)

Between microservices, add **consumer-driven contract tests**. With **Pact**, each consumer writes tests that record what it actually uses from a provider: *this request, and these fields in the response*. Those contracts are published to a **broker**, and the provider's pipeline **verifies it still satisfies every consumer** before it deploys. A `can-i-deploy` check answers that automatically.

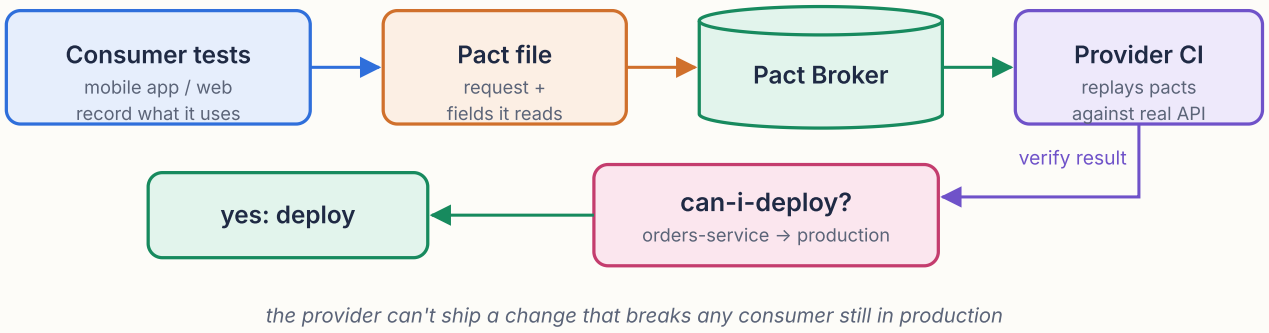


Figure 4. Each consumer states its real needs. The provider is free to change anything no consumer uses, and blocked only from changes that would break one.

WORKED EXAMPLE

The orders service has **3 consumers**: the mobile app, the web app and the billing service. The mobile pact says it reads `id`, `status` and `total`. A pull request renames `total` to `total_amount`.

- **oasdiff**: "property total removed from response" → build fails in about 10 seconds.
- **Pact verification**: the mobile pact fails; `can-i-deploy` says no.
- **Fix**: add `total_amount` and keep `total` (an additive change). Both checks pass. Remove `total` later, once no consumer reads it (next lesson).

COMMON MISTAKES

- Treating the spec as documentation written after the fact, so it drifts from the real API.
- Hand-writing client models that silently fall out of date.
- Testing only the provider's own behavior: unit tests pass while consumers break.
- Diffing against the main branch instead of what is **actually in production**.

6 Key terms

API contract: what an API accepts and returns

Contract-first: spec designed and agreed before code

oasdiff: diffs two OpenAPI files, flags breaking changes

OpenAPI: standard YAML/JSON format for HTTP API contracts

Mock server: fake API generated from the spec

Pact: consumer-driven contract testing tool

Code-first: spec generated from code

Typed client: generated code that fails to compile on mismatch

can-i-deploy: broker check: safe to release?

7 Interview questions

Q1. What is contract-first design? BASIC

Designing and agreeing on the API spec, usually OpenAPI, before writing the implementation, then generating mocks, stubs, clients and docs from it, so teams work in parallel against one source of truth.

Q2. What does an OpenAPI document contain? BASIC

Paths and methods, parameters, request and response schemas, status codes (200, 404 ...), and security schemes such as a bearer token. It is YAML or JSON and machine-readable.

Q3. Code-first or contract-first: when would you choose each? BASIC

Code-first (e.g. FastAPI generating /openapi.json) for internal APIs where one team owns both sides and speed matters. Contract-first for public, partner and cross-team APIs, where agreeing on the shape early is cheaper than changing it later.

Q4. How do you catch breaking changes before release? SCENARIO

Diff the OpenAPI spec against production in CI (oasdiff fails on removed fields, new required params, type changes); generate typed clients so mismatches fail at compile time; run consumer-driven contract tests like Pact, with can-i-deploy gating the provider's release.

Q5. A frontend team waits 3 weeks for the backend before they can start. How do you fix that? SCENARIO

Go contract-first: agree the OpenAPI spec in a PR in a day or two, generate a mock server from it, and let the frontend build against the mock while the backend implements the same spec.

Q6. You have 12 microservices calling each other. Why add Pact if you already have end-to-end tests? SCENARIO

End-to-end tests need all 12 services deployed together, are slow and flaky, and rarely cover every field. Pact tests each consumer–provider pair in isolation in seconds and tells you exactly which consumer a change breaks.

Q7. Your spec says total is required, but the server sometimes omits it. Tests pass. Why, and how do you catch it? TRICKY

Nothing checks responses against the spec. Add response validation in tests (or in staging), and contract tests that assert the fields consumers read are present. A spec that isn't enforced is just documentation.

Q8. Is adding a new required field to a request body breaking? To a response? TRICKY

Request: yes, old clients don't send it and now get 400. Response: usually safe for tolerant readers, since old clients ignore it; oasdiff treats new optional request fields and new response fields as non-breaking.

Next: SD-C-023 · API Backward Compatibility backendroadmap.com

API Backward Compatibility

breaking changes and expand-contract

Watch the 4-min video

In one line: every response must still work for every client version in the wild; make only additive changes, keep clients tolerant, never repurpose a field, and remove things with expand-contract.

1 Why it matters

Your mobile app has **five versions in the wild**, and some users never update. Every response your API sends must work for all of them: last week's app and the one from two years ago. **Backward compatibility** is the discipline that makes that possible: a new server must not break an old client.

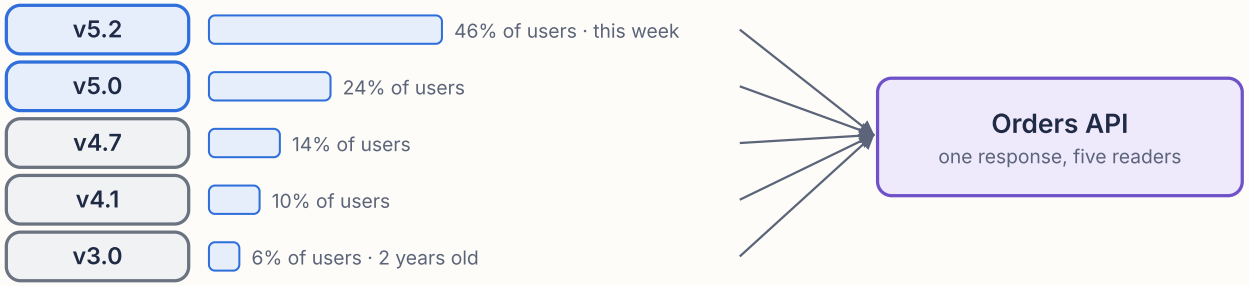


Figure 1. A typical spread of app versions. The server ships whenever it likes; the oldest client decides what it may still change.

2 What breaks, and what is safe

Breaking	Example
Removing or renaming a field	`total` → `total_amount`
Changing a type	`"id": 17` → `"id": "17"`
Changing meaning or units	`duration` in seconds → milliseconds
Making an optional input required	old clients don't send it, so they get `400`
Tightening validation	name max length 100 → 50
Adding an enum value old clients can't handle	`status: "on_hold"` crashes a strict switch

Safe: adding optional response fields, new endpoints, new optional parameters, and loosening validation (accepting more than before).

WORKED EXAMPLE

You add an optional `delivery_eta` field to the order response. Old apps simply ignore it; new apps use it. On the request side, a new optional field needs a **sensible default**, so an old app that never sends it still gets the old behavior: a new `gift_wrap` flag defaults to `false`, and the order is placed exactly as before.

3 Tolerant readers

Additive changes only work if clients are **tolerant readers**. A tolerant reader:

- **ignores unknown fields**;
- **doesn't depend on field order**;
- **handles unknown enum values gracefully**, for example as “other”.

So configure JSON parsers not to fail on unknown properties, and give every `switch` on an enum a **default branch**. The old rule (Postel's law): *be conservative in what you send, and liberal in what you accept*.

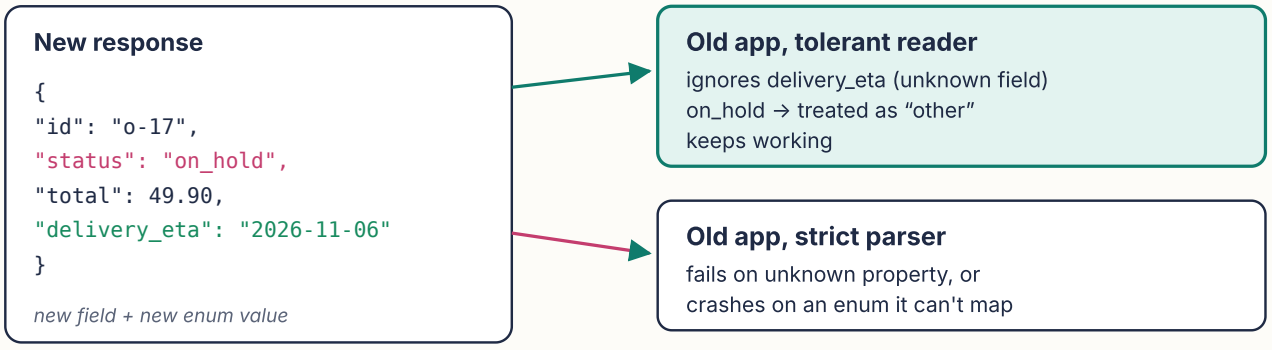


Figure 2. The same response, two old clients. The tolerant one keeps working; the strict one turns a harmless server change into a crash.

```
// Kotlin + kotlinx.serialization: don't fail on unknown fields
val json = Json { ignoreUnknownKeys = true }

// Map unknown enum values to a fallback instead of crashing
when (order.status) {
  "paid"    -> showPaid()
  "shipped" -> showShipped()
  else      -> showGeneric()   // on_hold, and anything added later
}
```

4 Never repurpose a field

If `status` used to mean **payment** status and you start using it for **shipping** status, old clients make **wrong decisions silently**, which is worse than crashing: a crash gets noticed, a wrong “paid” badge does not. **Add a new field with a new name** instead (`shipping_status`).

The same goes for units: if you need milliseconds, add `duration_ms` and keep `duration` in seconds. Putting the unit in the name makes the meaning impossible to misread.

5 Removing a field safely: expand and contract

1. **Expand.** Add the replacement field and send both.
2. **Migrate.** Move clients to the new field and mark the old one `deprecated: true` in the OpenAPI spec.
3. **Measure.** Log which clients still read or send the old field, by app version.
4. **Contract.** Only when usage is near zero *and* the minimum supported app version has moved past it, remove the field, or save the removal for the next major version.

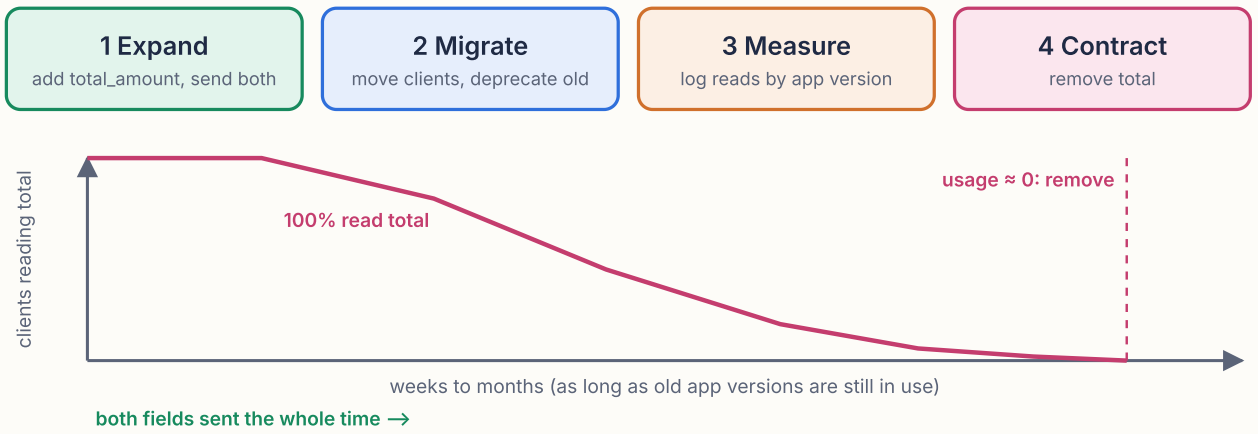


Figure 3. Expand-contract replaces one risky change with several safe ones. The removal waits for data, not for a date someone guessed.

6 Mobile: you can't force updates

- **Send the app version in a header** (for example `X-App-Version: 4.1.0`) on every request, so logs show who uses what.
- **Keep a minimum supported version**, and show a **force-update screen** for apps below it.
- **Server-side feature flags** make new behavior opt-in per app version: only 5.2+ gets the new checkout flow.
- **Keep contract tests for every version you still support**, so a change that breaks v4.1 fails CI.

COMMON MISTAKES

- Renaming a field "because the old name was ugly" in a minor release.
- Reusing a field or enum value for a new meaning.
- Strict JSON parsing in clients, so any new field crashes old apps.
- Removing a deprecated field on a date, without checking who still reads it.
- Changing units silently (seconds → milliseconds) without a new field name.

7 Key terms

Backward compatible: new server still works for old clients

Breaking change: a change that can break an existing client

Additive change: adds without removing or altering

Tolerant reader: client that ignores what it doesn't know

Expand-contract: add new, migrate, measure, then remove old

Deprecation: marking a field as going away

Minimum supported version: oldest app version still served

Feature flag: server switch to enable behavior per version

Postel's law: strict in sending, liberal in accepting

8 Interview questions

Q1. Which API changes are breaking? BASIC

Removing or renaming fields; changing types, meanings or units; making inputs required; tightening validation; and adding enum values old clients can't handle. Adding optional fields and endpoints is safe, as long as clients are tolerant readers.

Q2. How do you remove a field safely? BASIC

Expand and contract: add the replacement and send both, deprecate the old field in the spec, measure who still uses it by app version, and remove it only when usage is near zero or in a new major version.

Q3. What is a tolerant reader? BASIC

A client that ignores unknown fields, doesn't depend on field order, and maps unknown enum values to a fallback like “other”. It is what makes additive server changes safe.

Q4. You must change duration from seconds to milliseconds. How? SCENARIO

Don't change it. Add duration_ms, keep duration in seconds and send both, migrate clients, measure, and remove duration only when no client reads it. Same value, two names: 90 and 90000.

Q5. Logs show 3% of requests still come from app v3.0, which reads the old field. What do you do? SCENARIO

Keep sending the field for now. Decide whether v3.0 is below your minimum supported version; if so, show those users a force-update screen, wait until usage falls near 0%, then remove the field.

Q6. You add status = "on_hold" and old Android apps crash. What went wrong and what is the fix now? SCENARIO

The old parser had no default branch for unknown enum values. Short term, stop sending on_hold to old app versions (map it to an existing value by X-App-Version, or behind a flag). Long term, ship tolerant enum handling and contract-test old versions.

Q7. Why is repurposing a field worse than removing it? TRICKY

Removing breaks loudly: a crash or a missing value you'll see in error logs. Repurposing breaks silently: old clients read “status: paid” as payment status when it now means shipping, and make wrong decisions nobody notices.

Q8. Is loosening validation always safe? TRICKY

For old clients, yes: everything they sent before is still accepted. But check downstream: if you raise a name limit from 50 to 200 characters, the database column, other services and old clients that display the value must handle 200 too.

Next: SD-E-001 · Relational Model & SQL Basics backendroadmap.com